Smarter Regular Expression Matching

From Theory to Tools

Ilian Kohl

October 24, 2026

Contents

1 Introduction **4**
- 1.1 Motivation . . . 4
- 1.2 Problem Statement . . . 5
- 1.3 Contributions . . . 5
- 1.4 Thesis Outline . . . 6

2 Regular Expressions **7**
- 2.1 Syntax . . . 7
- 2.2 Semantics . . . 8
- 2.3 Rust Representation . . . 9

3 Derivatives and Partial Derivatives **10**
- 3.1 Naive Matching . . . 10
- 3.2 Brzozowski Derivatives . . . 12
- 3.3 Antimirov Partial Derivatives . . . 18

4 Matching and Parsing **22**
- 4.1 Matching: A Boolean Answer . . . 22
- 4.2 Parsing: Parse Trees as Matching Evidence . . . 22

5 Derivative-Based Parser **25**
- 5.1 The Standard Derivative-Based Parser . . . 25
  - 5.1.1 Building the Empty-Word Tree . . . 26
  - 5.1.2 Reinserting Characters via Injection . . . 27
  - 5.1.3 Parser . . . 29
  - 5.1.4 Worked Example: Applying the Standard Parser . . . 30
- 5.2 The Bit-Coded Derivative-Based Parser . . . 31
  - 5.2.1 Annotated Expressions . . . 32
  - 5.2.2 Fusing Bit-Lists . . . 33
  - 5.2.3 Internalizing Expressions . . . 34
  - 5.2.4 Extracting Bits . . . 35
  - 5.2.5 Annotated Derivatives . . . 36
  - 5.2.6 Decoding Bits . . . 38
  - 5.2.7 Parser . . . 40
  - 5.2.8 Worked Example: Applying the Bitcoded Parser . . . 41

6 Partial-Derivative-Based Parser **42**
- 6.1 The Standard Partial-Derivative Parser . . . 42

6.1.1 Partial Derivatives with Injections . . . 43
6.1.2 State Sets and Selection . . . 47
6.1.3 Parser . . . 49
6.1.4 Worked Example: Standard Partial-Derivative Parser . . . 49
6.2 The Bit-Coded Partial-Derivative Parser . . . 51
6.2.1 Bit-Coded Partial Derivatives . . . 51
6.2.2 Extracting Bits . . . 54
6.2.3 State Sets and Selection . . . 55
6.2.4 Parser . . . 56
6.2.5 Worked Example: Ambiguous Expression . . . 56

7 External versus Internal Syntax 58
7.1 The Interface and Its Design Choices . . . 58
7.2 The External AST . . . 59
7.3 Parsing: String to ExtPat . . . 60
7.3.1 Precedence and associativity . . . 60
7.3.2 Accepted surface syntax . . . 61
7.3.3 Rejected surface syntax . . . 61
7.4 PCRE Flags . . . 62
7.5 Translation: ExtPat to Regex . . . 63
7.5.1 Character classes and the working alphabet . . . 63
7.5.2 Bounds and unrolling . . . 63
7.5.3 Anchor recovery . . . 63
7.6 Translation Size . . . 64
7.7 Summary and Bridge to Evaluation . . . 64

8 Evaluation 66
8.1 Dataset Preparation . . . 66
8.1.1 Source selection . . . 66
8.1.2 Extraction . . . 67
8.1.3 Input generation . . . 68
8.1.4 A worked example through the pipeline . . . 69
8.1.5 Corpus overview . . . 70
8.1.6 Match intent . . . 71
8.1.7 Rejection causes . . . 71
8.1.8 Anchor shape distribution . . . 72
8.1.9 Prepared cases . . . 73
8.2 Parser Agreement . . . 73
8.2.1 Method . . . 73
8.2.2 Results . . . 74
8.2.3 Discussion . . . 74
8.3 Performance Analysis . . . 74
8.3.1 Cost drivers . . . 75
8.3.2 Timing . . . 75
8.3.3 Memory . . . 77
8.4 Discussion of Results . . . 79
8.4.1 Cost of the standard construction . . . 79
8.4.2 Cost of bit-coding . . . 79
8.4.3 Cost of partial derivatives . . . 79

8.4.4 When each parser is appropriate 80
8.4.5 Threats to validity 80
8.4.6 Summary 80

9 Related Work and Conclusion **82**
9.1 Related Work 82
9.2 Conclusion 82

1 Introduction

1.1 Motivation

Regular expressions are a fundamental formalism for describing regular languages and are among the most widely used abstractions in computer science. They underpin applications including lexical analysis, input validation, log processing, search tools, and network security. Across these applications, the problem is regular expression matching. Given a regular expression and an input string, determine whether the string belongs to the language denoted by the expression. Formally, this is the *membership problem*.

For example, the expression `(ab|a)*` matches `"aba"`, as it decomposes into `"ab"` followed by `"a"`, but does not match `"abb"`, since no sequence of subexpressions can produce it.

For many applications, a Boolean answer is enough. An input validator may only need to accept or reject a password or URL. Others require more. For instance, a lexer must determine how input decomposes into tokens. Capturing groups, schema validation, and information extraction require identifying which input parts correspond to parts of the expression. Recognition alone is not enough. The matcher must also recover the *structure*.

Efficiently solving these problems remains an active research area. The choice of algorithm influences performance, memory consumption, and security. Once matching becomes parsing, the way in which match structure is recovered also influences correctness and raises the question of how to handle ambiguity when multiple matches are possible.

Existing approaches include backtracking and finite-automata-based techniques, such as nondeterministic (NFA) and deterministic (DFA) automata. These approaches have different trade-offs in implementation complexity, memory consumption, and worst-case performance. Backtracking engines are flexible and widely used, but may exhibit exponential running time on constructed inputs, leading to regular expression denial-of-service (ReDoS) vulnerabilities [Cox, 2007]. Automata-based approaches provide stronger worst-case guarantees but may require preprocessing or suffer from state-space explosion.

Derivative-based algorithms provide an alternative by transforming the regular expression as input symbols are consumed. Each derivative represents the language remaining after an input symbol has been consumed. By maintaining a correspondence between an expression and its successive residuals, derivative-based methods provide an elegant foundation for regular expression matching and, with suitable extensions, structured parsing.

However, derivative-based techniques are less common in practical regular expression tooling. Their use for practical matching, parsing, and diagnostic tools motivates this thesis: "From Theory to Tools".

1.2 Problem Statement

Derivative-based approaches provide a principled foundation for regular expression matching and parsing. Brzozowski introduced regular expression derivatives as a way of transforming a regular expression after consuming an input symbol [Brzozowski, 1964]. Antimirov later introduced partial derivatives, which represent sets of residual expressions and are closely related to nondeterministic automata [Antimirov, 1996].

Building on Brzozowski's derivatives, Sulzmann and Lu developed a derivative-based algorithm for POSIX regular expression parsing [Sulzmann and Lu, 2014]. Their work also presents a bit-coded variant that represents parse information as a compact bit sequence, which can be decoded after a successful match rather than maintaining an explicit parse tree throughout the computation. A Haskell implementation of the POSIX parser is available in Sulzmann's *Programming Paradigms* teaching material [Sulzmann, 2026].

Sulzmann and Lu also studied partial derivatives in the context of regular expression sub-matching [Sulzmann and Lu, 2012]. While sub-matching is not the focus of this thesis, their work provides relevant background for the partial-derivative approach. The *Programming Paradigms* teaching material also presents a bit-coded partial-derivative-based parser in Haskell.

These constructions have primarily been presented in a theoretical and functional programming setting, leaving open practical questions about execution time, memory consumption, and data representation.

This thesis investigates these questions through implementations in Rust. As a systems programming language, Rust provides fine-grained control over memory allocation and data representation, allowing the derivative-based and partial-derivative-based parsers, including their bit-coded variants, to be evaluated in practice.

The implementation also provides traced versions of the parsing algorithms that record intermediate derivative computations during matching. These traces show the transformations performed by the algorithms and help explain successful and unsuccessful matches.

1.3 Contributions

The following contributions are being made:

- **Rust implementation of derivative-based parsers.** The thesis implements derivative-based and partial-derivative-based parsing constructions in Rust, including direct and bit-coded variants. The implementations are based on the constructions described in the literature and the corresponding Haskell material, resulting in four parser variants in total.
- **Empirical evaluation of the parser variants.** The four implementations are evaluated with respect to execution time and memory consumption using synthetic benchmarks and a corpus of real-world regular expressions from Snort/Suricata, SpamAssassin, and RegexLib.

1.4 Thesis Outline

The remainder of this thesis follows the parsing constructions from their formal concepts through their Rust implementation to their evaluation. Theory and implementation are presented together.

- **Chapter 2 - Regular Expressions.** This chapter introduces the syntax and semantics of regular expressions and the Rust representation used throughout the implementation.
- **Chapter 3 - Derivatives.** This chapter introduces Brzozowski's derivatives and Antimirov's partial derivatives, together with their basic matching algorithms and Rust implementations.
- **Chapter 4 - Matching vs. Parsing.** This chapter distinguishes regular expression matching from parsing and introduces the disambiguation problem.
- **Chapter 5 - Derivative Parser.** This chapter presents the derivative-based POSIX parsing approach of Sulzmann and Lu, along with a Rust implementation developed in this work, in both direct and bit-coded variants.
- **Chapter 6 - Partial-Derivative Parser.** This chapter presents the partial-derivative-based parsing approach of Sulzmann and Lu, along with a Rust implementation developed in this work, in both direct and bit-coded variants.
- **Chapter 7 - External versus Internal Syntax.** This chapter describes the parsing of regular expression strings into an external AST and their translation into the internal representation used by the parsers.
- **Chapter 8 - Evaluation.** This chapter evaluates the four parser variants with respect to execution time, memory consumption, and correctness. The evaluation uses synthetic benchmarks and a corpus of real-world regular expressions. It also examines optimization challenges and the insights gained from the evaluation.
- **Chapter 9 - Conclusion.** Finally, this chapter summarizes the findings of the thesis and discusses the practical implications of the results and directions for future work.

2 Regular Expressions

2.1 Syntax

Let Σ be an alphabet. Regular expressions over Σ are defined inductively by the grammar:

$$r, s ::= x \mid \varepsilon \mid \emptyset \mid r + s \mid r \cdot s \mid r^* \qquad (x \in \Sigma). \tag{2.1}$$

In the grammar, r and s range over regular expressions, while x ranges over the alphabet Σ and denotes an arbitrary symbol, such as a, b, or c. The constants ε and $\emptyset$ represent the *empty-word* and *empty-language* expressions. The operators $+$, $\cdot$, and * denote alternation, concatenation, and Kleene star, respectively.

Concatenation is written implicitly, so rs denotes $r \cdot s$. Given this notation, we adopt the precedence and associativity conventions. Unless parentheses are shown explicitly, Kleene star binds most tightly, followed by concatenation, followed by alternation. For example,

$$ab^*c = (a \cdot (b^*)) \cdot c,$$
$$a + bc^* = a + (b \cdot (c^*)).$$

For repeated operators we use left association, so

$$a + b + c = (a + b) + c,$$
$$abc = (ab)c.$$

The grammar above is the core syntax. Most regular expression libraries also offer one-or-more, optional, and bounded repetition (r^+, $r?$, $r^{\{m,n\}}$), but these add no expressive power since each can be defined by the core constructs: $r^+ = r \cdot r^*$, $r? = r + \varepsilon$, and

$$r^{\{m,n\}} = \underbrace{r \cdot \ldots \cdot r}_{m \text{ times}} \cdot \underbrace{(r + \varepsilon) \cdot \ldots \cdot (r + \varepsilon)}_{n-m \text{ times}}.$$

In the implementation, expressions using such operators are translated into the core constructs before they reach the parsing algorithms.

For example, consider the regular expression r_{01} over $\Sigma = \{0, 1\}$

$$r_{01} = (0 + 1)^* \cdot 0 \cdot 1. \tag{2.2}$$

By the conventions above, its syntax tree is $((0 + 1)^* \cdot 0) \cdot 1$. This tree represents the hierarchical structure of the expression: a Kleene star applied to the alternation $0 + 1$, followed by concatenation with the literals 0 and 1.

The syntactic structure must be given a semantic interpretation.

2.2 Semantics

A semantic interpretation assigns a language to each regular expression. A *word* over Σ is a finite sequence of symbols from Σ. The set of all words over Σ, including the empty word ε, is denoted by Σ^*. A *language* over Σ is a subset of Σ^*.

For a regular expression r, we write $L(r)$ for the language it denotes, where $L(r) \subseteq \Sigma^*$. The definition follows the structure of the syntax tree by *structural recursion*. Each case corresponds to one production of the grammar in (2.1):

$$\begin{aligned} L(x) &= \{x\}, \\ L(\varepsilon) &= \{\varepsilon\}, \\ L(\emptyset) &= \emptyset, \\ L(r + s) &= L(r) \cup L(s), \\ L(r \cdot s) &= \{vw \mid v \in L(r),\ w \in L(s)\}, \\ L(r^*) &= \{\varepsilon\} \cup \{w_1 \cdots w_n \mid n \geq 1,\ w_1, \ldots, w_n \in L(r)\}. \end{aligned} \tag{2.3}$$

Table 2.1 applies these definitions to small expressions. The literal a denotes the language containing only the word a. The expression ε denotes the language containing only the empty word, while $\emptyset$ denotes the language containing no words. The alternation $a + b$ combines the languages of a and b to give $\{a, b\}$, and concatenation ab combines words from the languages of a and b to give $\{ab\}$. The Kleene star a^* allows zero or more concatenations of words from the language of a, giving $\{\varepsilon, a, aa, aaa, \ldots\}$.

Expression r	Language $L(r)$
a	$\{a\}$
ε	$\{\varepsilon\}$
$\emptyset$	$\emptyset$
$a + b$	$\{a, b\}$
ab	$\{ab\}$
a^*	$\{\varepsilon, a, aa, aaa, \ldots\}$

Table 2.1: Languages denoted by basic expressions.

For example $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ from (2.2), we compute its language step by step:

$$\begin{aligned} L(0) &= \{0\}, \\ L(1) &= \{1\}, \\ L(0 + 1) &= L(0) \cup L(1) = \{0, 1\}, \\ L((0 + 1)^*) &= \{0, 1\}^*, \\ L(r_{01}) &= L((0 + 1)^*) \cdot L(0) \cdot L(1) = \{\, w \cdot 0 \cdot 1 \mid w \in \{0, 1\}^* \,\}. \end{aligned} \tag{2.4}$$

This is the set of binary words ending in 01. For example, the words 01, 001, and 1101 belong to $L(r_{01})$, whereas ε, 0, and 010 do not.

This is the question of whether a word *matches* a regular expression: w *matches* r if $w \in L(r)$. Determining this is the *membership problem*.

2.3 Rust Representation

The grammar from (2.1) is represented directly as a Rust enum with one variant for each syntactic construct, as shown in Figure 2.1:

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    Phi, Eps, Lit(char),
    Seq(Box<Regex>, Box<Regex>),  // r . s
    Alt(Box<Regex>, Box<Regex>),  // r + s
    Star(Box<Regex>),              // r*
}

impl Regex {
    pub fn seq(r: Regex, s: Regex) -> Regex {
        Regex::Seq(Box::new(r), Box::new(s))
    }
    pub fn alt(r: Regex, s: Regex) -> Regex {
        Regex::Alt(Box::new(r), Box::new(s))
    }
    pub fn star(r: Regex) -> Regex { Regex::Star(Box::new(r)) }
    pub fn lit(c: char) -> Regex { Regex::Lit(c) }
}
```

Figure 2.1: The `Regex` enum and constructors. Derived traits support operations needed by parsing: `Clone` for duplication, `PartialEq/Eq` for comparison, `Hash` for collections.

The type carries no precedence or associativity information, as these are determined by the syntax tree. Rust requires explicit indirection because enum sizes must be known at compile time. Recursive variants therefore use `Box<Regex>`. A direct definition such as `Seq(Regex, Regex)` would have infinite size, whereas `Box` stores recursive values separately, providing finite size [The Rust Project Developers, 2026a,b]. This differs from Haskell's heap-allocated data types.

The example $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ from (2.2) is represented by the syntax tree:

$$\mathrm{Seq}\Big(\mathrm{Seq}(\mathrm{Star}(\mathrm{Alt}(0, 1)), 0), 1\Big). \tag{2.5}$$

In Rust, this is represented as shown in Figure 2.2, with the outermost `Seq` containing `Lit('1')` and `Seq(Star(Alt(0,1)), 0)` as its right and left children.

```
Regex::seq(
    Regex::seq(
        Regex::star(Regex::alt(Regex::lit('0'), Regex::lit('1'))),
        Regex::lit('0')),
    Regex::lit('1'))
```

Figure 2.2: Rust representation of r_{01}.

3 Derivatives and Partial Derivatives

Chapter 2 introduced the syntax and semantics of regular expressions, and defined the *membership problem*: Decide whether a word w belongs to $L(r)$. This chapter introduces three algorithms for solving it, each returning a Boolean answer: match or no match.

The first is a naive recursive matcher that follows the structure of the regular-expression semantics. The other two transform the regular expression as input symbols are consumed. Brzozowski derivatives transform it into a single residual expression, while Antimirov's partial derivatives represent the possible residual expressions as a set.

3.1 Naive Matching

A direct way to decide whether $w \in L(r)$ is to interpret the regular-expression semantics as recursive matching rules, by following the structure of the expression and searching for possible decompositions of the input. Let $w[i:j]$ denote the substring from position i to j. A recursive procedure $\mathrm{match}(w, i, j, r)$ determines whether $w[i:j] \in L(r)$:

$$\begin{aligned}
\mathrm{match}(w,i,j,\varepsilon) &\iff i = j,\\
\mathrm{match}(w,i,j,x) &\iff j = i+1 \wedge w[i] = x,\\
\mathrm{match}(w,i,j,r+s) &\iff \mathrm{match}(w,i,j,r) \vee \mathrm{match}(w,i,j,s),\\
\mathrm{match}(w,i,j,r\cdot s) &\iff \exists k \in [i,j].\ \mathrm{match}(w,i,k,r) \wedge \mathrm{match}(w,k,j,s),\\
\mathrm{match}(w,i,j,r^*) &\iff i = j \vee \exists k \in (i,j].\ \mathrm{match}(w,i,k,r) \wedge \mathrm{match}(w,k,j,r^*).
\end{aligned} \tag{3.1}$$

Each rule corresponds directly to the semantics. Concatenation does not specify where to split, so the matcher must consider all possible split points k.

For matching $(a \cdot a) \cdot a$ against `aaa`, the split points $k \in \{0,1,2,3\}$ are:

$$\begin{aligned}
&\underbrace{\mathrm{match}(w,0,0,a\cdot a)}_{a\cdot a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\mathrm{match}(w,0,3,a)}_{a \text{ against } w[0:3]=\texttt{aaa}}\\
\vee\ &\underbrace{\mathrm{match}(w,0,1,a\cdot a)}_{a\cdot a \text{ against } w[0:1]=\texttt{a}} \wedge \underbrace{\mathrm{match}(w,1,3,a)}_{a \text{ against } w[1:3]=\texttt{aa}}\\
\vee\ &\underbrace{\mathrm{match}(w,0,2,a\cdot a)}_{a\cdot a \text{ against } w[0:2]=\texttt{aa}} \wedge \underbrace{\mathrm{match}(w,2,3,a)}_{a \text{ against } w[2:3]=\texttt{a}}\\
\vee\ &\underbrace{\mathrm{match}(w,0,3,a\cdot a)}_{a\cdot a \text{ against } w[0:3]=\texttt{aaa}} \wedge \underbrace{\mathrm{match}(w,3,3,a)}_{a \text{ against } w[3:3]=\epsilon}.
\end{aligned} \tag{3.2}$$

At $k = 2$, both operands match, triggering a recursive search on $(a \cdot a)$ against `aa` with split points $k \in \{0, 1, 2\}$, where $k = 1$ succeeds:

$$\begin{aligned} & \underbrace{\text{match}(w, 0, 0, a)}_{a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\texttt{aa}} \\ \vee\ & \underbrace{\text{match}(w, 0, 1, a)}_{a \text{ against } w[0:1]=\texttt{a}} \wedge \underbrace{\text{match}(w, 1, 2, a)}_{a \text{ against } w[1:2]=\texttt{a}} \\ \vee\ & \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\texttt{aa}} \wedge \underbrace{\text{match}(w, 2, 2, a)}_{a \text{ against } w[2:2]=\epsilon}. \end{aligned} \tag{3.3}$$

This completes the original match through the nested split points $k = 2$ and $k = 1$.

In Rust. The recursive rules translate directly into the corresponding Rust match arms, shown in Figure 3.1. Alternation tries both branches on the same input, while concatenation and star search possible split points. The star arm clones the body to construct the recursive `Star` expression. This is necessary because the recursive call needs to own the newly constructed expression. The range-based search short-circuits at the first successful split. Nested concatenations repeatedly search smaller substrings, exploring multiple paths before succeeding. This leads to exponential backtracking in the worst case.

```
Regex::Alt(r, s) => {
    match_naive_range(word, i, j, r)
        || match_naive_range(word, i, j, s)
}
Regex::Seq(r, s) => {
    (i..=j).any(|k| {
        match_naive_range(word, i, k, r)
            && match_naive_range(word, k, j, s)
    })
}
Regex::Star(r) => {
    i == j || (i + 1..=j).any(|k| {
        match_naive_range(word, i, k, r)
            && match_naive_range(word, k, j,
                &Regex::star(*r.clone()))
    })
}
```

Figure 3.1: Alternation, concatenation, and star arms of the naive matcher.

Although inefficient, this matcher is useful as a reference implementation because it follows the regular-expression semantics directly without any intermediate transformation. The cost is visible in the worked examples: nested concatenations repeatedly search smaller substrings, each exploring all possible split points before finding a match.

The derivative approach takes a different approach. Rather than searching for a decomposition of w while keeping r fixed, it transforms r as each character of w is consumed. The resulting expression describes the language that can still match the remaining suffix.

3.2 Brzozowski Derivatives

Brzozowski's derivative of r with respect to a symbol x, written $r\backslash x$, is the expression describing exactly what remains of r's language after consuming the leading symbol x of a word $w \in L(r)$ [Brzozowski, 1964]:

$$L(r\backslash x) \;=\; \{\, w \mid xw \in L(r) \,\}. \tag{3.4}$$

For $w = x_1 x_2 \cdots x_n$, consume one symbol at a time by repeatedly taking derivatives:

$$r, \quad r\backslash x_1, \quad (r\backslash x_1)\backslash x_2, \quad \ldots$$

Nullability. After all symbols have been consumed, the remaining input is ε. Thus, the resulting expression must accept ε for the original word to belong to $L(r)$. This is the question of whether the expression is *nullable*. Let $n(r)$ be true exactly when $\varepsilon \in L(r)$:

$$\begin{array}{llll} n(\emptyset) = \text{false} & (\mathsf{N}_\emptyset) & n(r+s) = n(r) \vee n(s) & (\mathsf{N}_+) \\ n(\varepsilon) = \text{true} & (\mathsf{N}_\varepsilon) & n(r \cdot s) = n(r) \wedge n(s) & (\mathsf{N}_{\text{seq}}) \\ n(x) = \text{false} & (\mathsf{N}_{\text{lit}}) & n(r^*) = \text{true} & (\mathsf{N}_*) \end{array} \tag{3.5}$$

The empty language $\emptyset$ is not nullable because it contains no words, including ε, while a literal cannot match the empty word. ε and r^* always match ε. An alternation is nullable when either branch is nullable, and a concatenation only when both components are.

In Rust. The definition is transcribed one `Regex` variant at a time, using the enum of Figure 2.1. The case split of (3.5) becomes the arms of a `match`, shown in Figure 3.2.

```
pub fn nullable(r: &Regex) -> bool {
    match r {
        Regex::Phi => false,
        Regex::Eps => true,
        Regex::Lit(_) => false,
        Regex::Alt(r, s) => nullable(r) || nullable(s),
        Regex::Seq(r, s) => nullable(r) && nullable(s),
        Regex::Star(_) => true,
    }
}
```

Figure 3.2: `nullable` implementation and its six cases: $\mathsf{N}_\emptyset$, N_ε, N_{lit}, N_+, N_{seq} and N_*.

The expression is passed by reference because `nullable` only inspects r and does not modify or consume it. The underscores in N_{lit} and N_* indicate that the corresponding operands are irrelevant. A literal is never nullable, regardless of its symbol, while a Kleene-star expression is always nullable, regardless of its body. The short-circuiting operators `||` and `&&` avoid evaluating the second operand when the result is already determined, while preserving the corresponding logical semantics.

Derivative Rules. The rules specify the expression that remains after consuming x:

$$\begin{aligned}
&\emptyset\backslash x = \emptyset && (\mathsf{D}_\emptyset) && (r+s)\backslash x = (r\backslash x) + (s\backslash x) && (\mathsf{D}_+)\\
&\varepsilon\backslash x = \emptyset && (\mathsf{D}_\varepsilon) && r^*\backslash x = (r\backslash x)\cdot r^* && (\mathsf{D}_*)\\
&y\backslash x = \begin{cases} \varepsilon & x = y \\ \emptyset & x \neq y \end{cases} && (\mathsf{D}_{\text{lit}}) && (r\cdot s)\backslash x = \begin{cases} (r\backslash x)\cdot s + s\backslash x & n(r) = \text{true} \\ (r\backslash x)\cdot s & n(r) = \text{false} \end{cases} && \begin{matrix}(\mathsf{D}^n_{\text{seq}})\\(\mathsf{D}_{\text{seq}})\end{matrix}
\end{aligned} \tag{3.6}$$

The empty language and ε cannot consume x. A literal y succeeds exactly when $x = y$, leaving ε. Alternation retains both derivatives and for r^*, one occurrence of r consumes x while further occurrences remain available.

Concatenation is the only rule that depends on nullability. If r is not nullable, r must consume the leading x, leaving $r\backslash x$ as the remainder of r while s remains untouched. If r is nullable, it may either consume x giving $(r\backslash x)\cdot s$, or r matches ε and s consumes x leaving $s\backslash x$. Hence the nullable case produces the alternation $(r\backslash x)\cdot s + s\backslash x$.

In Rust. Figure 3.3 implements (3.6) similarly, one variant at a time.

```
pub fn deriv(r: &Regex, x: char) -> Regex {
    match r {
        Regex::Phi => Regex::Phi,
        Regex::Eps => Regex::Phi,
        Regex::Lit(c) => if *c == x { Regex::Eps } else { Regex::Phi },
        Regex::Alt(r1, r2) =>
            Regex::alt(deriv(r1, x), deriv(r2, x)),
        Regex::Seq(r1, r2) => {
            let dr1 = Regex::seq(deriv(r1, x), *r2.clone());
            if nullable(r1) {
                Regex::alt(dr1, deriv(r2, x))
            } else { dr1 }
        }
        Regex::Star(r1) =>
            Regex::seq(deriv(r1, x), Regex::star(*r1.clone())),
    }
}
```

Figure 3.3: `deriv` implementation with six cases.

`deriv` differs from `nullable` in that it borrows its input but returns an owned `Regex`, since a derivative is a new expression. The `if nullable(r1)` implements the case split of (3.6), with branches in order $\mathsf{D}^n_{\text{seq}}$ then D_{seq}. This is the only place where a derivative rule consults nullability and where the output can be larger than the input, since the nullable branch produces an alternation while the other produces a concatenation.

The cloning in `*r2.clone()` is necessary because s appears untouched in both branches of the concatenation case, and the star arm must preserve r^*'s body both derived and underived. This copying mechanism drives expression growth, which justifies simplification after every derivative step.

Example: Growth without simplification. Consider $(x+y)^*$ against the word $\mathtt{xx}$: The expression is a star, so D_* applies with body $r = x + y$:

$$\begin{aligned} (x+y)^*\backslash\mathtt{x} &= ((x+y)\backslash\mathtt{x}) \cdot (x+y)^* && \mathsf{D}_*,\ r = x+y \\ &= (\varepsilon + \emptyset) \cdot (x+y)^* && \mathsf{D}_+,\ \mathsf{D}_{\text{lit}} \end{aligned} \tag{3.7}$$

The expression is now the concatenation $(\varepsilon + \emptyset) \cdot (x+y)^*$, and deriving it consumes the second $\mathtt{x}$. The result is an alternation because $\mathsf{D}^n_{\text{seq}}$ applies with $r = \varepsilon + \emptyset$ and $s = (x+y)^*$, its nullable case being selected since $n(r) = n(\varepsilon + \emptyset) = \text{true}$ by N_+ and N_ε:

$$\begin{aligned} &((\varepsilon + \emptyset) \cdot (x+y)^*)\backslash\mathtt{x} \\ &= ((\varepsilon + \emptyset)\backslash\mathtt{x}) \cdot (x+y)^* + (x+y)^*\backslash\mathtt{x} && \mathsf{D}^n_{\text{seq}} \\ &= (\emptyset + \emptyset) \cdot (x+y)^* + (x+y)^*\backslash\mathtt{x} && \mathsf{D}_\varepsilon \\ &= (\emptyset + \emptyset) \cdot (x+y)^* + (\varepsilon + \emptyset) \cdot (x+y)^* && \mathsf{D}_*,\ \mathsf{D}_+,\ \mathsf{D}_{\text{lit}} \end{aligned} \tag{3.8}$$

After both symbols are consumed, the result is checked for nullability:

$$\begin{aligned} &n((\emptyset + \emptyset) \cdot (x+y)^* + (\varepsilon + \emptyset) \cdot (x+y)^*) \\ &= n((\emptyset + \emptyset) \cdot (x+y)^*) \vee n((\varepsilon + \emptyset) \cdot (x+y)^*) && \mathsf{N}_+ \\ &= (n(\emptyset + \emptyset) \wedge n((x+y)^*)) \vee (n(\varepsilon + \emptyset) \wedge n((x+y)^*)) && \mathsf{N}_{\text{seq}} \\ &= (\text{false} \wedge \text{true}) \vee (\text{true} \wedge \text{true}) && \mathsf{N}_+,\ \mathsf{N}_\emptyset,\ \mathsf{N}_\varepsilon,\ \mathsf{N}_* \\ &= \text{true}. \end{aligned} \tag{3.9}$$

Thus $(x+y)^*$ matches $\mathtt{xx}$.

Simplification. Derivatives can grow unboundedly. Repeatedly deriving $(x+y)^*$ by the same symbol produces expressions of increasing size. This growth results from the nullable branch of $\mathsf{D}^n_{\text{seq}}$, which introduces an alternation as a redundant structure that is carried into subsequent derivatives. ε and $\emptyset$ accumulate without bound:

$$\begin{aligned} & (x+y)^* && 4 \\ \xrightarrow{\mathtt{x}}\ & (\varepsilon + \emptyset) \cdot (x+y)^* && 8 \\ \xrightarrow{\mathtt{x}}\ & (\emptyset + \emptyset) \cdot (x+y)^* \ +\ (\varepsilon + \emptyset) \cdot (x+y)^* && 17 \\ \xrightarrow{\mathtt{x}}\ & (\emptyset + \emptyset) \cdot (x+y)^* \ +\ \left((\emptyset + \emptyset) \cdot (x+y)^* \ +\ (\varepsilon + \emptyset) \cdot (x+y)^*\right) && 26 \end{aligned} \tag{3.10}$$

To counter the growth, expressions are simplified after each derivative step, using identities that preserve the denoted language:

$$\begin{aligned} \emptyset \cdot r &= \emptyset && (\mathsf{S}_1) \\ \varepsilon \cdot r &= r && (\mathsf{S}_2) \\ \emptyset + r &= r && (\mathsf{S}_3) \end{aligned} \tag{3.11}$$

S_1 eliminates dead concatenations with $\emptyset$, S_2 eliminates identity concatenations with ε, and S_3 eliminates $\emptyset$ from alternations. Each holds equally with the unit on the right.

Applying them to (3.8), after each step, returns the expression back to $(x+y)^*$, keeping the size bounded regardless of input length:

$$\begin{aligned}
&(\emptyset + \emptyset) \cdot (x+y)^* + (\varepsilon + \emptyset) \cdot (x+y)^* \\
&= \emptyset \cdot (x+y)^* + \varepsilon \cdot (x+y)^* && \mathsf{S}_3,\ \text{twice} \\
&= \emptyset + (x+y)^* && \mathsf{S}_1,\ \mathsf{S}_2 \\
&= (x+y)^* && \mathsf{S}_3
\end{aligned} \tag{3.12}$$

In Rust. The identities are implemented recursively, applying the rules bottom-up over the expression tree. Each arm of the inner match implements an identity from (3.11).

`simplify` takes ownership of its input and returns simplified subexpressions directly, unlike `deriv`, which borrows and constructs. This model is visible where the two compose:

```
current = simplify(deriv(&current, c));
```

`deriv` borrows `current`, constructs a new expression, and `simplify` takes ownership of that result to return it without additional copying. The recursive calls ensure that all dead code is eliminated before an expression is returned, keeping size bounded at each step. Together, derive-then-simplify forms the core loop of the matcher, transforming and cleaning the expression at every input symbol.

```
pub fn simplify(r: Regex) -> Regex {
    match r {
        Regex::Seq(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) | (_, Regex::Phi) => Regex::Phi,
                (Regex::Eps, _) => s,
                (_, Regex::Eps) => r,
                _ => Regex::seq(r, s),
            }
        }
        Regex::Alt(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) => s,
                (_, Regex::Phi) => r,
                _ if r == s => r,
                _ => Regex::alt(r, s),
            }
        }
        Regex::Star(r) => match simplify(*r) {
            Regex::Eps | Regex::Phi => Regex::Eps,
            r => Regex::star(r),
        },
        other => other,
    }
}
```

Figure 3.4: Simplification rules: S_1 and S_2 in concatenation, S_3 in both branches.

The matcher. The implementation can be summarized by the sequence of residual expressions produced while consuming the input. For $w = x_1 x_2 \cdots x_n$, the initial residual is $r_0 = r$, and each subsequent residual is obtained by deriving with respect to the next symbol and simplifying the result:

$$r_0 = r, \qquad r_i = \text{simplify}(r_{i-1} \backslash x_i) \quad (1 \le i \le n). \tag{3.13}$$

The residual r_i denotes exactly the set of suffixes that complete the consumed prefix $x_1 \cdots x_i$ to a word in $L(r)$. Since simplification preserves the represented language, each r_i is equivalent to the unsimplified derivative after consuming that prefix. After the final symbol has been consumed, the matcher accepts when the remaining residual is nullable:

$$w \in L(r) \iff n(r_n) = \text{true}.$$

Each step depends only on the current residual and the next symbol, so the matcher never revisits a prefix and never backtracks into an earlier residual. The cost of one step is the cost of `deriv` on the current residual, which is proportional to its size.

So $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \cdots \xrightarrow{x_n} r_n$ can be viewed as a path through a lazily constructed DFA. Each residual expression represents a state, and $\xrightarrow{x}$ denotes the transition obtained by taking the derivative with respect to x. Unlike an explicitly constructed DFA, only the states reached by the input are constructed. A residual is an accepting state exactly when it is nullable, so acceptance is determined by whether the final residual is nullable.

In Rust. Figure 3.5 implements (3.13) by composing `deriv`, `simplify`, and `nullable`. The implementation follows the mathematical definition directly. The current residual is transformed at each step, and acceptance is determined after the complete input has been consumed.

```
pub fn match_deriv(input: &str, r: &Regex) -> bool {
    let mut current = r.clone();
    for c in input.chars() {
        current = simplify(deriv(&current, c));
    }
    nullable(&current)
}
```

Figure 3.5: `match_deriv`: the core matcher loop. For each input character, derive the current residual expression and simplify it in place. Accept if the final residual is nullable.

The loop implements the residual sequence. At each iteration, `current` contains the current residual, which is then derived and simplified before processing the next input symbol. Only the current residual is retained, since the previous residual is dropped when `current` is reassigned. In contrast to parsing, where intermediate results may be required to reconstruct a parse tree, Boolean matching requires only the final residual.

The derivative-based matcher avoids the exponential backtracking of naive matching by transforming the expression rather than searching for decompositions. Simplification keeps the resulting expressions compact, preventing the growth shown in (3.10).

Worked Example: Applying the derivative-based matcher Apply `match_deriv` to $r_0 = (a + ab)(b + \varepsilon)$ and $w = \mathtt{ab}$ with the expression written as:

$$r_0 = \mathit{Seq}(s,\, t), \qquad s = a + ab, \qquad t = b + \varepsilon,$$

Derive by `a`. The first factor s is not nullable, so D_{seq} applies to the sequence r_0:

$$r_1 = r_0 \backslash \mathtt{a} = (s \backslash \mathtt{a}) \cdot t \qquad (\mathsf{D}_{\text{seq}}).$$

Since s cannot match the empty word, the second factor t is not touched by this derivative and appears unchanged. To derive s, the alternation rule D_{+} is applied to give:

$$s \backslash \mathtt{a} = (a \backslash \mathtt{a}) + (ab \backslash \mathtt{a}) \qquad (\mathsf{D}_{+}).$$

The two summands are derived separately, each by its own rule:

$$\begin{aligned} a \backslash \mathtt{a} &= \varepsilon && (\mathsf{D}_{\text{lit}}), \\ ab \backslash \mathtt{a} &= (a \backslash \mathtt{a}) \cdot b = \varepsilon \cdot b && (\mathsf{D}_{\text{seq}},\ \mathsf{D}_{\text{lit}}). \end{aligned}$$

Substituting both summands into the alternation and the result into the outer sequence:

$$\begin{aligned} s \backslash \mathtt{a} &= \varepsilon + \varepsilon \cdot b && (\mathsf{D}_{+}), \\ r_0 \backslash \mathtt{a} &= (\varepsilon + \varepsilon \cdot b) \cdot t && (\mathsf{D}_{\text{seq}}). \end{aligned}$$

Then, applying the simplification rule S_2 to the occurrences of $\varepsilon \cdot b$, gives the residual:

$$r_1 = (\varepsilon + b) \cdot t \qquad (\mathsf{S}_2).$$

Derive by `b`. The residual r_1 is again a sequence, but now the first factor $\varepsilon + b$ is nullable, so the nullable sequence rule $\mathsf{D}^{n}_{\text{seq}}$ applies when deriving by b:

$$r_2 = r_1 \backslash \mathtt{b} = \Big((\varepsilon + b) \backslash \mathtt{b}\Big) \cdot t + (t \backslash \mathtt{b}) \qquad (\mathsf{D}^{n}_{\text{seq}}).$$

Since the first factor $\varepsilon + b$ is nullable, it can either consume the character itself, leaving t untouched, or match ε and let t consume it. Both are possible, so $\mathsf{D}^{n}_{\text{seq}}$ produces one summand for each, and D_{+} derives the first factor and t in their respective summands:

$$\begin{aligned} (\varepsilon + b) \backslash \mathtt{b} &= (\varepsilon \backslash \mathtt{b}) + (b \backslash \mathtt{b}) = \emptyset + \varepsilon && (\mathsf{D}_{+},\ \mathsf{D}_{\varepsilon},\ \mathsf{D}_{\text{lit}}), \\ t \backslash \mathtt{b} = (b + \varepsilon) \backslash \mathtt{b} &= (b \backslash \mathtt{b}) + (\varepsilon \backslash \mathtt{b}) = \varepsilon + \emptyset && (\mathsf{D}_{+},\ \mathsf{D}_{\text{lit}},\ \mathsf{D}_{\varepsilon}). \end{aligned}$$

Substituting both summands into the two-summand expression:

$$r_1 \backslash \mathtt{b} = (\emptyset + \varepsilon) \cdot t + (\varepsilon + \emptyset).$$

Applying the simplification rules S_3 and S_2 to the first summand and S_3 to the second, gives the final residual after deriving the second character:

$$r_2 = t + \varepsilon \qquad (\mathsf{S}_3,\ \mathsf{S}_2,\ \mathsf{S}_3).$$

Acceptance. The residual $r_2 = (b + \varepsilon) + \varepsilon$ is nullable by N_{+} and N_{ε}, so $\mathtt{ab} \in L(r_0)$.

3.3 Antimirov Partial Derivatives

The derivative of Section 3.2 combines alternative residuals into a single expression using $+$. Antimirov's partial derivatives instead keep them as a *set* of expressions, one per strand of nondeterminism [Antimirov, 1996]. Writing $pd(r, x)$ for this set:

$$\begin{aligned}
pd(\emptyset, x) &= \{\} && (\mathsf{P}_\emptyset)\\
pd(\varepsilon, x) &= \{\} && (\mathsf{P}_\varepsilon)\\
pd(y, x) &= \begin{cases} \{\varepsilon\} & x = y \\ \{\} & x \neq y \end{cases} && (\mathsf{P}_{\text{lit}})\\
pd(r + s, x) &= pd(r, x) \cup pd(s, x) && (\mathsf{P}_+)\\
pd(r^*, x) &= \{\, smartC(r', r^*) \mid r' \in pd(r, x) \,\} && (\mathsf{P}_*)\\
pd(r \cdot s, x) &= \begin{cases} \{\, smartC(r', s) \mid r' \in pd(r, x) \,\} \cup pd(s, x) & n(r) = \text{true} \\ \{\, smartC(r', s) \mid r' \in pd(r, x) \,\} & n(r) = \text{false} \end{cases} && \begin{matrix}(\mathsf{P}^n_{\text{seq}})\\(\mathsf{P}_{\text{seq}})\end{matrix}
\end{aligned} \tag{3.14}$$

The empty language and ε produce the empty set, since neither has a residual with respect to x: where $\mathsf{D}_\emptyset$ and D_ε returned $\emptyset$ as a node in the result, $\mathsf{P}_\emptyset$ and P_ε return no residual at all. A literal y gives $\{\varepsilon\}$ when $x = y$ and $\{\}$ otherwise. Alternation unions the two residual sets, and for r^* one iteration of r consumes x, with each resulting residual placed in front of r^* so the star may continue. None of these alternatives are combined with $+$; each stays a separate element of the set.

Concatenation depends on nullability for the same reason as D_{seq}. If r is not nullable it must consume x, and each of its residuals is concatenated with the untouched s. If r is nullable, r may instead match ε and let s consume x, in which case $pd(s, x)$ joins the set. Where $\mathsf{D}^n_{\text{seq}}$ combined the two cases with $+$ inside one expression, $\mathsf{P}^n_{\text{seq}}$ keeps them as two elements of one set.

The smart constructor. The concatenation in P_{seq} uses an auxiliary constructor *smartC*. Since each element of $pd(r, x)$ is a separate residual rather than a branch of a sum, *smartC* is applied to each independently. The constructor avoids building $\varepsilon \cdot s$:

$$smartC(r', s) = \begin{cases} s & r' = \varepsilon \\ r' \cdot s & \text{otherwise.} \end{cases} \tag{3.15}$$

In Rust. Figure 3.6 implements *smartC* directly as `smart_seq`. Matching on `r` implements the case split of (3.15): the `Eps` arm returns a clone of `s` and never builds $\varepsilon \cdot s$, while the second arm wraps `r` in front of a cloned `s`

```
pub fn smart_seq(r: Regex, s: &Regex) -> Regex {
    match r {
        Regex::Eps => s.clone(),
        r => Regex::seq(r, s.clone()),
    }
}
```

Figure 3.6: `smart_seq`, the smart constructor *smartC*

Figure 3.7 presents the implementation of (3.14), with each `match` arm corresponding to one case. The `Phi`, `Eps`, and `Lit` variants implement $\mathsf{P}_{\emptyset}$, P_{ε}, and P_{lit} directly, returning `Vec::new()` where the equation gives $\{\}$. `Alt` concatenates the two child vectors, implementing the union of P_{+}.

`Star` maps `smart_seq` over r's own residuals, matching P_{*}. The star's body is derived, and each resulting residual is placed in front of a fresh `Regex::star` so the star may continue. `Regex::star(*r.clone())` clones because `Regex::star` needs an owned `Regex` and `r` is only borrowed, once per residual since the closure runs once for each.

`Seq` implements the case split directly. The `map` applies `smart_seq` to each residual of the left operand, matching P_{seq}, and the guarded `out.extend(pderiv(s, x))` adds $pd(s, x)$ for $\mathsf{P}^{n}_{\text{seq}}$. Although the mathematical definition uses sets, the implementation uses `Vec<Regex>` throughout the recursion and defers deduplication to a single `HashSet` insertion at the top, rather than deduplicating at every `Alt` or `Seq` node on the way down.

Every union in `pderiv` is therefore a vector `extend`, not a set operation. Duplicate residuals reached by different strands are only merged once inserted into the `HashSet` used by the matcher below to keep the residual generation separate from deduplication

```
fn pderiv(r: &Regex, x: char) -> Vec<Regex> {
    match r {
        Regex::Phi => Vec::new(),
        Regex::Eps => Vec::new(),
        Regex::Lit(c) => {
            if *c == x { vec![Regex::Eps] } else { Vec::new() }
        }
        Regex::Alt(r, s) => {
            let mut out = pderiv(r, x);
            out.extend(pderiv(s, x));
            out
        }
        Regex::Seq(r, s) => {
            let mut out: Vec<Regex> = pderiv(r, x)
                .into_iter()
                .map(|r_prime| smart_seq(r_prime, s))
                .collect();
            if nullable(r) {
                out.extend(pderiv(s, x));
            }
            out
        }
        Regex::Star(r) => {
            pderiv(r, x)
                .into_iter()
                .map(|r_prime| smart_seq(r_prime, &Regex::star(*r.clone())))
                .collect()
        }
    }
}
```

Figure 3.7: `pderiv`, implementing (3.14) one arm per case of the equation.

Example: No growth under *pd*. Consider $(x+y)^*$ against $\mathtt{xx}$ under pd.

P_* applies with star body $r = x+y$:

$$\begin{aligned} pd((x+y)^*, \mathtt{x}) &= \{\, smartC(r', (x+y)^*) \mid r' \in pd(x+y, \mathtt{x}) \,\} && \mathsf{P}_* \\ &= \{\, smartC(r', (x+y)^*) \mid r' \in \{\varepsilon\} \,\} && \mathsf{P}_+,\ \mathsf{P}_{\text{lit}} \text{ twice} \\ &= \{(x+y)^*\} && \mathsf{SC},\ r' = \varepsilon. \end{aligned} \tag{3.16}$$

The set is $\{(x+y)^*\}$, so deriving by the second $\mathtt{x}$ repeats the computation, and the final partial derivative accepts ε since $(x+y)^*$ is nullable by N_*. So $\mathtt{xx} \in L((x+y)^*)$.

Equation (3.17) traces the derivative and the partial derivative on the same expression:

$$\begin{aligned} \text{Derivative (unsimplified):} \quad & \underbrace{(x+y)^*}_{4} \xrightarrow{\mathtt{x}} \underbrace{(\varepsilon+\emptyset)(x+y)^*}_{8} \xrightarrow{\mathtt{x}} \underbrace{(\emptyset+\emptyset)(x+y)^*+(\varepsilon+\emptyset)(x+y)^*}_{17} \\ \text{Partial derivative:} \quad & \underbrace{\{(x+y)^*\}}_{4} \xrightarrow{\mathtt{x}} \underbrace{\{(x+y)^*\}}_{4} \xrightarrow{\mathtt{x}} \underbrace{\{(x+y)^*\}}_{4} \end{aligned} \tag{3.17}$$

The unsimplified derivative grows at every step, while the partial derivative stays at one element because $\mathsf{P}_\emptyset$ and P_ε contribute no residual, so there is nothing to simplify.

Finiteness. For a regular expression r, the number of distinct partial derivatives reachable by any input is bounded by $\|r\| + 1$ [Antimirov, 1996, Theorem 3.4, p. 309]. This is a ceiling, not a guarantee that all $\|r\| + 1$ are reached. For $(x+y)^*$, $\|r\| = 2$ the ceiling is 3, yet only $(x+y)^*$ is reachable, since deriving by either $\mathtt{x}$ or $\mathtt{y}$ returns it unchanged.

Ordinary derivatives need equivalence classes to be finite [Brzozowski, 1964, Theorem 4.3], giving $d_R \leq 2^{d_P} - 1$. Simplification removes units and redundant $\emptyset$, but not all equivalent forms. In contrast the partial-derivative bound is linear without simplification.

The matcher. The implementation can be summarized by the sequence of state sets produced while consuming the input. For $w = x_1 x_2 \cdots x_n$, the initial state set is $S_0 = \{r\}$, and each subsequent set is obtained by taking pd of every residual in the previous set and unioning the results:

$$S_0 = \{r\}, \qquad S_i = \bigcup_{r' \in S_{i-1}} pd(r', x_i) \quad (1 \leq i \leq n). \tag{3.18}$$

The state set S_i collects every residual reachable after consuming the prefix $x_1 \cdots x_i$. Unlike the derivative matcher's single residual r_i, no element of S_i grows with the input. The number of elements is bounded by $\|r\| + 1$ as shown above. After the final symbol has been consumed, the matcher accepts when some residual in S_n is nullable:

$$w \in L(r) \iff \exists r_n \in S_n.\ n(r_n) = \text{true}.$$

So $S_0 \xrightarrow{x_1} S_1 \xrightarrow{x_2} \cdots \xrightarrow{x_n} S_n$ can be viewed as a path through a lazily constructed NFA. Each state set represents the set of NFA states reachable after reading the corresponding prefix, and $\xrightarrow{x}$ denotes the transition obtained by taking pd with respect to x and unioning. Unlike an explicitly constructed NFA, only the states reached by the input are constructed. A state set is accepting exactly when it contains at least one nullable residual, so acceptance is determined by whether the final state set contains a nullable residual.

In Rust. Figure 3.8 implements (3.18) with a `HashSet<Regex>`, whose `Hash` and `Eq` implementations collapse residuals reached by different paths. The early return when the set empties follows from construction: an empty set has no states to derive. The final `any(nullable)` checks whether some residual is nullable. Where the derivative matcher holds one growing `Regex`, this holds a set of small residuals that never need simplification.

```
pub fn match_pderiv(input: &str, r: &Regex) -> bool {
    let mut states: HashSet<Regex> = HashSet::new();
    states.insert(r.clone());
    for c in input.chars() {
        let mut next: HashSet<Regex> = HashSet::new();
        for state in &states {
            next.extend(pderiv(state, c));
        }
        if next.is_empty() { return false; }
        states = next;
    }
    states.iter().any(nullable)
}
```

Figure 3.8: The partial-derivative matcher.

Worked Example: Applying the partial-derivative-based matcher Applying `match_pderiv` to $r_0 = (a + ab)(b + \varepsilon)$ and $w = \mathtt{ab}$ with the expression written as:

$$r_0 = \mathit{Seq}(s,\, t), \qquad s = a + ab, \qquad t = b + \varepsilon.$$

Derive by `a`. Since s is not nullable, t is untouched and P_{seq} applies to r_0:

$$S_1 = \{\, \mathit{smartC}(r', t) \mid r' \in \mathit{pd}(s, \mathtt{a}) \,\} \qquad (\mathsf{P}_{\text{seq}}).$$

The alternation rule gives the two branches of s separately:

$$\begin{aligned} \mathit{pd}(a, \mathtt{a}) &= \{\varepsilon\} && (\mathsf{P}_{\text{lit}}),\\ \mathit{pd}(ab, \mathtt{a}) &= \{\mathit{smartC}(\varepsilon, b)\} = \{b\} && (\mathsf{P}_{\text{seq}}), \end{aligned}$$

So $\mathit{pd}(s, \mathtt{a}) = \{\varepsilon, b\}$. Applying smartC to each element against t prepends that element to t, with ε absorbed so that $\varepsilon \cdot t$ is never built:

$$S_1 = \{\, t,\ b \cdot t \,\} = \{\, b + \varepsilon,\ \ b\,(b + \varepsilon) \,\}.$$

Derive by `b`. Each state in S_1 is derived separately and the results unioned.

For the first state $t = b + \varepsilon$:

$$\mathit{pd}(b + \varepsilon, \mathtt{b}) = \mathit{pd}(b, \mathtt{b}) \cup \mathit{pd}(\varepsilon, \mathtt{b}) = \{\varepsilon\} \cup \{\} = \{\varepsilon\} \qquad (\mathsf{P}_{+},\ \mathsf{P}_{\text{lit}},\ \mathsf{P}_{\varepsilon}).$$

For the second state $b \cdot t = b\,(b + \varepsilon)$:

$$\mathit{pd}\Big(b\,(b + \varepsilon), \mathtt{b}\Big) = \{\, \mathit{smartC}(\varepsilon,\, b + \varepsilon) \,\} = \{b + \varepsilon\} \qquad (\mathsf{P}_{\text{seq}}).$$

Only the left factor b is derived, as it is a non-nullable literal. The union gives:

$$S_2 = \{\varepsilon\} \cup \{b + \varepsilon\} = \{\, \varepsilon,\ \ b + \varepsilon \,\}.$$

Acceptance. Both states in S_2 are nullable, ε by N_{ε} and $b + \varepsilon$ by N_{+} and N_{ε}, so `match_pderiv("ab",` r_0`)` returns `true` and $\mathtt{ab} \in L(r_0)$, agreeing with Section 3.2.

4 Matching and Parsing

Chapter 3 established matching with a Boolean answer: $w \in L(r)$. A matcher only answers whether the input is accepted by the regular expression. Parsing identifies the evidence for the match and *parse trees* act as their structured witnesses.

4.1 Matching: A Boolean Answer

Consider $r = (a + ab)(b + \varepsilon)$ and $w = \texttt{ab}$. The matcher can determine $w \in L(r)$ using the algorithms of Chapter 3. There are, however, two ways in which r can match w:

$$\underbrace{a}_{a+ab}\ \underbrace{b}_{b+\varepsilon} \quad \text{and} \quad \underbrace{\texttt{ab}}_{a+ab}\ \underbrace{\varepsilon}_{b+\varepsilon} \tag{4.1}$$

Both choices establish the same membership result, but assign the input differently to the subexpressions of r. The Boolean result alone cannot distinguish these cases, so the matcher cannot reveal which decomposition was chosen.

Parsing makes this additional structure explicit, by showing which alternatives were chosen and which parts of the input were consumed by the corresponding subexpressions.

4.2 Parsing: Parse Trees as Matching Evidence

Parse Tree. A *parse tree* provides a structured witness for such a derivation. Sulzmann and Lu view regular expressions as types and *parse trees* as their values [Sulzmann and Lu, 2014]. The judgment $\vdash v : r$ states that v is a valid parse tree for r, recording which alternatives were chosen and how each part of the input was consumed during matching:

$$\frac{}{\vdash () : \varepsilon}\ (\mathsf{T}_\varepsilon) \qquad \frac{\vdash v_1 : r_1}{\vdash \mathit{Left}\ v_1 : r_1 + r_2}\ (\mathsf{T}^L_+)$$

$$\frac{l \in \Sigma}{\vdash l : l}\ (\mathsf{T}_{\text{lit}}) \qquad \frac{\vdash v_2 : r_2}{\vdash \mathit{Right}\ v_2 : r_1 + r_2}\ (\mathsf{T}^R_+) \tag{4.2}$$

$$\frac{\vdash v_1 : r_1 \qquad \vdash v_2 : r_2}{\vdash (v_1, v_2) : r_1 r_2}\ (\mathsf{T}_{\text{seq}}) \qquad \frac{}{\vdash [\,] : r^*}\ (\mathsf{T}^0_*)$$

$$\frac{\vdash v : r \qquad \vdash vs : r^*}{\vdash (v : vs) : r^*}\ (\mathsf{T}_*)$$

In Rust. Figure 4.1 shows the parse-tree type, with one variant for each constructor.

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ParseTree {
    Empty,                                  // ()
    Char(char),                             // l
    Pair(Box<ParseTree>, Box<ParseTree>),   // (v1, v2)
    Left(Box<ParseTree>),                   // Left v
    Right(Box<ParseTree>),                  // Right v
    Star(Vec<ParseTree>),                   // [v1, ..., vn]
}
```

Figure 4.1: The parse tree type, representing the structure of a parse.

The definitions above translate directly to Rust, except for the star case. $\mathsf{T}\varepsilon$ corresponds to `Empty` for the empty parse, T_{lit} to `Char` for a literal, T_{seq} to `Pair` for a concatenation of parse trees, and T_{+}^{L} and T_{+}^{R} to `Left` and `Right`, recording the chosen branch of an alternation. Both star rules are represented by `Star(vec![])`.

Flattening. The typing judgment describes the structure of a match, but a parse tree must also be related to the word that it matches. For this purpose, they define a *flattening* function $|v|$ that maps a parse tree to the word represented by its structure:

$$\begin{aligned} |()| &= \varepsilon, & |(v_1, v_2)| &= |v_1||v_2|, \\ |l| &= l, & |\mathit{Left}\ v| &= |v|, \\ |[]| &= \varepsilon & |\mathit{Right}\ v| &= |v|, \\ & & |v : vs| &= |v||vs|. \end{aligned} \tag{4.3}$$

The flattening function directly connects parse trees to $L(r)$, and thus validates that a parse tree witnesses membership in $L(r)$, serving as an correctness check:

$$L(r) = \{\, |v| \mid\mid\!\vdash v : r \,\}. \tag{4.4}$$

In Rust. Figure 4.2 shows how `flatten` recursively traverses the parse tree and concatenates its leaf characters. The function borrows the tree to allow subsequent inspection and comparison. The `Pair` case concatenates the results of flattening both children, while `Left` and `Right` delegate to their child, and `Star` concatenates the results for each element.

```
pub fn flatten(v: &ParseTree) -> String {
    match v {
        ParseTree::Empty        => String::new(),
        ParseTree::Char(c)      => c.to_string(),
        ParseTree::Pair(v1, v2) => format!("{}{}", flatten(v1), flatten(v2)),
        ParseTree::Left(v)      => flatten(v),
        ParseTree::Right(v)     => flatten(v),
        ParseTree::Star(vs)     => vs.iter().map(flatten).collect(),
    }
}
```

Figure 4.2: The `flatten` function extracts the matched word from a parse tree.

Unambiguous example. Consider $r = ab$ and $w = \texttt{ab}$. Since each subexpression is a literal, the rule T_{lit} gives each literal a unique witness. Then T_{seq} requires the parse tree for a to be followed by the parse tree for b, combining them into a parse tree for ab:

$$\frac{a \in \Sigma}{\vdash a : a}\,(\mathsf{T}_{\text{lit}}) \qquad \frac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}}) \qquad \frac{\vdash a : a \qquad \vdash b : b}{\vdash (a, b) : ab}\,(\mathsf{T}_{\text{seq}}) \tag{4.5}$$

The parse tree $v = (a, b)$ flattens to $|(a, b)| = \texttt{ab}$.

Ambiguous Example. Recall $r = (a + ab)(b + \varepsilon)$ and $w = \texttt{ab}$ from Section 4.1. The word admits two distinct decompositions, corresponding to different parse trees.

The first decomposition uses the right alternatives of both subexpressions:

$$\underbrace{\frac{\dfrac{a \in \Sigma}{\vdash a : a}\,(\mathsf{T}_{\text{lit}}) \qquad \dfrac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}})}{\vdash (a, b) : ab}\,(\mathsf{T}_{\text{seq}})}{\vdash \mathit{Right}(a, b) : a + ab}\,(\mathsf{T}^R_+)}_{a + ab} \qquad \underbrace{\frac{\dfrac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}}) \qquad \vdash () : \varepsilon\,(\mathsf{T}_\varepsilon)}{\vdash \mathit{Right}() : b + \varepsilon}\,(\mathsf{T}^R_+)}_{b + \varepsilon}$$

By T_{seq}, these derivations combine into the parse tree:

$$\frac{\dfrac{\vdash a : a \qquad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \mathit{Right}(a, b) : a + ab} \qquad \frac{\vdash () : \varepsilon}{\vdash \mathit{Right}() : b + \varepsilon} \tag{4.6}$$

$$v_1 = \Big(\mathit{Right}(a, b), \mathit{Right}()\Big), \qquad |v_1| = \texttt{ab}.$$

The second decomposition uses the left alternatives of both subexpressions:

$$\underbrace{\frac{\dfrac{a \in \Sigma}{\vdash a : a}\,(\mathsf{T}_{\text{lit}})}{\vdash \mathit{Left}\, a : a + ab}\,(\mathsf{T}^L_+)}_{a + ab} \qquad \underbrace{\frac{\dfrac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}})}{\vdash \mathit{Left}\, b : b + \varepsilon}\,(\mathsf{T}^L_+)}_{b + \varepsilon}$$

By T_{seq}, these derivations combine into the second parse tree:

$$\frac{\vdash a : a}{\vdash \mathit{Left}\, a : a + ab} \qquad \frac{\vdash b : b}{\vdash \mathit{Left}\, b : b + \varepsilon} \tag{4.7}$$

$$v_2 = \Big(\mathit{Left}\, a, \mathit{Left}\, b\Big), \qquad |v_2| = \texttt{ab}.$$

Both trees are valid parse trees for the same expression and word, yet they differ in structure. The typing judgment alone does not determine which parse tree should be selected. A *disambiguation policy* specifies this choice.

5 Derivative-Based Parser

Chapter 3 showed how a Brzozowski-derivative matcher determines whether $w \in L(r)$ and Chapter 4 showed that a matching word may admit multiple valid parse trees. A parser must therefore select a single parse tree according to a disambiguation policy.

Following Sulzmann and Lu [Sulzmann and Lu, 2014], this chapter presents two derivative-based methods for constructing POSIX parse trees. The direct method performs a forward derivative pass followed by a backward pass for tree reconstruction. The bit-coded method records parse-tree information during the forward pass, avoiding a separate backward pass.

5.1 The Standard Derivative-Based Parser

Consider the example $r = (a + ab)(b + \varepsilon)$ with input $w = \texttt{ab}$. Chapter 4 showed that it is ambiguous, as the word admits two parse trees. The question here is whether a parse tree can be recovered from the final derivative:

$$\begin{aligned} r &\xrightarrow{a} (\varepsilon + \varepsilon b)(b + \varepsilon) \\ &\xrightarrow{b} (\emptyset + (\emptyset b + \varepsilon))(b + \varepsilon) + (\varepsilon + \emptyset). \end{aligned} \tag{5.1}$$

The derivation was already computed in Section 3.2, shown here unsimplified. The final derivative is nullable, so $w \in L(r)$ and both disjuncts accept the empty word. Its first disjunct, $(\emptyset + (\emptyset b + \varepsilon))(b + \varepsilon)$, reflects $ab \cdot \varepsilon$, and its second, $(\varepsilon + \emptyset)$, reflects $a \cdot b$.

Since a derivative step only records the remaining input language and not how consumed characters were matched, the final derivative alone does not show which decomposition was selected, and cannot be used to recover the parse tree. However, the derivation sequence itself contains this information. Each step consumed one character and is invertible given that character, so reversing the steps recovers the tree, as shown in Figure 5.1:

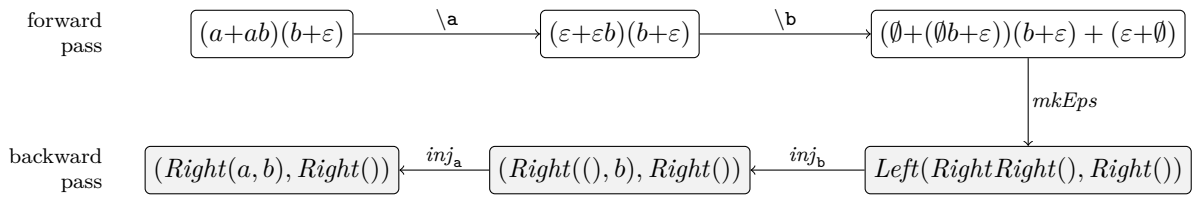


Figure 5.1: Overview of the standard derivative-based parser on the ambiguous example. The forward pass computes derivatives, while the backward pass uses *mkEps* to construct a parse tree for the final nullable derivative and *inj* to reconstruct the original parse tree.

The two-pass pipeline. The direct construction uses two passes. The forward pass applies `deriv` (Figure 3.3) and `nullable` (Figure 3.2) to process the input.

The backward pass applies two new functions to walk the derivation sequence in reverse and reconstruct the parse tree. First *mkEps* builds a parse tree for the empty word from the final nullable expression. Then injection steps are applied, with *inj* reinserting one consumed character into the parse tree, moving in w from right to left. Each injection step takes the current parse tree, the corresponding derivative step, and its consumed character, and returns the parse tree for the preceding expression.

As shown in Figure 5.1 after the first injection of b, the tree is $(\mathit{Right}((), b), \mathit{Right}())$, where $()$ marks the position still to be filled. After the second injection of a, that position is replaced by the subtree $\mathit{Right}(a, b)$, giving the complete parse tree $(\mathit{Right}(a, b), \mathit{Right}())$.

In the following the operations for the standard derivative parser are shown in more detail.

5.1.1 Building the Empty-Word Tree

The backward pass begins with *mkEps*, which reads a parse tree for the empty word off the final nullable expression. The function follows [Sulzmann and Lu, 2014, Fig. 3] and is defined recursively:

$$\begin{aligned}
\mathit{mkEps}_{\varepsilon} &= () && (\mathsf{E}_{\varepsilon}) \\
\mathit{mkEps}_{r^*} &= [\,] && (\mathsf{E}_{*}) \\
\mathit{mkEps}_{r_1 r_2} &= (\mathit{mkEps}_{r_1}, \mathit{mkEps}_{r_2}) && (\mathsf{E}_{\mathrm{seq}}) \\
\mathit{mkEps}_{r_1+r_2} &= \begin{cases} \mathit{Left}\ \mathit{mkEps}_{r_1} & \text{if } n(r_1) \quad (\mathsf{E}^L_+) \\ \mathit{Right}\ \mathit{mkEps}_{r_2} & \text{otherwise} \quad (\mathsf{E}^R_+) \end{cases}
\end{aligned} \tag{5.2}$$

The first three rules follow from the structure of the expression: $(\mathsf{E}_{\varepsilon})$ gives ε an empty parse tree, (E_{*}) treats a star matching the empty word as zero iterations, and $(\mathsf{E}_{\mathrm{seq}})$ matches a concatenation against ε only when both factors do. No rules are needed for $\emptyset$ or literals, since neither can match the empty word. The alternation rule is where disambiguation occurs. If exactly one of r_1 and r_2 is nullable, that branch is the only possible witness. If both are, (E^L_+) breaks the tie by preferring the left one, following POSIX's leftmost-longest policy [Sulzmann and Lu, 2014]. In the example, the final expression is:

$$(\emptyset + (\emptyset b + \varepsilon))(b + \varepsilon) + (\varepsilon + \emptyset).$$

At top-level $+$, both summands are nullable, so (E^L_+) applies and picks the left one. Within the left summand's first factor $\emptyset + (\emptyset b + \varepsilon)$, only the right alternative is nullable, so (E^R_+) applies, and within it only the right sub-alternative ε is nullable, so (E^R_+) applies again, contributing $\mathit{RightRight}()$. The second factor $b + \varepsilon$ contributes $\mathit{Right}()$ via (E^R_+), giving:

$$\mathit{Left}(\mathit{RightRight}(), \mathit{Right}()).$$

Since *mkEps* handles disambiguation, the forward pass cannot simplify. It must keep the branch that consumed a, or injection cannot recover $\mathit{Right}(a, b)$. Simplifying gives:

$$r \xrightarrow{a} (\varepsilon + \varepsilon b)(b + \varepsilon) \xrightarrow{\text{simp}} (\varepsilon + b)(b + \varepsilon) \xrightarrow{b} (\emptyset + \varepsilon)(b + \varepsilon) + (\varepsilon + \emptyset) \xrightarrow{\text{simp}} (b + \varepsilon) + \varepsilon,$$

This still witnesses $w \in L(r)$, but $(\emptyset + \varepsilon) \to \varepsilon$ and $(\varepsilon + \emptyset) \to \varepsilon$ erase the pair structure holding both characters. *mkEps* on $(b + \varepsilon) + \varepsilon$ gives $\mathit{Left}(\mathit{Right}(()))$ and injecting b fills that one empty leaf, leaving none for a, so $\mathit{Right}(a, b)$ is unrecoverable.

In Rust. The recursive definition of *mkEps* (5.2) is implemented directly in Rust, as shown in Figure 5.2, following the same cases for each expression form.

```
pub fn mk_eps(r: &Regex) -> ParseTree {
    match r {
        Regex::Eps          => ParseTree::Empty,
        Regex::Star(_)      => ParseTree::Star(Vec::new()),
        Regex::Seq(r1, r2) => ParseTree::Pair(
                               Box::new(mk_eps(r1)), Box::new(mk_eps(r2))),
        Regex::Alt(r1, r2) =>
            if nullable(r1)   {ParseTree::Left(Box::new(mk_eps(r1)))}
            else              {ParseTree::Right(Box::new(mk_eps(r2)))},
        Regex::Lit(c)       => panic!("mk_eps called on Lit('{}')", c),
        Regex::Phi          => panic!("mk_eps called on Phi"),
    }
}
```

Figure 5.2: `mk_eps`: construct a parse tree for the empty word.

The expression r is borrowed rather than consumed because the backward pass needs it again during injection. The alternation arm encodes disambiguation as a conditional, examining `r2` only when `nullable(r1)` is false. The panic arms enforce the precondition that *mkEps* applies only to nullable expressions, rejecting the non-nullable $\emptyset$ and literals.

5.1.2 Reinserting Characters via Injection

After *mkEps* constructed the parse tree v for the empty input, the reconstruction starts from this initial tree and proceeds backwards through the derivation sequence. Each application of *inj* takes the current tree, the corresponding derivative step, and its consumed character, returning a tree for the preceding expression. *inj* recursively follows the structure of the residual until it reaches the subexpression containing the literal that consumed the character. It reinserts the character there and then unwinds to rebuild the surrounding tree. After all characters have been reinserted, the parse tree for the original expression and input is recovered. The injection function is defined by the following rules [Sulzmann and Lu, 2014, Fig. 3]:

$$
\begin{aligned}
inj_{x\backslash x}() &= x && (\mathsf{I}_{\text{lit}}) \\
inj_{(r_1+r_2)\backslash x}(\mathit{Left}\ v_1) &= \mathit{Left}\ (inj_{r_1\backslash x}\ v_1) && (\mathsf{I}^L_+) \\
inj_{(r_1+r_2)\backslash x}(\mathit{Right}\ v_2) &= \mathit{Right}\ (inj_{r_2\backslash x}\ v_2) && (\mathsf{I}^R_+) \\
inj_{r^*\backslash x}(v, vs) &= (inj_{r\backslash x}\ v) : vs && (\mathsf{I}_*) \\
inj_{(r_1 r_2)\backslash x}(v_1, v_2) &= (inj_{r_1\backslash x}\ v_1,\ v_2) && (\mathsf{I}_{\text{seq}}),\ n(r_1) = \text{false} \\
inj_{(r_1 r_2)\backslash x}(\mathit{Left}(v_1, v_2)) &= (inj_{r_1\backslash x}\ v_1,\ v_2) && (\mathsf{I}^L_{\text{seq}}),\ n(r_1) = \text{true} \\
inj_{(r_1 r_2)\backslash x}(\mathit{Right}\ v_2) &= (mkEps_{r_1},\ inj_{r_2\backslash x}\ v_2) && (\mathsf{I}^R_{\text{seq}}),\ n(r_1) = \text{true}
\end{aligned}
\tag{5.3}
$$

Rules $(\mathsf{I}_{\text{lit}})$ and (I_*) invert their derivative rules directly, reconstructing the consumed character and the current iteration, the latter prepended to the remaining ones. Since the consumed character lies in exactly one branch of an alternation, the incoming tree is either *Left* v_1 or *Right* v_2. Rule (I^L_+) recurses into v_1 and rebuilds the *Left* node, and (I^R_+) recurses

into v_2 and rebuilds the *Right* node. The concatenation rules cover the three cases of the sequence derivative. If $n(r_1) = \text{false}$, r_1 cannot match ε and must consume x, so (I_{seq}) recurses into v_1. If $n(r_1) = \text{true}$, the derivative recorded which component consumed x as a tag: ($\mathsf{I}^L_{\text{seq}}$) handles an incoming $\mathit{Left}(v_1, v_2)$ tag and recurses into v_1, while ($\mathsf{I}^R_{\text{seq}}$) handles a *Right* v_2 tag by reconstructing r_1's empty match with mkEps_{r_1} and recursing into v_2. Since $\emptyset$ consumes no characters, *inj* is never applied with $\emptyset$ as the residual.

```
pub fn inject(r: &Regex, l: char, v: ParseTree) -> ParseTree {
    match r {
        Regex::Lit(c) => { assert!(*c == l);
            ParseTree::Char(l) }
        Regex::Alt(r1, r2) => match v {
            ParseTree::Left(v1) =>
                ParseTree::Left(Box::new(inject(r1, l, *v1))),
            ParseTree::Right(v2) =>
                ParseTree::Right(Box::new(inject(r2, l, *v2))),
            _ => panic!("inject/Alt: want Left|Right"),
        },
        Regex::Star(r1) => match v {
            ParseTree::Pair(v1, vs) => {
                let mut it = vec![inject(r1, l, *v1)];
                if let ParseTree::Star(rest) = *vs { it.extend(rest); }
                ParseTree::Star(it)
            }
            _ => panic!("inject/Star: want Pair"),
        },
        Regex::Seq(r1, r2) => if !nullable(r1) {
            match v {
                ParseTree::Pair(v1, v2) => ParseTree::Pair(
                    Box::new(inject(r1, l, *v1)), v2),
                _ => panic!("inject/Seq: want Pair"),
            }
        } else {
            match v {
                ParseTree::Left(vp) => match *vp {
                    ParseTree::Pair(v1, v2) => ParseTree::Pair(
                        Box::new(inject(r1, l, *v1)), v2),
                    o => panic!("inject/SeqL: want Pair, got {:?}", o),
                },
                ParseTree::Right(v2) => ParseTree::Pair(
                    Box::new(mk_eps(r1)),
                    Box::new(inject(r2, l, *v2))),
                _ => panic!("inject/Seq: want Left|Right"),
            }
        },
        Regex::Eps | Regex::Phi => panic!("inject called on Eps/Phi"),
    }
}
```

Figure 5.3: `inject`: reinsert a character into a parse tree.

In Rust. `inject(r, l, v)` takes three arguments because *inj* is indexed by both an expression and a character. Figure 5.3 shows the complete implementation.

The function matches on the expression `r` first and on the tree `v` second, following the order in which (5.3) indexes its rules. This order matters because a tree tag is interpreted relative to the expression that produced it. Under an alternation, `Left` selects the left disjunct, while under a nullable concatenation it indicates that the derivative continued inside the left operand. The `Lit` arm reconstructs a leaf after asserting that the consumed character matches, and the `Star` arm inverts (I_*), reinserting the character into the current iteration and prepending it to the remaining ones.

The expression `r` is borrowed and read only, while the tree `v` is consumed and decomposed. In the concatenation arms where the right operand was not derived, `v2` moves directly into the new pair, since (5.3) leaves it unchanged. The `Right` arm of concatenation is the only concatenation case that synthesizes content, calling `mk_eps(r1)` because the derivative skipped `r1` and the forward pass produced no tree for that half of the pair.

Calling `nullable(r1)` in the `Seq` arm recomputes a decision the forward pass already made when it chose between D_{seq} and $\mathsf{D}^n_{\text{seq}}$. However, carrying this as a flag at every step would require additional storage, while `nullable` is cheap to evaluate on demand.

5.1.3 Parser

The standard parser has two passes. The forward pass stores each residual and the backward pass applies *mkEps* then *inj* in reverse. The split is needed because *inj* requires the pre-character residual, which only the stored sequence provides:

$$\begin{aligned} r_0 &= r, & r_i &= r_{i-1}\backslash x_i & (1 \le i \le n),\\ v_n &= \mathit{mkEps}(r_n), & v_i &= \mathit{inj}_{r_{i-1}\backslash x_i}(x_i,\ v_{i+1}) & (0 \le i < n). \end{aligned} \tag{5.4}$$

Every r_i is kept because each injection step needs r_{i-1}; re-deriving it would duplicate the forward pass. The vector holds the entire sequence $r_0, \ldots, r_n$, not just the final residual.

In Rust. Figure 5.4 implements (5.4). The forward loop fills the `Vec expressions` with $r_0, \ldots, r_n$, the nullability check returns early if r_n is not nullable, and the backward loop walks the vector in reverse, applying `inject` to each residual and its character.

```
pub fn parse_deriv_std_loop(input: &str, r: &Regex) -> Option<ParseTree>{
    let chars: Vec<char> = input.chars().collect();
    let mut expressions = Vec::with_capacity(chars.len() + 1);
    expressions.push(r.clone());
    for &c in &chars {
        expressions.push(deriv(expressions.last().unwrap(), c));
    }
    if !nullable(expressions.last().unwrap()) { return None; }
    let mut tree = mk_eps(expressions.last().unwrap());
    for i in (0..chars.len()).rev() {
        tree = inject(&expressions[i], chars[i], tree);
    }
    Some(tree)
}
```

Figure 5.4: `parse_deriv_std_loop`: the standard parser.

5.1.4 Worked Example: Applying the Standard Parser

Apply the standard parser to the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ on input $\mathtt{ab}$.

Forward pass. There are two derivative steps, one per input symbol. After deriving $\mathtt{a}$, the left factor $\varepsilon + \varepsilon b$ is nullable, so the second derivation on $\mathtt{b}$ uses $\mathsf{D}^n_{\text{seq}}$ rather than D_{seq}:

$$\begin{aligned} r_0 = (a + ab)(b + \varepsilon) &\xrightarrow[\mathsf{D}_{\text{seq}}]{a} r_1 = (\varepsilon + \varepsilon b)(b + \varepsilon) \\ &\xrightarrow[\mathsf{D}^n_{\text{seq}}]{b} r_2 = \underbrace{(\emptyset + (\emptyset b + \varepsilon))(b + \varepsilon)}_{X} + \underbrace{(\varepsilon + \emptyset)}_{Y}. \end{aligned}$$

The final expression $r_2 = X + Y$ is nullable, and both disjuncts accept the empty word. X records $ab \cdot \varepsilon$ (the first factor matched ab, the second ε); Y records $a \cdot b$ (the first matched a, the second b).

Backward pass: mkEps. The backward pass begins by building the parse tree for the empty word, $mkEps(r_2)$. Since $r_2 = X + Y$ and both disjuncts are nullable, E^L_+ selects the left one:

$$mkEps(r_2) = Left\Big(mkEps(X)\Big) \qquad (\mathsf{E}^L_+),$$

where $X = (\emptyset + (\emptyset b + \varepsilon))(b + \varepsilon)$. Since X is a concatenation, E_{seq} combines the trees for its two factors:

$$mkEps(X) \xrightarrow{\mathsf{E}_{\text{seq}}} \Big(mkEps(\emptyset + (\emptyset b + \varepsilon)),\ mkEps(b + \varepsilon)\Big).$$

The first factor selects ε by applying E^R_+ twice, since $\emptyset$ and $\emptyset b$ are not nullable; the second factor selects ε by E^R_+, since b is not nullable. These give

$$\begin{aligned} mkEps(\emptyset + (\emptyset b + \varepsilon)) &\xrightarrow{\mathsf{E}^R_+} Right\ mkEps(\emptyset b + \varepsilon) \\ &\xrightarrow{\mathsf{E}^R_+} Right\ Right\ mkEps(\varepsilon) \\ &\xrightarrow{\mathsf{E}_\varepsilon} Right\ Right\ () \end{aligned} \qquad \begin{aligned} mkEps(b + \varepsilon) &\xrightarrow{\mathsf{E}^R_+} Right\ mkEps(\varepsilon) \\ &\xrightarrow{\mathsf{E}_\varepsilon} Right\ () \end{aligned}$$

and hence

$$v_2 = mkEps(r_2) = Left\Big(Right\ Right\ (), Right\ ()\Big).$$

Backward pass: Inj. Reinsertion begins with $\mathtt{b}$ and maps v_2 to a tree for $r_1 = (\varepsilon + \varepsilon b)(b + \varepsilon)$. The *Left* tag of v_2 sends $\mathsf{I}^L_{\text{seq}}$ into the first factor $\varepsilon + \varepsilon b$, leaving the second factor's tree $Right\ ()$ unchanged. Inside $\varepsilon + \varepsilon b$, I^R_+ selects the ab branch; since ε is nullable, $\mathsf{I}^R_{\text{seq}}$ supplies the empty tree for ε before I_{lit} reinserts $\mathtt{b}$:

$$\begin{aligned} inj_{r_1 \backslash \mathtt{b}}(v_2) = inj_{r_1 \backslash \mathtt{b}}\Big(\underbrace{Left(Right\ Right\ (),\ Right\ ())}_{v_2}\Big) \\ \xrightarrow{\mathsf{I}^L_{\text{seq}}} \Big(inj_{(\varepsilon + \varepsilon b) \backslash \mathtt{b}}(Right\ Right\ ()),\ Right\ ()\Big) \\ \xrightarrow{\mathsf{I}^R_+} \Big(Right\ inj_{\varepsilon b \backslash \mathtt{b}}(Right\ ()),\ Right\ ()\Big) \\ \xrightarrow{\mathsf{I}^R_{\text{seq}}} \Big(Right\ (mkEps(\varepsilon),\ inj_{b \backslash \mathtt{b}}()),\ Right\ ()\Big) \\ \xrightarrow{\mathsf{E}_\varepsilon,\ \mathsf{I}_{\text{lit}}} \underbrace{\Big(Right((), b),\ Right\ ()\Big)}_{v_1}. \end{aligned}$$

Reinsertion of $\mathtt{a}$ maps v_1 to a tree for $r_0 = (a + ab)(b + \varepsilon)$. The outer sequence's first factor $a+ab$ is not nullable, so I_{seq} fires directly on v_1's first component, leaving the second component $\mathit{Right}\,()$ untouched. That first component, $\mathit{Right}((), b)$, carries a Right tag, so I^R_+ selects ab. Inside $ab = \mathit{Seq}(a, b)$ the first factor a is again not nullable, so I_{seq} fires once more and recurses into a, where I_{lit} reinserts the character:

$$
\begin{aligned}
\mathit{inj}_{r_0\backslash\mathtt{a}}(v_1) &= \mathit{inj}_{r_0\backslash\mathtt{a}}\Big(\underbrace{\big(\mathit{Right}((), b),\ \mathit{Right}\,()\big)}_{v_1}\Big) \\
&\xrightarrow{\mathsf{I}_{\text{seq}}} \Big(\mathit{inj}_{(a+ab)\backslash\mathtt{a}}(\mathit{Right}((), b)),\ \mathit{Right}\,()\Big) \\
&\xrightarrow{\mathsf{I}^R_+} \Big(\mathit{Right}\,\mathit{inj}_{ab\backslash\mathtt{a}}(((), b)),\ \mathit{Right}\,()\Big) \\
&\xrightarrow{\mathsf{I}_{\text{seq}}} \Big(\mathit{Right}\,(\mathit{inj}_{a\backslash\mathtt{a}}(()),\ b),\ \mathit{Right}\,()\Big) \\
&\xrightarrow{\mathsf{I}_{\text{lit}}} \underbrace{\big(\mathit{Right}(a, b),\ \mathit{Right}\,()\big)}_{v_0}.
\end{aligned}
$$

The implementation stores r_0, r_1, and r_2 in its vector. One entry per character, plus the initial expression, kept for the backward pass to reconstruct from. The tree it returns is $(\mathit{Right}(a, b), \mathit{Right}())$, which flattens to $\mathtt{ab}$ and matches the input.

5.2 The Bit-Coded Derivative-Based Parser

The direct construction of Sulzmann and Lu [Sulzmann and Lu, 2014] reconstructs the tree only after the full input, since each injection step needs the preceding expression. Their bit-coded optimization records this information during a single pass instead.

Each subexpression carries the bit-sequence it has committed to so far, written $bs@r$. P and Q denote the annotated factors for $r_0 = (a + ab)(b + \varepsilon)$. Deriving $w = \mathtt{ab}$ gives:

$$
\begin{aligned}
ri_0 &= [\,]@\Big(\underbrace{[0]@a \oplus [1]@(a\,b)}_{P} \cdot \underbrace{[0]@b \oplus [1]@\varepsilon}_{Q}\Big) \\
&\xrightarrow{a} ri_1 = [\,]@\Big(\underbrace{[0]@\varepsilon \oplus [1]@b}_{P'} \cdot Q\Big) \\
&\xrightarrow{b} ri_2 = [\,]@\Big([1]@Q \ \oplus\ [0, 0]@\varepsilon\Big).
\end{aligned}
\tag{5.5}
$$

Figure 5.5 shows this derivation together with the extraction step it feeds into:

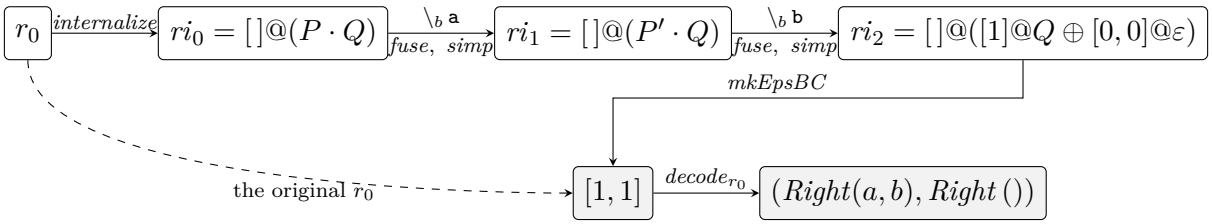


Figure 5.5: The bit-coded parser on the ambiguous example. *internalize* introduces the bits, derivative threads them forward, and *mkEpsBC* and $decode_{r_0}$ extract and read them against r_0.

The bit-coded pipeline. *internalize* tags each alternation branch with a bit that records its preference, producing $ri_0 = []@(P \cdot Q)$ with $P = [0]@a \oplus [1]@(a\,b)$ and $Q = [0]@b \oplus [1]@\varepsilon$, where 0 selects the left branch and 1 the right.

Deriving by `a` extracts P's leading a, giving P' while Q is untouched; deriving by `b` consumes the rest. Since P''s ε branch is nullable, both decompositions survive: in the first, the derivative continues inside P', which finishes and folds its bit 1 onto Q; in the second, Q matches `b` instead, and P''s completion bit fuses with Q's match into $[0,0]@\varepsilon$. This yields $ri_2 = []@([1]@Q \oplus [0,0]@\varepsilon)$.

Unlike the standard construction, the bit-coded construction requires simplification to prevent unbounded growth of annotated expressions. The function *simp*, from [Sulzmann and Lu, 2014, Fig. 6], discards dead branches and collapses finished subexpressions, while *fuse* preserves bits on surviving nodes, as when P collapses into P' and then onto Q.

Once `ab` is exhausted, *mkEpsBC* walks ri_2 the way *mkEps* walks a plain expression, taking the leftmost nullable branch at each remaining choice: $[1]@Q$ is nullable, so it contributes its 1, and inside Q the ε branch is nullable, contributing a second 1, giving $bits = [1,1]$. $decode_{r_0}$ then replays these two bits against the original r_0. The first 1 selects $a+ab$'s right branch and the second selects $b+\varepsilon$'s right branch, giving the tree $(Right(a,b), Right\,())$.

This shows that the bit-coded construction reconstructs the parse tree in a single step, while the direct construction reverses residuals step by step.

5.2.1 Annotated Expressions

Building parse-tree information during the forward pass extends the derivative computation with bit-sequence annotations, propagated by the bit-threaded derivative $ri\backslash_b l$. An annotated expression ri extends r by associating each non-$\emptyset$ node with a bit-sequence bs recording the reconstruction choices made during the forward pass.

$$ri ::= \emptyset \mid (bs@\varepsilon) \mid (bs@x) \mid (bs@ri_1 \oplus ri_2) \mid (bs@ri_1\,ri_2) \mid (bs@ri^*). \tag{5.6}$$

In Rust. Figure 5.6 defines `ARegex`, the annotated counterpart of `Regex`. Every variant except `Phi` carries a leading `Bits` field (cf. (5.6)). `Bits` is a persistent, `Rc`-backed cons list, not a `Vec<bool>`: prepending a prefix costs only the prefix, independent of the accumulated history. A `Vec<bool>` would copy the entire prefix on every derivative step, making an n-character run cost $O(n^2)$. the persistent representation keeps it $O(n)$.

```
pub enum ARegex {
    Phi,
    Eps(Bits), Lit(Bits, char),
    Alt(Bits, Box<ARegex>, Box<ARegex>),
    Seq(Bits, Box<ARegex>, Box<ARegex>),
    Star(Bits, Box<ARegex>),
}
```

Figure 5.6: The annotated-expression type extends `Regex` with a leading bit-sequence, `Bits`.

All following operations act on this `ARegex`. Since derivative and simplification steps remove nodes while the bits attached to them must survive, each operation needs a way to transfer those bits onto the node that remains. The helper *fuse* provides this transfer.

5.2.2 Fusing Bit-Lists

fuse, defined in [Sulzmann and Lu, 2014, Fig. 5], transfers bits from a node that is about to be removed onto the node that survives in its place. It prepends bs to the top node's annotation bs' without descending into its children, so outer reconstruction choices precede those already recorded in the subtree.

$$\begin{aligned} \mathit{fuse}(bs, \emptyset) &= \emptyset \\ \mathit{fuse}(bs, (bs'@\varepsilon)) &= (bs +\!\!+ bs')@\varepsilon \\ \mathit{fuse}(bs, (bs'@x)) &= (bs +\!\!+ bs')@x \\ \mathit{fuse}(bs, (bs'@ri_1 \oplus ri_2)) &= (bs +\!\!+ bs')@(ri_1 \oplus ri_2) \\ \mathit{fuse}(bs, (bs'@ri_1\, ri_2)) &= (bs +\!\!+ bs')@(ri_1\, ri_2) \\ \mathit{fuse}(bs, (bs'@ri^*)) &= (bs +\!\!+ bs')@ri^*. \end{aligned} \tag{5.7}$$

In Rust. Figure 5.7 implements the six rules of (5.7), one per variant. The helper `Bits::prepend` corresponds to the concatenation $+\!\!+$ and places `bs` before the node's existing bits:

```
pub fn fuse(bs: &[bool], ri: ARegex) -> ARegex {
    if bs.is_empty() { return ri; }
    match ri {
        ARegex::Phi => ARegex::Phi,
        ARegex::Eps(p) => ARegex::Eps(p.prepend(bs)),
        ARegex::Lit(p, c) => ARegex::Lit(p.prepend(bs), c),
        ARegex::Alt(p, r1, r2) => ARegex::Alt(p.prepend(bs), r1, r2),
        ARegex::Seq(p, r1, r2) => ARegex::Seq(p.prepend(bs), r1, r2),
        ARegex::Star(p, r) => ARegex::Star(p.prepend(bs), r),
    }
}

impl Bits {
    // prefix ++ self, O(prefix.len()) regardless of how long self
    // already is -- the existing chain is shared, not copied.
    pub fn prepend(&self, prefix: &[bool]) -> Bits {
        if prefix.is_empty() { return self.clone(); }
        Bits(Rc::new(Node::Prepend(Rc::from(prefix), self.0.clone())))
    }
}
```

Figure 5.7: `fuse`: prepend a bit-list to the annotation of the top node.

The `Phi` arm is the only case that returns without calling `prepend`, because `Phi` represents the empty language and carries no annotation slot, so there is no bit-list for `fuse` to prepend to. Every other arm rebuilds the top node with a combined annotation and moves its two children, or single child in the star case, unchanged. Since `fuse` prepends `bs` only to the top node's annotation, the bits carried by its children are not touched, and the operation rewrites exactly one node regardless of how large the subtree below it is.

The function is therefore non-recursive, and no subtree needs to be copied. The implementation realizes this directly, since `ri` is consumed by value and its untouched children

move into the rebuilt node without a clone. `Bits::prepend` wraps the node's existing chain in a new `Rc`, sharing rather than copying it: the allocation is proportional only to `bs`'s length, independent of how many bits the node already carries. This is why `Bits` replaces a plain `Vec<bool>` here: with a `Vec`, `prepend` would have to copy the existing content on every call, a cost proportional to the combined length of the two bit-lists – the node's own history can grow with every character consumed so far, which would make a single `fuse` call as expensive as the whole parse up to that point. An empty `bs` is the one case that still skips any work, since prepending an empty prefix returns the node's existing chain unchanged.

5.2.3 Internalizing Expressions

The forward pass operates on annotated expressions, but the input is a plain expression, so r must first be converted into its annotated counterpart ri. The function *internalize*, following [Sulzmann and Lu, 2014, Fig. 5], performs this conversion recursively, and it is the only step that introduces new bits:

$$\begin{aligned}
\mathit{internalize}(\emptyset) &= \emptyset \\
\mathit{internalize}(\varepsilon) &= [\,]@\varepsilon \\
\mathit{internalize}(x) &= [\,]@x \\
\mathit{internalize}(r_1 + r_2) &= [\,]@\Big(\mathit{fuse}([0], \mathit{internalize}(r_1)) \oplus \mathit{fuse}([1], \mathit{internalize}(r_2))\Big) \\
\mathit{internalize}(r_1\, r_2) &= [\,]@\Big(\mathit{internalize}(r_1)\ \mathit{internalize}(r_2)\Big) \\
\mathit{internalize}(r^*) &= [\,]@\mathit{internalize}(r)^*.
\end{aligned} \tag{5.8}$$

The tags [0] and [1] on the two branches of $r_1 + r_2$ mark the left and right branch respectively, so the decoder can later recover which one was taken. Every other case introduces no new bit and uses only the empty list $[\,]$. The two calls to *fuse* in (5.8) are the only points in the construction where new bits are introduced.

In Rust. Figure 5.8 directly maps the constructors of `Regex` to their annotated counterparts. Each recursive call applies `internalize` to the corresponding subexpression, preserving the structure of the original expression in the annotated representation.

```
pub fn internalize(r: &Regex) -> ARegex {
    match r {
        Regex::Phi => ARegex::Phi,
        Regex::Eps => ARegex::Eps(Bits::empty()),
        Regex::Lit(c) => ARegex::Lit(Bits::empty(), *c),
        Regex::Alt(r1, r2) => {
            let ri1 = fuse(&[false], internalize(r1));
            let ri2 = fuse(&[true], internalize(r2));
            ARegex::Alt(Bits::empty(), Box::new(ri1), Box::new(ri2))
        }
        Regex::Seq(r1, r2) => ARegex::Seq(Bits::empty(),
            Box::new(internalize(r1)), Box::new(internalize(r2))),
        Regex::Star(r1) => ARegex::Star(Bits::empty(),
            Box::new(internalize(r1))),
    }
}
```

Figure 5.8: `internalize` converts a `Regex` into an `ARegex`. Each alternation branch receives a tag recording its side, and the two `fuse` calls are the only points where new bits are introduced.

The `Alt` case calls `fuse` with a non-empty bit-list, tagging the left operand with `[false]` and the right with `[true]` before wrapping them in a `Alt` with an empty outer bit-list. Every later bit originates from one of these two tags and is propagated during derivation.

The `Phi`, `Eps`, `Lit`, `Seq`, and `Star` cases use an empty bit-list, since they introduce no disambiguation choice at this stage. Star iteration choices are instead determined during derivative computation.

5.2.4 Extracting Bits

mkEpsBC, from [Sulzmann and Lu, 2014, Fig. 5], is the bit-list counterpart of *mkEps*. It keeps the left preference and star closing bit, but extracts bits instead of building a tree. Each rule prepends the current node's bits to those returned recursively.

$$
\begin{aligned}
mkEpsBC_{(bs@\varepsilon)} &= bs \\
mkEpsBC_{(bs@ri^*)} &= bs +\!\!+ [1] \\
mkEpsBC_{(bs@ri_1\, ri_2)} &= bs +\!\!+ mkEpsBC_{ri_1} +\!\!+ mkEpsBC_{ri_2} \\
mkEpsBC_{(bs@ri_1 \oplus ri_2)} &= bs +\!\!+ \begin{cases} mkEpsBC_{ri_1} & \varepsilon \in L(ri_1) \\ mkEpsBC_{ri_2} & \text{otherwise.} \end{cases}
\end{aligned}
\tag{5.9}
$$

As with *mkEps*, the rules cover only the four nullable forms. $\emptyset$ and literals cannot match the empty word, so *mkEpsBC* is called only on nullable expressions and both are handled as preconditions.

In Rust. Figure 5.9 implements `mk_eps_bc` alongside `mk_eps` (Figure 5.2), with one difference. Where `mk_eps` builds a `ParseTree`, `mk_eps_bc` concatenates bit-lists, calling `bs.to_vec()` where `mk_eps` would borrow or clone: this is the point where a node's `Bits` chain (Section 5.2.1) is finally walked and materialized into a concrete `Vec<bool>`,

something `fuse` deliberately avoids doing on every derivative step. The `Star` arm also does not recurse, since a star matching the empty word uses zero iterations, so it only appends the closing `true` and never examines the child expression:

```
pub fn mk_eps_bc(ri: &ARegex) -> Vec<bool> {
    match ri {
        ARegex::Phi => panic!("mk_eps_bc called on Phi"),
        ARegex::Lit(_, c) => panic!("mk_eps_bc called on Lit('{}')", c),
        ARegex::Eps(bs) => bs.to_vec(),
        ARegex::Alt(bs, r1, r2) => {
            let mut result = bs.to_vec();
            if nullable_bc(r1) {
                result.extend(mk_eps_bc(r1));
            } else {
                result.extend(mk_eps_bc(r2));
            }
            result
        }
        ARegex::Seq(bs, r1, r2) => {
            let mut result = bs.to_vec();
            result.extend(mk_eps_bc(r1));
            result.extend(mk_eps_bc(r2));
            result
        }
        ARegex::Star(bs, _) => {
            let mut result = bs.to_vec();
            result.push(true);
            result
        }
    }
}
```

Figure 5.9: `mk_eps_bc` implements (5.9). `Phi` and `Lit` panic, since neither can be nullable.

5.2.5 Annotated Derivatives

$ri\backslash_b l$ has the same case structure as the plain derivative (3.6), but extended with bits threaded through the rules. Each rule corresponds to its unannotated counterpart:

$$
\begin{aligned}
\emptyset\backslash_b x &= \emptyset && (\mathsf{B}_\emptyset)\\
(bs@\varepsilon)\backslash_b x &= \emptyset && (\mathsf{B}_\varepsilon)\\
(bs@c)\backslash_b x &= \begin{cases} bs@\varepsilon & c = x \\ \emptyset & \text{otherwise} \end{cases} && (\mathsf{B}_{\text{lit}})\\
(bs@ri_1 \oplus ri_2)\backslash_b x &= (bs@(ri_1\backslash_b x) \oplus (ri_2\backslash_b x)) && (\mathsf{B}_+)\\
(bs@ri_1\ ri_2)\backslash_b x &= (bs@(ri_1\backslash_b x)\ ri_2) && (\mathsf{B}_{\text{seq}}),\ \varepsilon \notin L(ri_1)\\
(bs@ri_1\ ri_2)\backslash_b x &= bs@\Big(([\,]@(ri_1\backslash_b x)\ ri_2) \oplus \mathit{fuse}(\mathit{mkEpsBC}_{ri_1}, ri_2\backslash_b x)\Big) && (\mathsf{B}^n_{\text{seq}}),\ \varepsilon \in L(ri_1)\\
(bs@ri^*)\backslash_b x &= (bs@\mathit{fuse}([0], ri\backslash_b x)\ ([\,]@ri^*)) && (\mathsf{B}_*).
\end{aligned}
\tag{5.10}
$$

The rules B_{+}, B_{seq}, and B_{*} preserve the outer bs, since derivation changes the residual but not the choices already recorded above it.

The nullable sequence case $\mathsf{B}^{n}_{\text{seq}}$ is the exception. Since ri_1 is nullable, x may be matched by either factor, so the rule introduces a new alternation. Its left branch continues into ri_1, while its right branch recurses into ri_2, with the empty match's bits extracted by *mkEpsBC* and fused onto ri_2's derivative, so the choice inside ri_1 survives. This performs during the forward pass what $\mathsf{I}^{R}_{\text{seq}}$ performs during injection. The outer bs attaches to the new alternation, since the alternation as a whole is what $ri_1\ ri_2$ derived into.

In Rust. Figure 5.10 implements (5.10), following the case structure of plain `deriv` with bits threaded through the constructors. The base cases `Phi`, `Eps`, and `Lit` match plain `deriv`, except that `Lit` carries its bit-list onto the resulting `Eps`.

The `Alt` arm derives both operands and rebuilds the alternation with the outer `bs` on the wrapper, not pushed into either branch. This matches B_{+}, where the recorded choice belongs to the alternation itself and derivation does not change it, so the bit stays on the outer constructor until the decoder reads it.

The `Seq` arm reflects B_{seq} and $\mathsf{B}^{n}_{\text{seq}}$. In the non-nullable case, `r1` must consume the character, so only its derivative is needed and `r2` moves directly into the rebuilt sequence. In the nullable case, both alternatives are possible. The bits of `r1`'s empty match are extracted by `mk_eps_bc` *before* `r1` is consumed by `deriv_bc`, since after that call the original `r1` is gone. Those bits are then fused onto the derivative of `r2`, so the choice made inside `r1` reaches the surviving branch. The left branch is wrapped in a fresh `Seq` with an empty bit-list, since the outer `bs` moves to the new `Alt` wrapper, matching $\mathsf{B}^{n}_{\text{seq}}$. Two clones are therefore needed. `r2.clone()` provides one copy for the left branch and one for its own derivative, and `nullable_bc` borrows `r1` before `deriv_bc` consumes it.

The `Star` arm follows the same pattern as the nullable case of `Seq`. The body is needed both for its derivative and for the reconstructed star, so `r.clone()` provides the second copy, and the reconstructed star is wrapped in a fresh `Star` with an empty bit-list for the same reason as the left branch above. The leading `[false]` prepended to the derivative marks one more iteration, matching the outermost bit in B_{*}

```
pub fn deriv_bc(ri: ARegex, l: char) -> ARegex {
    match ri {
        // ...
        ARegex::Alt(bs, r1, r2) => {
            let d1 = deriv_bc(*r1, l);
            let d2 = deriv_bc(*r2, l);
            ARegex::Alt(bs, Box::new(d1), Box::new(d2))
        }
        ARegex::Seq(bs, r1, r2) => {
            if nullable_bc(&r1) {
                let eps_bits = mk_eps_bc(&r1);
                let d1 = deriv_bc(*r1, l);
                let d2 = deriv_bc(*r2.clone(), l);
                let left_branch = ARegex::Seq(Bits::empty(), Box::new(d1), r2);
                let right_branch = fuse(&eps_bits, d2);
                ARegex::Alt(bs, Box::new(left_branch), Box::new(right_branch))
            } else {
                let d1 = deriv_bc(*r1, l);
                ARegex::Seq(bs, Box::new(d1), r2)
            }
        }
        ARegex::Star(bs, r) => {
            let d = deriv_bc(*r.clone(), l);
            let fused = fuse(&[false], d);
            let new_star = ARegex::Star(Bits::empty(), r);
            ARegex::Seq(bs, Box::new(fused), Box::new(new_star))
        }
    }
}
```

Figure 5.10: Annotated derivative `deriv_bc`, implementing $\mathsf{B}_{\emptyset}$, B_{ε}, B_{lit}, B_{+}, $\mathsf{B}_{\text{seq}}/\mathsf{B}^{n}_{\text{seq}}$, B_{*}.

5.2.6 Decoding Bits

$decode_r$ reconstructs a `ParseTree` from the bit sequence of $mkEpsBC$, walking r with one bit per alternation to choose a branch. Its worker $decodeInner$ returns the current tree and the remaining bits, so the recursion continues at the correct position:

$$
\begin{aligned}
decode_r(bs) &= v \quad \text{if } decodeInner_r(bs) = (v, [\,]) \\[2ex]
decodeInner_{\varepsilon}(bs) &= (Empty,\ bs) \\
decodeInner_{x}(bs) &= (Char(x),\ bs) \\
decodeInner_{r_1+r_2}(0{:}bs) &= (Left(v),\ bs_1) \quad \text{if } decodeInner_{r_1}(bs) = (v, bs_1) \\
decodeInner_{r_1+r_2}(1{:}bs) &= (Right(v),\ bs_1) \quad \text{if } decodeInner_{r_2}(bs) = (v, bs_1) \\
decodeInner_{r_1 r_2}(bs) &= (Pair(v_1, v_2),\ bs_2) \\
&\qquad \text{if } decodeInner_{r_1}(bs) = (v_1, bs_1) \text{ and } decodeInner_{r_2}(bs_1) = (v_2, bs_2) \\
decodeInner_{r^*}(1{:}bs) &= (Star([\,]),\ bs) \\
decodeInner_{r^*}(0{:}bs) &= (Star(v{:}vs),\ bs_2) \\
&\qquad \text{if } decodeInner_{r}(bs) = (v, bs_1) \text{ and } decodeInner_{r^*}(bs_1) = (Star(vs), bs_2)
\end{aligned}
\tag{5.11}
$$

In Rust. Figure 5.11 shows `decode_inner`, the worker behind `decode`. The public entry point `decode` wraps it and asserts that no bits remain. This assertion is the side condition under which $decode_r$ is defined. The worker must consume the entire bit-string.

The worker consumes the `&[bool]` one bit per step using Rust's slice patterns: `[false, rest @ ..]` matches a bit-string whose first element is `false` and binds the remainder to `rest`, and `[true, rest @ ..]` does the same for `true`.

```
fn decode_inner<'a>(r: &Regex, bs: &'a [bool]) -> (ParseTree, &'a [bool]) {
    match r {
        Regex::Eps => (ParseTree::Empty, bs),
        Regex::Lit(c) => (ParseTree::Char(*c), bs),
        Regex::Alt(r1, r2) => match bs {
            [false, rest @ ..] => {
                let (v, remaining) = decode_inner(r1, rest);
                (ParseTree::Left(Box::new(v)), remaining)
            }
            [true, rest @ ..] => {
                let (v, remaining) = decode_inner(r2, rest);
                (ParseTree::Right(Box::new(v)), remaining)
            }
            [] => panic!("decode: unexpected end of bits at Alt"),
        },
        Regex::Seq(r1, r2) => {
            let (v1, after_r1) = decode_inner(r1, bs);
            let (v2, after_r2) = decode_inner(r2, after_r1);
            (ParseTree::Pair(Box::new(v1), Box::new(v2)), after_r2)
        }
        Regex::Star(r1) => {
            let mut iterations = Vec::new();
            let mut remaining = bs;
            loop {
                match remaining {
                    [true, rest @ ..] => {
                        remaining = rest;
                        break;
                    }
                    [false, rest @ ..] => {
                        let (v, after) = decode_inner(r1, rest);
                        iterations.push(v);
                        remaining = after;
                    }
                    [] => panic!("decode: unexpected end of bits inside Star"),
                }
            }
            (ParseTree::Star(iterations), remaining)
        }
        Regex::Phi => panic!("decode: called on Phi"),
    }
}
```

Figure 5.11: `decode_inner`: the worker behind `decode`, one arm per expression form.

The `Alt` arm matches the leading bit with `[false, rest @ ..]` or `[true, rest @ ..]` and recurses into the branch it names, returning `rest` as the remaining bits, since only the leading bit belongs to the alternation.

The `Seq` arm shows why the worker returns a pair. It recurses into the first factor, receives both a tree and the bits the first factor left unconsumed, and passes those bits to the second factor. The second factor then returns its own tree and the bits that follow it, which are the bits for any enclosing expression. Returning only the tree would leave the second recursion with no way to know where to start.

The `Star` arm loops, since the number of iterations is not known in advance and is itself encoded in the bits. Each leading `false` is matched by `[false, rest @ ..]` and corresponds to one iteration, so the arm recurses into the body and continues the loop from whatever bits the body left behind. The loop ends at the leading `true` matched by `[true, rest @ ..]`, which `mk_eps_bc` appended.

5.2.7 Parser

The bit-coded parser performs three steps on the input. First, *internalize* converts the pattern r into an annotated expression ri. Then, for each input character c, the parser takes the annotated derivative $ri\backslash_b c$ and simplifies it, replacing ri with $simp(ri\backslash_b c)$. After the last character, the parser checks whether the final ri is nullable. If it is, the parser extracts the bits with $mkEpsBC$ and decodes them against the original r:

$$\begin{aligned} ri_0 &= internalize(r), \\ ri_i &= simp(ri_{i-1}\backslash_b x_i) \quad (1 \le i \le n), \\ result &= \begin{cases} decode_r(mkEpsBC(ri_n)) & n(ri_n) \\ \text{no match} & \text{otherwise.} \end{cases} \end{aligned} \tag{5.12}$$

The loop holds only the current ri. Unlike the standard derivative parser, which keeps a growing vector of residuals for the backward reconstruction, the bit-coded parser needs no history. The bits record everything needed to decode, so the forward pass can discard each intermediate expression as soon as the next one is computed.

In Rust. Figure 5.12 implements (5.12).

```
pub fn parse_bitcoded_loop(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut ri = internalize(r);
    for l in input.chars() { ri = simp(deriv_bc(ri, l)); }
    if !nullable_bc(&ri) { return None; }
    Some(decode(r, &mk_eps_bc(&ri)))
}
```

Figure 5.12: `parse_bitcoded_loop`: the bit-coded parser implementation.

The loop matches the three cases of (5.12) one for one. `internalize` initialises `ri`, the `for` loop applies `simp(deriv_bc(ri, l))` to each character and the nullability check gates the final `decode(mk_eps_bc(ri))`.

5.2.8 Worked Example: Applying the Bitcoded Parser

Consider the same input $\mathtt{ab}$ and regular expression $r_0 = (a + ab)(b + \varepsilon)$ as in Section 5.1.4.

Internalize. Before consuming input, *internalize* tags each alternative with [0] for the left branch and [1] for the right branch:

$$\mathit{internalize}(r_0) = ri_0 = [\,]@\Big([\,]@([0]@a \oplus [1]@(a\,b)) \cdot [\,]@([0]@b \oplus [1]@\varepsilon)\Big). \tag{5.13}$$

Bit-Threaded Derivatives. The two input symbols are consumed by successive bit-threaded derivatives with *simp* applied after each step. The first derivative uses B_{seq}. After consuming $\mathtt{a}$, the first factor becomes nullable, so the second derivative uses $\mathsf{B}^n_{\text{seq}}$:

$$\begin{aligned}
ri_0 &\xrightarrow[\mathsf{B}_{\text{seq}}]{a} ri_1 = [\,]@\Big([\,]@([0]@\varepsilon \oplus [1]@b) \cdot [\,]@([0]@b \oplus [1]@\varepsilon)\Big) \\
ri_1 &\xrightarrow[\mathsf{B}^n_{\text{seq}}]{b} ri_2' = [\,]@\Big(\underbrace{[\,]@(\emptyset \oplus [1]@\varepsilon) \cdot [\,]@([0]@b \oplus [1]@\varepsilon)}_{X} \oplus \underbrace{[0]@\Big([0]@\varepsilon \oplus \emptyset\Big)}_{Y} \Big) \\
&\xrightarrow{\;\mathit{simp}\;} ri_2 = [\,]@\Big([1]@([0]@b \oplus [1]@\varepsilon) \oplus [0,0]@\varepsilon\Big).
\end{aligned} \tag{5.14}$$

Each disjunct of ri_2' has a dead branch. In X, the first factor's derivative by $\mathtt{b}$ is $\emptyset \oplus [1]@\varepsilon$: the [0] side cannot consume $\mathtt{b}$, the [1] side owes one. In Y, the second factor's derivative by $\mathtt{b}$ is $[0]@\varepsilon \oplus \emptyset$: the [0] side matches, the [1] side cannot. The outer [0] on Y is the bit that $\mathsf{B}^n_{\text{seq}}$ fused on from the first factor's empty match.

simp drops the dead branches. From X, the survivor is $[1]@\varepsilon$, and since ε contributes nothing to the concatenation, the [1] is carried onto the second factor, giving $[1]@([0]@b \oplus [1]@\varepsilon)$. From Y, the survivor is $[0]@\varepsilon$; the outer [0] is merged with the inner [0] and the resulting $[0,0]$ is carried onto the ε, giving $[0,0]@\varepsilon$. The two results are the disjuncts of ri_2, so both bits are recorded there and the tree can later be decoded from the bits alone.

Extracting the bits. Both disjuncts of ri_2 are nullable, so EB^L_+ selects the left one. Inside, b is not nullable while ε is, so EB^R_+ contributes its [1] tag:

$$\mathit{mkEpsBC}(ri_2) \xrightarrow{\mathsf{EB}^L_+} \mathit{mkEpsBC}\Big([1]@([0]@b \oplus [1]@\varepsilon)\Big) \xrightarrow{\mathsf{EB}^R_+} [1,1]. \tag{5.15}$$

Unlike *mkEps*, which constructs a partial parse tree, *mkEpsBC* extracts only the committed choices as a bit-string.

Decoding the bits. The decoder interprets $[1,1]$ against the original r_0: the first bit selects the right branch of $a + ab$ (producing ab), and the second selects the right branch of $b + \varepsilon$:

$$\mathit{decode}_{r_0}([1,1]) = \Big(\mathit{Right}(a,b), \mathit{Right}(\,)\Big) = v_0. \tag{5.16}$$

The decoder consumes the bits left to right, one per alternation it encounters, in the same order that *internalize* introduced them. Because *mkEpsBC* preserved that order when it fused and prepended bits, the first bit always belongs to the outermost alternation, and so on down.

Flattening gives $\mathit{flatten}(v_0) = a \cdot b \cdot \varepsilon = \mathtt{ab}$, confirming the parse. The bit-coded construction produces the same tree as the direct construction, but stores only ri_2, a single expression bounded by the pattern instead of one residual per character.

6 Partial-Derivative-Based Parser

Chapter 5 presented two derivative-based methods for constructing POSIX parse trees. The standard parser stores the derivation sequence and reconstructs the tree in a backward pass using *mkEps* and the injections. The bit-coded parser records the choices as bits during the forward pass and decodes them against the original expression at the end.

This chapter presents the partial-derivative alternative of Sulzmann and Lu [Sulzmann and Lu, 2012]. Instead of one residual per step, the partial derivative produces a set of states, each pairing a residual expression with the reconstruction information for one path through the original expression. The parser maintains this set as an NFA state set, and at the end walks it in generation order, taking the first accepting state. Because partial derivatives break apart the structure of the original expression, this walk yields the leftmost alternative at every choice rather than the longest one, and therefore a non-POSIX parse tree.

As in Chapter 5, two variants are presented. The standard parser attaches an injection closure to each state. The bit-coded parser replaces these closures with bit-sequences. Both compute the same parse tree.

6.1 The Standard Partial-Derivative Parser

Consider again $r_0 = (a + ab)(b + \varepsilon)$ with input $w = \mathtt{ab}$. Where *deriv* reduces r_0 to a single residual at each step, Antimirov's partial derivative *pderiv* reduces it to a *set* of states, one per way the next character can be consumed. Each state pairs a residual expression with an injection that rebuilds a parse tree for r_0 from a parse tree for that residual. The injection is attached to its state as soon as the state is created, rather than recovered afterwards from a stored sequence the way *inj* was for the standard derivative parser in Section 5.1.

Deriving r_0 by $\mathtt{a}$ produces two states, one for each way $\mathtt{a}$ can be consumed:

$$\begin{aligned} \mathit{pderiv}(r_0, \mathtt{a}) = \big\{ &(\ \varepsilon \cdot (b + \varepsilon), \ \lambda(_, v_2).\ (\mathit{Left}\, a, \ v_2) \ ), \\ &(\ (\varepsilon \cdot b)\,(b + \varepsilon), \ \lambda((v_1, v_2), v_2').\ (\mathit{Right}(a, v_2), \ v_2') \ ) \big\}. \end{aligned} \tag{6.1}$$

A state is a pair: a residual expression describing what is left to match, and an injection rebuilding a parse tree of the original expression from a parse tree of the residual. The two states above correspond to the two branches of the alternation $a + ab$ in the first factor. The second factor $b + \varepsilon$ is untouched in both and appears as the trailing component of each residual.

Each residual has the shape $r' \cdot (b + \varepsilon)$: the derivative of the first factor, followed by the untouched second factor, never simplified. The leading ε in each r' records that the consumed character matched a leading literal and left nothing behind in that position; it is not removed, because the injection below pattern-matches on exactly this pair shape.

The first state's residual $\varepsilon \cdot (b + \varepsilon)$ is a sequence, so its tree is a pair (v_1, v_2): v_1 is the tree of the leading ε, which is necessarily *Empty*, and v_2 is the tree of the untouched second factor $b + \varepsilon$. The closure returns $(\mathit{Left}(a), v_2)$. It discards v_1 because the tree of ε carries no information, and replaces the first position with $\mathit{Left}(a)$ to record that the left branch of the alternation $a + ab$ was taken. The second factor's tree v_2 is passed through unchanged.

The second state's residual $(\varepsilon \cdot b)\,(b + \varepsilon)$ is a sequence whose first component is itself a sequence, so its tree is a pair $((v_1, v_2), v_2')$: the inner pair (v_1, v_2) is the tree of $\varepsilon \cdot b$, with v_1 the tree of the leading ε and v_2 the tree of the trailing b; the outer v_2' is the tree of the untouched second factor. The closure returns $(\mathit{Right}(a, v_2), v_2')$. It rebuilds the consumed a (which the leading ε records) and the still-owed b (which v_2 records) as the pair (a, v_2), places that pair inside a right injection to record that the right branch ab of the alternation was taken, and passes the second factor's tree v_2' through unchanged. The binder v_1 is bound but not used in the body, for the same reason as the first state's `_`: the tree of ε carries no information.

Transitioning both states on the second character `b` gives a new state set:

$$\begin{aligned} \mathit{pderiv}(r_0, \mathtt{a}, \mathtt{b}) = \big\{&(\varepsilon,\ \lambda_.\ (\mathit{Left}\, a,\ \mathit{Left}\, b)\ ),\\ &(\varepsilon \cdot (b + \varepsilon),\ \lambda(_, v_2).\ (\mathit{Right}(a, b),\ v_2)\ )\big\}. \end{aligned} \tag{6.2}$$

The first state now has residual ε, meaning every input character has been consumed, and is accepting; the second has residual $\varepsilon \cdot (b + \varepsilon)$ – the second factor is untouched by `b`, since it was the first factor's leftover b that matched – which is also accepting. Selection walks the state set in generation order and returns the first accepting state. Applying its injection to $\mathit{mkEps}_{\varepsilon} = ()$ yields $(\mathit{Left}\, a, \mathit{Left}\, b)$, the leftmost alternative at every choice (Figure 6.1).

Two properties of the state set determine which tree is selected. First, the order of states in the set encodes the preference. When a state produces several successors, they are appended in the order the rules generate them, with the left alternative first, so the leftmost path appears earliest. Second, when two states share a residual, *nub* keeps the first and discards the later one with its injection, so the earlier path wins ties. Both mechanisms favour the leftmost alternative at every choice over the longest one, which is what distinguishes the tree this parser returns from the POSIX tree of Chapter 5.

6.1.1 Partial Derivatives with Injections

A partial derivative of an expression r with respect to a character x is not a single expression but a set of pairs. Each pair consists of a residual expression r' whose language is one component of the left quotient, together with an injection closure ι that maps a parse tree for r' back to a parse tree for r. Stripped of the injection, the residual side of this definition is exactly *pd*, the Antimirov partial derivative already used by the matcher of Section 3.3 [Antimirov, 1996], so *pderiv* reuses its rule names $\mathsf{P}_{\emptyset}, \mathsf{P}_{\varepsilon}, \mathsf{P}_{\mathrm{lit}}, \mathsf{P}_{+}, \mathsf{P}_{\mathrm{seq}}, \mathsf{P}^{n}_{\mathrm{seq}}, \mathsf{P}_{*}$

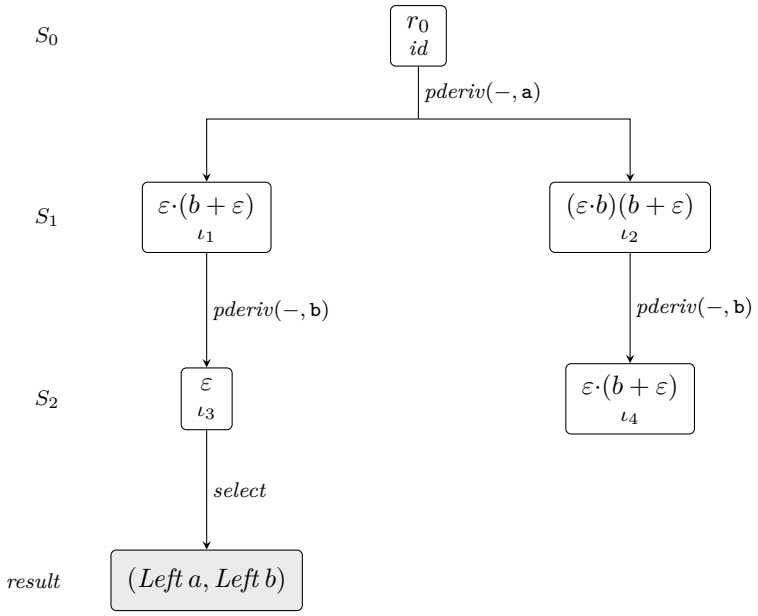


Figure 6.1: The standard partial-derivative parser on $r_0 = (a + ab)(b + \varepsilon)$ with input `ab`. Each box is a state (r, ι): a residual and an injection, both shown without any simplification. $pderiv(-, x)$ advances every state by one character; *select* takes the first accepting state and applies its injection to *mkEps* of its residual.

from Equation 3.14, each now additionally threading an injection alongside its residual. The idea of pairing a residual with a reconstruction function is, in turn, in the style of Sulzmann and Lu's pattern partial derivative [Sulzmann and Lu, 2012, Fig. 11], which pairs a residual *pattern* with a function building a variable-binding environment; here the same idea builds a *ParseTree* instead.

The rules that do not depend on whether their operand is nullable are:

$$
\begin{aligned}
pderiv(\emptyset, x) &= \{\} && (\mathsf{P}_\emptyset)\\
pderiv(\varepsilon, x) &= \{\} && (\mathsf{P}_\varepsilon)\\
pderiv(y, x) &= \begin{cases} \{(\varepsilon,\ \lambda_.\, y)\} & x = y \\ \{\} & x \neq y \end{cases} && (\mathsf{P}_{\text{lit}})\\
pderiv(r + s, x) &= \{\, (r',\ \lambda v.\, Left\,(\iota\, v)) \mid (r', \iota) \in pderiv(r, x) \,\}\\
&\quad \cup\ \{\, (s',\ \lambda v.\, Right\,(\iota\, v)) \mid (s', \iota) \in pderiv(s, x) \,\} && (\mathsf{P}_+)\\
pderiv(r^*, x) &= \{\, (r'\, r^*,\ \lambda(v, Star(vs)).\, Star([\iota\, v] +\!\!+ vs)) \mid (r', \iota) \in pderiv(r, x) \,\} && (\mathsf{P}_*)
\end{aligned}
\tag{6.3}
$$

exactly as in *pd*, with the recursive call's injection wrapped by *Left*, *Right*, or prepended to the iteration list. No rule is needed for $\emptyset$ or ε beyond their empty results, since neither can consume a character.

Concatenation alone must branch on $n(r)$, so it is shown separately:

$$
pderiv(r \cdot s, x) = \begin{cases}
\{\, (r'\, s,\ \lambda(v_1, v_2).\, (\iota\, v_1,\ v_2)) \mid (r', \iota) \in pderiv(r, x) \,\} & n(r) \quad (\mathsf{P}^n_{\text{seq}})\\
\cup\ \{\, (s',\ \lambda v.\, (mkEps_r,\ \iota\, v)) \mid (s', \iota) \in pderiv(s, x) \,\} & \\
\{\, (r'\, s,\ \lambda(v_1, v_2).\, (\iota\, v_1,\ v_2)) \mid (r', \iota) \in pderiv(r, x) \,\} & \neg\, n(r) \quad (\mathsf{P}_{\text{seq}})
\end{cases}
\tag{6.4}
$$

After expansion, each rule's result is deduplicated by residual, keeping the first occurrence; this is *nub*, applied wherever a transition can produce two states with the same residual, namely P_+, $\mathsf{P}^n_{\text{seq}}$, and P_*. P_{seq} has only one source of states and needs no deduplication of its own, since its single recursive call is already duplicate-free. This is the step that breaks apart the original expression: where the derivative of $r + s$ produces a single alternation, the partial derivative produces separate states for each branch, and the outer alternation disappears from the residual.

Rule (P_{seq}) – the only one present regardless of nullability – passes the outer pair through unchanged, with the recursive injection applied to the first component only. When r is nullable, rule ($\mathsf{P}^n_{\text{seq}}$) additionally unions in the states that skip r entirely, using $mkEps_r$ to synthesise its empty match for the first component. Rule (P_+) wraps the recursive injection in a *Left* or *Right* node, reinserting the alternation choice. Rule (P_*) reconstructs the current iteration by applying the recursive injection and prepending it to the remaining iterations.

Unlike *pd*, which applies the smart constructor *smartC* in P_{seq} and P_* to avoid building $\varepsilon \cdot s$ (Equation 3.15), *pderiv* never simplifies: $r' \cdot s$ above is built with the plain pairing constructor, even when $r' = \varepsilon$. A simplified residual would collapse exactly the pair-shaped node that the injection above pattern-matches on, so *pderiv* trades the smaller residuals *pd* produces for residuals whose shape the injection can always follow. This is the same tradeoff the standard derivative parser makes by never simplifying (Section 5.1), for the same reason: *inj* there pattern-matches on the derivative's structure just as ι does here.

In Rust. The recursive definition in Equations 6.3 and 6.4 is implemented directly in Rust, with two departures from a literal reading of the equations, both for performance rather than semantics.

First, `pderiv_tree` itself builds only *one* transition's closure, type `InjStep`, a reference-counted closure exactly as before. The accumulated injection a state carries across many characters – what the equations call ι once composition has happened several times – is a separate type, `Inj`, described in Section 6.1.2: a persistent list of `InjStep`s rather than a nested closure, so that applying or dropping a long-lived `Inj` does not recurse one native stack frame per character.

Second, the `Seq` and `Star` arms use `smart_seq` – the Rust counterpart of *smartC* used by `pderiv` in Figure 3.7 – wherever the equations build $\varepsilon \cdot s$, except in `Seq`'s nullable branch, which still pairs its continue-strand with the plain `Regex::seq` constructor. Collapsing $\varepsilon \cdot s$ to s changes the shape the next transition's closure must match against, so each arm that uses `smart_seq` branches on whether the residual collapsed (`r1_ == Regex::Eps`) and builds the closure for that shape specifically, rather than assuming a `Pair` is always present. Without this, two strands that independently derive to the same residual but reached it through different wrapper shapes would not dedup against each other, and the frontier can grow unboundedly on an ambiguous pattern even though the number of distinct residuals stays bounded.

```
pub(crate) type InjStep = Rc<dyn Fn(ParseTree) -> ParseTree>;

pub(crate) fn pderiv_tree(r: &Regex, x: char) -> Vec<(Regex, InjStep)> {
    match r {
        Regex::Eps | Regex::Phi => vec![],

        Regex::Lit(c) => {
            if *c == x {
                vec![(Regex::Eps,
                      Rc::new(move |_| ParseTree::Char(x))) as InjStep]
            } else { vec![] }
        }

        Regex::Alt(r1, r2) => {
            let mut out: Vec<(Regex, InjStep)> = pderiv_tree(r1, x)
                .into_iter()
                .map(|(r1_, inj1)| {
                    let f: InjStep = Rc::new(move |v| {
                        ParseTree::Left(Box::new(inj1(v)))
                    });
                    (r1_, f)
                })
                .collect();
            out.extend(pderiv_tree(r2, x).into_iter().map(|(r2_, inj2)| {
                let f: InjStep = Rc::new(move |v| {
                    ParseTree::Right(Box::new(inj2(v)))
                });
                (r2_, f)
            }));
            nub_tree(out)
        }

        Regex::Seq(r1, r2) => {
            if nullable(r1) {
                // Continue-strand: not smart-constructed, matching
                // the reference literally -- the skip-strand below
                // needs r1's own closure untouched to inject into.
                let mut out: Vec<(Regex, InjStep)> = pderiv_tree(r1, x)
                    .into_iter()
                    .map(|(r1_, inj1)| {
                        let r2c = (**r2).clone();
                        let f: InjStep = Rc::new(move |v| match v {
                            ParseTree::Pair(v1, v2) =>
                                ParseTree::Pair(Box::new(inj1(*v1)), v2),
                            _ => panic!("pderiv_tree/Seq: want Pair"),
                        });
                        (Regex::seq(r1_, r2c), f)
                    })
                    .collect();

                let v1_eps = mk_eps(r1);
                out.extend(pderiv_tree(r2, x).into_iter().map(|(r2_, inj2)| {
                    let v1c = v1_eps.clone();
                    let f: InjStep = Rc::new(move |v2| {
                        ParseTree::Pair(Box::new(v1c.clone()),
                                        Box::new(inj2(v2)))
                    });
```

The `Phi`, `Eps`, and `Lit` arms implement $\mathsf{P}_\emptyset$, P_ε, and P_{lit} directly, matching the same arms of `pderiv` in Figure 3.7, except that each returns a closure alongside its residual instead of nothing. The `Alt` arm implements P_+ and is where the partial derivative visibly breaks apart the expression: each branch produces its own states, tagged by *Left* or *Right* in the injection, then deduplicated by `nub_tree`.

The `Seq` arm's `if nullable(r1)` matches the case split between $\mathsf{P}^n_{\text{seq}}$ and P_{seq}. The nullable branch's continue-strand is built exactly as the equations describe, with the plain pairing constructor, since the skip-strand alongside it needs `r1`'s own closure to still expect a `Pair`; the nullable branch additionally computes `r1`'s empty tree once with `mk_eps` and captures a clone of it in every skip-strand closure, then deduplicates the combined list. The non-nullable branch, by contrast, does use `smart_seq`, with the closure split on whether the residual collapsed to `Eps`, as explained above.

The `Star` arm implements P_*, with the closure appending the reconstructed iteration to the star's list (left-to-right order, reversed once at the end by `select` rather than paying `insert(0, ..)`'s cost on every iteration), again split on whether `smart_seq` collapsed the residual.

6.1.2 State Sets and Selection

The parser maintains a state set of (residual, injection) configurations. For each input character, every configuration in the state set is transitioned with `pderiv_tree`, and the resulting configurations are collected into a new state set, replacing the old one – unlike the standard derivative parser, nothing from earlier state sets is retained once the next one has been computed. A configuration's parent injection is composed with the transition injection, so that at the end the state set's injections map residuals all the way back to the original expression.

Conceptually the composition is direct: when a parent configuration (r, inj_p) transitions to (r', inj_s), the new configuration's injection is $\lambda v.\, inj_p(inj_s(v))$. Composing this way as nested closures, however, makes both calling the result and dropping it recurse one native stack frame per character – fine at tens of thousands of characters, a real stack-overflow risk beyond that. The `Inj` type instead stores a configuration's accumulated steps as a persistent cons-list and applies them in an explicit loop:

```
enum InjNode { Identity, Step(InjStep, Rc<InjNode>) }
pub(crate) struct Inj(Rc<InjNode>);

impl Inj {
    pub(crate) fn then(&self, step: InjStep) -> Self {
        Inj(Rc::new(InjNode::Step(step, self.0.clone())))
    }
    pub(crate) fn apply(&self, v: ParseTree) -> ParseTree {
        let mut v = v;
        let mut cur = &self.0;
        loop {
            match &**cur {
                InjNode::Identity => return v,
                InjNode::Step(step, rest) => { v = step(v); cur = rest; }
```

```
            }
        }
    }
}
```

`then` adds one more character's step in O(1), just wrapping the existing chain in a new node, the same way `Bits::prepend` (Section 5.2.2) wraps an existing bit chain; `apply` walks the whole chain in one iterative pass, applying each step in the order it was added, with no recursive call regardless of how long the chain is. `Inj` also has its own iterative `Drop`, for the same reason `Bits`'s does: the derived recursive drop glue would walk the chain one stack frame per character too.

```
pub(super) fn step_frontier(
    frontier: Vec<(Regex, Inj)>, x: char
) -> Vec<(Regex, Inj)> {
    let mut out = Vec::new();
    for (r, inj_parent) in frontier {
        for (r_next, inj_step) in pderiv_tree(&r, x) {
            out.push((r_next, inj_parent.then(inj_step)));
        }
    }
    nub_tree(out)
}
```

Figure 6.3: `step_frontier`: one transition of the NFA state set on a single character. The function name retains the older "frontier" terminology; it computes the successor state set $\delta(S, x)$. The final `nub_tree` deduplicates *across* parent configurations, not just within one parent's own expansion (which `pderiv_tree` already does on its own output) – without it, two strands that independently derive to the same residual both survive, and the state set can double every character on an ambiguous pattern even though the number of distinct residuals stays bounded by the pattern's own size.

After the input is exhausted, selection walks the state set in priority order and returns the tree for the first accepting configuration:

```
pub(super) fn select(frontier: &[(Regex, Inj)]) -> Option<ParseTree> {
    for (r, inj) in frontier {
        if nullable(r) {
            return Some(reverse_stars(inj.apply(mk_eps(r))));
        }
    }
    None
}
```

Figure 6.4: `select`: return the tree for the first accepting configuration in the state set. `reverse_stars` undoes, in one O(total output size) pass, the left-to-right-via-push construction order `pderiv_tree`'s `Star` arm uses (Figure 6.2).

Because the state set lists configurations in the order the rules generated them, and because `nub_tree` keeps the first of each duplicate residual, selection favours the configuration that committed earliest to the leftmost alternative. This is the greedy leftmost

policy. Unlike the standard derivative parser, there is no separate backward pass here: by the time `select` runs, every configuration's injection is already the full composition back to the original expression, so selection only has to apply it once.

6.1.3 Parser

The parser has a single pass: every input character advances the whole state set at once via *step_frontier*, and the final state set is handed directly to *select*:

$$S_0 = \{(r, id)\}, \qquad S_i = step_frontier(S_{i-1}, x_i) \;\; (1 \le i \le n), \qquad result = select(S_n). \tag{6.5}$$

Unlike the standard derivative parser's two-pass driver (Equation 5.4), no backward pass is needed here: each configuration in S_i already carries its injection composed all the way back to r_0 (Section 6.1.2), so *select* only ever applies one injection, to the single accepting configuration it finds in S_n.

In Rust. Figure 6.5 implements (6.5).

```
pub fn parse_pderiv_std(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier: Vec<(Regex, Inj)> =
        vec![(r.clone(), Inj::identity())];
    for c in input.chars() {
        frontier = step_frontier(frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    select(&frontier)
}
```

Figure 6.5: The standard partial-derivative parser.

The loop computes $S_0, \ldots, S_n$ in place, overwriting `frontier` on every iteration rather than appending to a growing `Vec` the way `parse_deriv_std_loop` (Figure 5.4) does – nothing from an earlier state set is retained once the next one exists. The early return on an empty frontier corresponds to $S_i = \{\}$: no configuration can consume the current character, so the input cannot be matched. The final `select` returns `None` if no configuration in S_n is accepting, and otherwise the tree for the first one.

6.1.4 Worked Example: Standard Partial-Derivative Parser

Recall the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ on input `ab`.

Forward pass. The initial state set holds a single configuration:

$$S_0 = \big[(r_0,\ id)\big].$$

Transitioning with respect to `a`:

$$pderiv(r_0, \mathtt{a}) = pderiv\Big(Alt(a, ab), \mathtt{a}\Big) \cdot Alt(b, \varepsilon).$$

Inside the first factor:

$$\begin{aligned} \mathit{pderiv}(a, \mathtt{a}) &= \Big[(\varepsilon,\ \lambda_.\, a)\Big] \\ \mathit{pderiv}(ab, \mathtt{a}) &= \Big[(\mathit{Seq}(\varepsilon, b),\ \lambda(v_1, v_2).\, (a, v_2))\Big]. \end{aligned}$$

After the alternation rule wraps each state with *Left* or *Right*:

$$\begin{aligned} \mathit{pderiv}(a + ab, \mathtt{a}) = \Big[&(\varepsilon,\ \lambda v.\, \mathit{Left}\,(\lambda_.\, a)(v)), \\ &(\mathit{Seq}(\varepsilon, b),\ \lambda(v_1, v_2).\, \mathit{Right}\,(a, v_2))\Big]. \end{aligned}$$

The outer sequence rule combines this with the untouched second factor:

$$\begin{aligned} S_1 = \Big[&(\mathit{Seq}(\varepsilon, b + \varepsilon),\ \lambda(v_1, v_2).\, (\mathit{Left}\, a,\ v_2)), \\ &(\mathit{Seq}(\mathit{Seq}(\varepsilon, b), b + \varepsilon),\ \lambda((v_1, v_2), v_2').\, (\mathit{Right}(a, v_2),\ v_2'))\Big]. \end{aligned}$$

Transitioning with respect to `b` steps both configurations. For the first, the left factor ε is nullable, so the sequence rule produces the state that jumps directly into the second factor:

$$\mathit{pderiv}(b + \varepsilon, \mathtt{b}) = \Big[(\varepsilon,\ \lambda v.\, \mathit{Left}\, b)\Big].$$

The second factor's injection inserts the character at the leftmost position of the pair:

$$\mathit{inj}_1(\mathit{Left}\, b) = (\mathit{Left}\, a,\ \mathit{Left}\, b).$$

For the second configuration, the leading $\mathit{Seq}(\varepsilon, b)$ is not nullable (ε is, but b is not), so rule ($\mathsf{P}_{\mathrm{seq}}$) fires and the derivative continues inside b:

$$\mathit{pderiv}(\mathit{Seq}(\varepsilon, b), \mathtt{b}) = \Big[(\varepsilon,\ \lambda_.\, ((), b))\Big],$$

and the outer sequence rule rebuilds the pair *without* simplifying the leading ε away, giving the second configuration's residual $\varepsilon \cdot (b + \varepsilon)$, not $b + \varepsilon$. The two resulting state-set entries are:

$$\begin{aligned} S_2 = \Big[&(\varepsilon,\ \lambda_.\, (\mathit{Left}\, a,\ \mathit{Left}\, b)), \\ &(\varepsilon \cdot (b + \varepsilon),\ \lambda(_, v_2).\, (\mathit{Right}(a, b),\ v_2))\Big]. \end{aligned}$$

Selection. The first configuration's residual is ε, which is nullable, and selection returns immediately:

$$\mathit{select}(S_2) = (\lambda_.\, (\mathit{Left}\, a, \mathit{Left}\, b))(\mathit{mkEps}_{\varepsilon}) = (\mathit{Left}\, a, \mathit{Left}\, b).$$

Flattening confirms the result:

$$\mathit{flatten}\Big((\mathit{Left}\, a, \mathit{Left}\, b)\Big) = \mathtt{ab}.$$

The second configuration is never inspected: its residual $\varepsilon \cdot (b + \varepsilon)$ is also nullable, and would likewise decode to a valid tree, but generation order already placed the first configuration ahead of it, and *select* stops at the first match.

This is the greedy parse tree. The derivative-based parser from Chapter 5 returns $(\mathit{Right}(a, b), \mathit{Right}())$ for the same input, the POSIX tree. The two trees agree on which word they spell out, but differ on how the input is decomposed: the greedy tree commits to the shorter first alternative a in the first factor, while the POSIX tree commits to the longer alternative ab.

6.2 The Bit-Coded Partial-Derivative Parser

The standard partial-derivative parser carries a closure per configuration, which accumulates the reconstruction path during the forward pass. The bit-coded variant replaces closures with bit-sequences: each configuration's annotation records the choices made so far, and decoding the final bit-sequence against the original expression yields the parse tree.

Transitioning $w = \texttt{ab}$ from $r_0 = (a + ab)(b + \varepsilon)$ gives:

$$\begin{aligned} S_0 &= \Big[(r_0,\ [\,])\Big] \\ &\xrightarrow{\texttt{a}} S_1 = \Big[(b + \varepsilon,\ [\mathit{false}]),\ (b\,(b + \varepsilon),\ [\mathit{true}])\Big] \\ &\xrightarrow{\texttt{b}} S_2 = \Big[(\varepsilon,\ [\mathit{false}, \mathit{false}]),\ (b + \varepsilon,\ [\mathit{true}])\Big]. \end{aligned} \tag{6.6}$$

The first configuration of S_1 records that the character `a` was matched by the left branch of the first factor's alternation. The second configuration records that it was matched by the right branch, leaving the residual b to be consumed later. After the second character, only the first configuration is accepting, and its bits $[\mathit{false}, \mathit{false}]$ decode to $(\mathit{Left}\, a, \mathit{Left}\, b)$.

Figure 6.6 shows this transition together with the selection and decode steps.

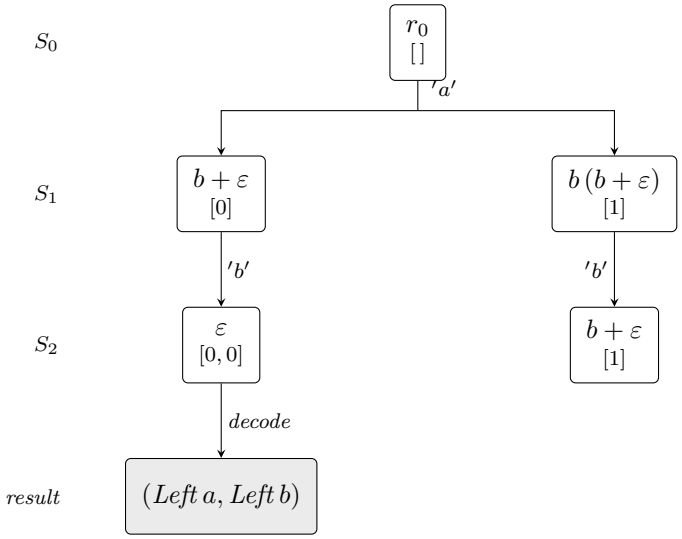


Figure 6.6: Overview of the bit-coded partial-derivative parser on the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ with input `ab`. Each box is a configuration: a residual expression and a bit-sequence, written 0 for *false* and 1 for *true*. Selection takes the first accepting configuration in S_2 and appends *mkEpsBC* of its residual, yielding the bit-string $[0, 0]$, which decode_{r_0} reads against the original expression.

The bit-coded parser keeps the same state-set structure as the standard one. Only the configuration's annotation changes, from a closure to a sequence of bits, and the final step changes from applying an injection to calling *decode*.

6.2.1 Bit-Coded Partial Derivatives

The bit-coded partial derivative $\mathit{pderivBC}(r, x)$ returns a list of (residual, bit-sequence) pairs. It follows the same case structure as (**??**), with one difference: where the standard

version builds an injection closure, the bit-coded version prepends a fixed bit-sequence to the configuration's existing bits.

The bit conventions are inherited from Chapter 5.2. Each alternation contributes a leading *false* for the left branch or a leading *true* for the right branch. Each star iteration contributes a leading *false*, and each star termination contributes a *true*. Concatenation contributes no bits of its own.

$$
\begin{aligned}
\mathit{pderivBC}(\phi, x) &= \emptyset \\
\mathit{pderivBC}(\varepsilon, x) &= \emptyset \\
\mathit{pderivBC}(c, x) &= \begin{cases} \{(\varepsilon,\, [\,])\} & c = x \\ \emptyset & \text{otherwise} \end{cases} \\
\mathit{pderivBC}(r_1 + r_2, x) &= \{\, (r_1',\, [\mathit{false}] +\!\!+\, bs) \mid (r_1', bs) \in \mathit{pderivBC}(r_1, x) \,\} \\
&\quad \cup \{\, (r_2',\, [\mathit{true}] +\!\!+\, bs) \mid (r_2', bs) \in \mathit{pderivBC}(r_2, x) \,\} \\
\mathit{pderivBC}(r_1\, r_2, x) &= \{\, (\mathit{seq}(r_1', r_2),\, bs) \mid (r_1', bs) \in \mathit{pderivBC}(r_1, x) \,\} \\
&\quad \cup \begin{cases} \{\, (r_2',\, \mathit{mkEpsBC}_{r_1} +\!\!+\, bs) \mid (r_2', bs) \in \mathit{pderivBC}(r_2, x) \,\} & n(r_1) \\ \emptyset & \text{otherwise} \end{cases} \\
\mathit{pderivBC}(r^*, x) &= \{\, (\mathit{seq}(r', r^*),\, [\mathit{false}] +\!\!+\, bs) \mid (r', bs) \in \mathit{pderivBC}(r, x) \,\}.
\end{aligned}
\tag{6.7}
$$

As in the standard version, the result is deduplicated by residual, keeping the first occurrence.

The bit-coded sequence rule is the counterpart of the injection rule in the standard version. When the first factor is nullable, the bits recorded inside its empty match are extracted with $\mathit{mkEpsBC}_{r_1}$ and prepended to the second factor's bits, so the choice made inside the first factor survives in the surviving configuration.

In Rust. Figure 6.7 implements (6.7) directly. The base cases and the alternation case mirror the standard version with bit-lists in place of closures. The sequence case inlines both the non-nullable and nullable paths. The star case prepends the leading *false* that marks one more iteration.

```
pub fn pderiv_bc(r: &Regex, x: char) -> Vec<(Regex, Vec<bool>)> {
    match r {
        Regex::Phi | Regex::Eps => vec![],

        Regex::Lit(y) => {
            if *y == x { vec![(Regex::Eps, vec![])] } else { vec![] }
        }

        Regex::Alt(r1, r2) => {
            let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
                .into_iter()
                .map(|(r1_, mut bs)| {
                    let mut b = vec![false]; b.append(&mut bs);
                    (r1_, b)
                })
                .collect();
            out.extend(pderiv_bc(r2, x).into_iter().map(|(r2_, mut bs)| {
                let mut b = vec![true]; b.append(&mut bs);
                (r2_, b)
            }));
            nub2(out)
        }

        Regex::Seq(r1, r2) => {
            if nullable(r1) {
                let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
                    .into_iter()
                    .map(|(r1_, bs)| (Regex::seq(r1_, *r2.clone()), bs))
                    .collect();

                let eps_bits = mk_eps_bits(r1);
                out.extend(pderiv_bc(r2, x).into_iter().map(|(r2_, bs)| {
                    let mut b = eps_bits.clone(); b.extend(bs);
                    (r2_, b)
                }));
                nub2(out)
            } else {
                pderiv_bc(r1, x)
                    .into_iter()
                    .map(|(r1_, bs)| (smart_seq(r1_, r2), bs))
                    .collect()
            }
        }

        Regex::Star(r1) => {
            let out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
                .into_iter()
                .map(|(r1_, mut bs)| {
                    let mut b = vec![false]; b.append(&mut bs);
                    (smart_seq(r1_, &Regex::star(*r1.clone())), b)
                })
                .collect();
            nub2(out)
        }
    }
}
```


Figure 6.7: `pderiv_bc`: one bit-coded partial-derivative transition

The `Seq` arm keeps the outer sequence constructor in the nullable case, matching the standard version's behaviour. It uses `smart_seq` only in the non-nullable case, where the derivative of the first factor is guaranteed to be a proper expression and no outer pair needs to be preserved. This asymmetry mirrors the paper's reference implementation and is needed so that the residual's shape stays compatible with the decoder.

6.2.2 Extracting Bits

When a configuration's residual is nullable, its accumulated bits must be extended with the bits of the residual's empty match. This is the role of $mkEpsBC$, which mirrors $mkEps$ from Chapter 5 but produces a bit-sequence instead of a parse tree.

$$\begin{aligned} mkEpsBC_{\varepsilon} &= [\,] \\ mkEpsBC_{r^*} &= [true] \\ mkEpsBC_{r_1 r_2} &= mkEpsBC_{r_1} +\!\!+ mkEpsBC_{r_2} \\ mkEpsBC_{r_1 + r_2} &= \begin{cases} [false] +\!\!+ mkEpsBC_{r_1} & n(r_1) \\ [true] +\!\!+ mkEpsBC_{r_2} & \text{otherwise.} \end{cases} \end{aligned} \tag{6.8}$$

The alternation rule reproduces the left preference of $mkEps$, emitting *false* when the left branch is nullable and *true* otherwise. The star rule emits a single terminating *true*, since a star matching the empty word uses zero iterations.

```
pub fn mk_eps_bits(r: &Regex) -> Vec<bool> {
    match r {
        Regex::Phi => panic!("mk_eps_bits called on Phi"),
        Regex::Lit(c) => panic!("mk_eps_bits called on Lit('{}')", c),
        Regex::Eps => Vec::new(),
        Regex::Alt(r1, r2) => {
            if nullable(r1) {
                let mut bits = vec![false];
                bits.extend(mk_eps_bits(r1));
                bits
            } else {
                let mut bits = vec![true];
                bits.extend(mk_eps_bits(r2));
                bits
            }
        }
        Regex::Seq(r1, r2) => {
            let mut bits = mk_eps_bits(r1);
            bits.extend(mk_eps_bits(r2));
            bits
        }
        Regex::Star(_) => vec![true],
    }
}
```

Figure 6.8: `mk_eps_bits`: the bit-string analogue of `mk_eps` for plain `Regex`.

The `Phi` and `Lit` arms panic, matching the precondition that *mkEpsBC* is applied only to nullable expressions. Neither empty language nor a literal can match the empty word.

6.2.3 State Sets and Selection

State-set transition for the bit-coded parser mirrors the standard version, with bit-list concatenation replacing closure composition. Each configuration's bits accumulate across the whole input so far, so a plain `Vec<bool>` clone-then-extend on every transition is O(current length) per step, $O(n^2)$ over n characters – the same cost `Bits`/`Inj` avoid elsewhere in this and the previous chapter. `AppendBits` is the append-optimized mirror of `Bits` (Section 5.2.1): it stores the reverse of the real sequence internally, so appending a suffix becomes prepending its reverse, reusing `Bits::prepend`'s O(suffix length) guarantee, and materializes the sequence back into a plain `Vec<bool>` only once, when the bits are actually consumed:

```
pub(super) fn step_frontier(
    frontier: &[(Regex, AppendBits)], x: char
) -> Vec<(Regex, AppendBits)> {
    let mut out = Vec::new();
    for (r, bits) in frontier {
        for (r_next, extra) in pderiv_bc(r, x) {
            out.push((r_next, bits.append(&extra)));
        }
    }
    nub2(out)
}
```

Figure 6.9: `step_frontier` (bit-coded): as with the standard parser's version, `nub2` deduplicates across the whole combined state set, not just within one configuration's own expansion (which `pderiv_bc` already does via its own `nub2` call).

Selection walks the state set in priority order and returns the bits of the first accepting configuration, extended with *mkEpsBC* of the residual:

```
pub(super) fn select_bits(
    frontier: &[(Regex, AppendBits)]
) -> Option<Vec<bool>> {
    for (r, bits) in frontier {
        if nullable(r) {
            return Some(bits.append(&mk_eps_bits(r)).to_vec());
        }
    }
    None
}
```

The resulting bit-sequence is interpreted by the shared `decode` function from Chapter 5.2, which walks the original expression and consumes one bit per alternation and per star iteration.

6.2.4 Parser

The parser composes `step_frontier` over the input, checks that the state set is non-empty, and calls `select_bits` followed by `decode`.

```
pub fn parse_pderiv_bc(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier = vec![(r.clone(), AppendBits::empty())];
    for c in input.chars() {
        frontier = step_frontier(&frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    let bits = select_bits(&frontier)?;
    Some(decode(r, &bits))
}
```

Figure 6.10: The bit-coded partial-derivative parser.

The driver has the same structure as the standard one. Only the configuration type and the final step differ: bits and `decode` in place of closures and injection application.

6.2.5 Worked Example: Ambiguous Expression

Consider again the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ on input `ab`.

Forward pass. The initial state set holds a single configuration with empty bits:

$$S_0 = \Big[(r_0,\ [\,])\Big].$$

Transitioning by `a`, the first factor is not nullable, so *pderivBC* recurses into it:

$$\begin{aligned} \mathit{pderivBC}(a + ab, \mathtt{a}) &= \mathit{pderivBC}(a, \mathtt{a}).\mathit{map}(+[\mathit{false}]) \cup \mathit{pderivBC}(ab, \mathtt{a}).\mathit{map}(+[\mathit{true}]) \\ &= \Big[(\varepsilon,\ [\mathit{false}]),\ (b,\ [\mathit{true}])\Big]. \end{aligned}$$

The sequence rule then places the second factor after each derivative:

$$S_1 = \Big[(b + \varepsilon,\ [\mathit{false}]),\ (b\,(b + \varepsilon),\ [\mathit{true}])\Big].$$

Transitioning by `b`, both configurations are stepped. The first configuration reaches the second factor directly:

$$\mathit{pderivBC}(b + \varepsilon, \mathtt{b}) = \Big[(\varepsilon,\ [\mathit{false}])\Big],$$

giving bits $[\mathit{false}, \mathit{false}]$. The second configuration's derivative comes from b inside the leading sequence:

$$\mathit{pderivBC}(b, \mathtt{b}) = \Big[(\varepsilon,\ [\,])\Big],$$

and the outer sequence rule reconstructs the residual $b + \varepsilon$ with the bits unchanged:

$$S_2 = \Big[(\varepsilon,\ [\mathit{false}, \mathit{false}]),\ (b + \varepsilon,\ [\mathit{true}])\Big].$$

Selection and decoding. The first configuration is accepting, so *selectBits* returns its bits extended with $mkEpsBC_{\varepsilon} = [\,]$:

$$selectBits(S_2) = [false, false].$$

Decoding these bits against r_0:

$$decode_{r_0}\Big([false, false]\Big) = \Big(Left(a), Left(b)\Big).$$

The first *false* selects the left branch of the first factor's alternation, committing to the shorter alternative a. The second *false* selects the left branch of the second factor's alternation, committing to the shorter alternative b. The resulting tree is the greedy parse tree, identical to the one produced by the standard partial-derivative parser.

Flattening validates the result:

$$flatten\Big((Left(a), Left(b))\Big) = \texttt{ab}.$$

Both partial-derivative parsers therefore produce the same tree on this example, and both differ from the derivative-based parsers of Chapter 5, which commit to the longer first alternative and return $(Right(a, b), Right())$.

7 External versus Internal Syntax

The four parsers of Chapters 5 and 6 all operate on the six-constructor core `Regex` from Chapter 2. Real patterns are written in a richer surface language. A rule authored by a Snort or SpamAssassin contributor uses shorthand classes, bounded repetition, alternation over character ranges, and a flag suffix — none of which appear in the core language.

Bridging the two is the job of the frontend. It is the layer that takes a pattern string, decides what the pattern means, and produces the `Regex` the parsers consume. This chapter explains that layer in three stages: what intermediate representation it produces (Section 7.2), how it parses a string into that representation (Section 7.3), and how it lowers that representation to `Regex` (Section 7.5). It closes with a treatment of PCRE flags (Section 7.4) and a measurement of the translation's own cost (Section 7.6).

7.1 The Interface and Its Design Choices

Two functions form the public interface. The first, for a bare pattern, is

```
parse_pattern(s: &str) -> Result<Regex, String>
```

The second, for a rule wrapped in the PCRE form `/PATTERN/FLAGS`, additionally reads the flag suffix:

```
parse_pcre_rule(s: &str) -> Result<Regex, String>
```

Both return either a translated `Regex` or a rejection reason. Three design decisions shape everything below.

Full-string matching only. A translated pattern matches the entire input or nothing. There is no search-padding, no substring search, and no anchor handling. This choice is what makes the four parsers of the previous chapters directly usable: each decides membership in the pattern's own language, without a notion of "found somewhere in the input". It is also the reason anchors are rejected rather than translated. A source pattern beginning with `^` or ending with `$` means "match at the start/end of the buffer" under the source tool's own semantics, and no lowering into a full-string matcher preserves that meaning — except in the one case where both anchors wrap the whole pattern, which is provably redundant. That one case is discussed in Section 7.5.3.

A fixed working alphabet. Shorthand classes and negated classes expand to alternations over a fixed set of 98 characters: printable ASCII plus tab, newline, and carriage return. The dot metacharacter lowers to the same set when the `s` flag is set, and to the 97-character complement of `\n` otherwise, matching PCRE's default reading of `.`. A

literal character written directly in the pattern is not bounded by this set — the core `Regex::Lit` carries a full Unicode `char` — so `café` still matches the string `"café"`, but `\W` cannot match a non-ASCII character, since the alphabet has no such character to produce. The size consequences of this choice are quantified in Section 7.6.

Rejection over approximation. When a construct cannot be translated without changing what the pattern denotes, the frontend rejects the pattern rather than silently mismatching it. The rejected constructs are exactly the ones no derivative or partial-derivative parser can consume without extension: backreferences, lookaround assertions, word boundaries, and anchors outside the recoverable top-level `^...$` wrap. The two remaining flags, `m` and `x`, are rejected for the same underlying reason: honouring them would require a matching model this frontend does not have. `m` redefines `^` and `$` to match at newlines as well as at buffer boundaries, and the frontend has no anchors to begin with; `x` makes whitespace insignificant, which this parser does not model.

Everything outside this small set is accepted and translated. The remainder of this chapter explains how.

7.2 The External AST

The frontend parses into an intermediate type, `ExtPat`, which sits between the surface string and the core `Regex`. It carries enough structure to preserve what the source pattern says — precedence, grouping, quantifiers, and character classes as units — without committing to any particular lowering. The 19 variants fall into four groups.

Group	Variant	Surface syntax
Structural	`Empty`	empty branch of `a|` or `(|b)`
	`Concat`	`abc`
	`Or`	`a|b|c`
	`Group`	`( ... )`
	`GroupNonMarking`	`(?: ... )`
Quantifier	`Opt`	`r?`
	`Plus`	`r+`
	`Star`	`r*`
	`Bound`	`r{m,n}`
Class	`Any`	`[abc]`
	`NoneOf`	`[^abc]`
	`Escape`	`\d`, `\w`, `\s`, ...
	`Char`	a single literal
Atomic / special	`Carat`	`^`
	`Dollar`	`$`
	`Dot`	`.`
	`WordBoundary`	`\b`, `\B`
	`Eps`	ε
	`Never`	$\emptyset$

Table 7.1: The 19 `ExtPat` variants, grouped by role. Of the six in the last group, `Carat`, `Dollar`, and `WordBoundary` are parsed but rejected by `translate`; `Dot` lowers to the working alphabet (or its `\n`-free complement); `Eps` and `Never` lower to the core `Regex::Eps` and `Regex::Phi`.

Two properties of `ExtPat` matter for the rest of the chapter. First, the quantifier variants

are single-level: `r*` carries its atom, but not a chain. Chained quantifiers such as `a**` are not produced by the parser, since PCRE treats them as syntax errors. Second, the anchor and word-boundary variants are first-class. Parsing does not reject them; it produces an `ExtPat::Carat` or `ExtPat::WordBoundary` and leaves the rejection to `translate`. The reason is diagnostic: a pattern that fails at translation with the message "anchor is not supported" is more informative to a corpus analyst than one that fails at parse time with "unexpected `^`".

Rejection at parse time covers two distinct cases. The first is constructs the parser cannot even represent: backreferences and lookaround assertions have no `ExtPat` constructor, so the parser has no tree to produce for them. The second is POSIX classes, which *are* representable — as the literal characters `[`, `:`, `a`, and so on — but whose silent misinterpretation would change the pattern's language. Their rejection is a correctness measure, described in Section 7.3.3.

The deliberate asymmetry between parse and translate — parse succeeds on more than translate accepts — is what makes a per-rejection-cause breakdown of a corpus possible at all. Without it, a pipeline could only say how many patterns failed, not why.

7.3 Parsing: String to ExtPat

Parsing is a single recursive-descent pass, one function per level of the grammar. Precedence is fixed by the call chain.

7.3.1 Precedence and associativity

Three levels, from loosest to tightest binding:

1. alternation (`|`), left-associated;
2. concatenation (implicit), left-associated;
3. postfix quantifiers (`?`, `+`, `*`, `{m,n}`), at most one per atom.

So `abc*` parses as `ab(c*)` and `a|b|c` parses as `(a|b)|c`. The parser never relies on these associations for correctness — `translate` produces the same `Regex` for any equivalent bracketing of an associative operator — but the association is observable in the resulting `Regex` tree, and Chapter 8's growth measurements depend on it.

A trailing `?` after a quantifier (`r*?`, `r+?`, `r??`, `r{m,n}?`) is accepted and discarded. Laziness is a backtracking-engine concept: it governs the order in which a search explores matches, and it becomes observable only when that order changes an externally visible result — which substring a search returns, or what a capture group holds. Neither channel exists here. The core `Regex` has no capture structure, and the parsers of Chapters 5 and 6 decide membership rather than search, so no operation ever returns a substring. The marker is therefore discarded during parsing, before any `ExtPat` is built: `r*` and `r*?` produce the same `ExtPat::Star`.

This is a separate concern from the disambiguation policy the parsers apply. A source pattern such as `(a+ab)(b+\varepsilon)` admits more than one valid parse tree for the same input, and which tree a parser returns is decided by the parser's own policy (POSIX leftmost-longest for the derivative-based parsers, greedy leftmost for the partial-derivative

ones), not by anything in the source quantifier syntax. The `?` marker is not the mechanism by which a PCRE author selects between those policies; it is a hint that only a backtracking engine reads, and there is no backtracking engine here to read it.

7.3.2 Accepted surface syntax

Beyond the core constructs already listed in Table 7.1, the parser accepts several forms from the PCRE family.

Groups. `( ... )` and `(?: ... )` both parse; the distinction is preserved in `ExtPat` but erased at translation. Named groups `(?<name>...)` and `(?P<name>...)` are accepted with the name dropped. The name is not recoverable from the resulting `Regex`, since `Regex` has no capture structure, and the parser does not attempt to preserve it.

Character classes. A class body is read as a sequence of members. A `-` between two literal characters denotes a range; a leading or trailing `-` is a literal member, and so is a leading `]`. Shorthand escapes inside a class (`\d`, `\w`, `\s`, and their capitalised negations) expand inline as separate members. A shorthand cannot be a range endpoint: `[a-\d]` is rejected.

Escapes outside a class. `\n`, `\t`, `\r` produce the corresponding control characters; `\xHH` produces the byte given by two hex digits; `\0` produces an octal escape (up to two further octal digits). A single digit `\1`–`\9` is rejected, since PCRE resolves those as backreferences using a capture-group count this parser does not track.

Octal escapes inside a class. Inside a class, a `\` followed by an octal digit reads up to three octal digits. This is a real-corpus concern: SpamAssassin rules use patterns such as `[\042\223\224\262\263\271]` to match Windows codepage punctuation, and misreading `\042` as the three literal characters `0`, `4`, `2` would silently change the pattern's language. `\8` and `\9` are not octal digits and are read as literals `8` and `9`, matching PCRE's own behaviour.

Explicit ε and $\emptyset$. The two Unicode tokens are accepted as `ExtPat::Eps` and `ExtPat::Never`. They are extensions for the worked examples of earlier chapters, not part of the PCRE family.

7.3.3 Rejected surface syntax

Rejection falls into two categories: constructs the grammar cannot represent, and constructs the frontend chooses not to represent.

Not representable. Backreferences (`\1`–`\9`, `(?P=name)`), lookahead `(?=...)` and `(?!...)`, and lookbehind `(?<=...)` and `(?<!...)` are rejected at parse time. None of them is a regular-language construct, and the external AST has no constructor for them. Real corpus examples:

```
/^(?P<addr1>.{4})(?P<addr2>.{4})(?P=addr2)(?P=addr1)/Rsi
    (Snort, named backreference)
/(?=(?:(0x[a-f0-9]{2}|0+\d{1,3})\s*?,\s*?)*?\d{1,3})...,/Rsi
    (Snort, chained lookahead)
```

Representable but rejected. Word boundaries `\b` and `\B`, and anchors `^` and `$`, parse successfully into `ExtPat` but are rejected by `translate`. Both are position-dependent: their meaning depends on the boundary between the matched substring and the surrounding buffer, which a full-string matcher has no way to model. Rejecting them at parse time would have worked just as well, but the parse-time success lets the reject message name the construct. Corpus examples:

```
/\bsolusvmc-node\b/          (Snort, word boundary)
/\.ida$/iU                   (Snort, end anchor)
/^Argument\s+\//smi          (Snort, start anchor + m flag)
```

POSIX classes. A class of the form `[:name:]` inside a bracket expression is rejected with a specific message. Without this check, `[[:alpha:]]` would parse as the five literal characters `[`, `:`, `a`, `l`, `p`, followed by `h`, `a`, and a closing bracket — a pattern that denotes a completely different language and that would have been accepted silently. The rejection is a correctness measure, not a stylistic choice.

7.4 PCRE Flags

A pattern passed to `parse_pcre_rule` may be wrapped in a `/PATTERN/FLAGS` form. The frontend reads the flag suffix and sorts the characters into three groups: honoured, rejected, and ignored.

Flag	Group	Effect
`i`	honoured	case-fold every literal letter
`s`	honoured	`.` includes newline
`m`	rejected	would change anchor meaning this frontend lacks
`x`	rejected	would change whitespace meaning this parser lacks
`R`	ignored	positioned a search this frontend does not perform
`A`	ignored	anchored a search this frontend does not perform
`E`	ignored	tuned a `$` this frontend rejects
`U`, `H`, `P`, `C`, ...	ignored	named a buffer or normalisation strategy

Table 7.2: The three flag groups. "Ignored" means the pattern parses exactly as it would without the flag.

The distinction between "honoured" and "ignored" is whether the flag changes what the pattern denotes. A buffer selector such as `U` says which region of the input to test against, not what the pattern itself matches; under full-string matching there is only one region. A relative-position flag `R` says how to position a subsequent search, which does not exist in this frontend. Neither of these changes the language the pattern denotes, so both are harmless to ignore. An honoured flag such as `i` does change the language, so it must be applied.

Honoured flags are applied between parse and translate, as preprocessing passes over the `ExtPat`. This is the natural place: the passes are structural rewrites that do not need the pattern's own scope information — `case_fold` turns every literal letter into a two-element `Any`, and `strip_dot_newline` turns every `Dot` into `NoneOf(\n)` — so they can run once on the whole tree.

7.5 Translation: ExtPat to Regex

Translation is a second structural pass, this time over the whole `ExtPat`, producing a `Regex`. Most variants have a direct counterpart: `Concat`, `Or`, `Opt`, `Plus`, and `Star` map to the corresponding core constructors or a one-line desugaring of them. Three parts of the translation are less direct and are described below.

7.5.1 Character classes and the working alphabet

An `ExtPat::Any(cs)` lowers to an alternation over `cs`: one `Alt` node per element, folded left. `NoneOf(cs)` first takes the complement of `cs` against the working alphabet and then does the same. `Dot` lowers to `Any` over the full alphabet when the `s` flag is set, and has already been rewritten to `NoneOf(\n)` otherwise. Shorthand escapes (`\d`, `\w`, `\s`, and their capitalised forms) lower through the same path.

The consequence is that `Dot`, the shorthand classes, and any negated class cannot match a character outside the working alphabet. The evaluation's corpus contains a few Snort patterns that include non-ASCII literals; those are unaffected, because a direct literal lowers to a single `Regex::Lit` that carries the full `char`. The working alphabet only constrains the constructs that need to enumerate a set.

7.5.2 Bounds and unrolling

`ExtPat::Bound` is the only quantifier without a direct core construct, since `Regex` has no `{m,n}` form. It lowers by unrolling: `r{m,n}` becomes `m` exact copies of the translated body, followed by `n-m` optional copies. `r{m,}` becomes `m` exact copies followed by a `Star`. Both forms clone the body, so the resulting `Regex` has size proportional to the bound.

Two caps guard this unrolling. A bound whose upper or lower endpoint exceeds 1000 is rejected rather than unrolled. Real Snort patterns sometimes carry a bound in the thousands — `/(GET).\/%.{1586,}/i` is one such rule — and unrolling 1586 copies would blow up the subsequent derivative computation for no matching benefit. The cap is enforced before any translation, so the rejection is immediate.

A second, coarser cap is applied further downstream, by the corpus preparation of Chapter 8, and rejects a pattern if the translated `Regex` would exceed a node count. That cap does not affect the frontend's own behaviour, only the pipeline's decision to skip a pattern before generation.

7.5.3 Anchor recovery

Translation is where anchors are rejected, with one exception. A pattern whose top level is a single `Concat` beginning with `Carat` and ending with `Dollar` has those two anchors stripped before translation. The reasoning is that under full-string matching they are provably redundant: consuming the whole input already requires both endpoints, so removing them changes nothing about what the pattern denotes.

The exception is narrow. A lone `^`, a lone `$`, or either anchor nested inside an alternation branch or under a quantifier is not stripped, and reaches `translate` where it is rejected.

The recovery is not part of the frontend's own public interface. Neither `parse_pattern` nor `parse_pcre_rule` strips an anchor; both translate every anchor to a rejection. A caller that has a reason to recover the redundant wrap can do so before invoking `translate`, and the corpus preparation of Chapter 8 is such a caller. The frontend's behaviour is unchanged either way: it accepts every pattern that `translate` accepts, and rejects every anchor on its own.

The evaluation classifies every accepted pattern by whether it carried the recoverable wrap or had no anchor at all. Both classes are generated for and benchmarked identically, since the anchors' redundancy means the two patterns have the same `Regex`.

7.6 Translation Size

Because classes expand to alternations and bounds unroll, the translated `Regex` can be much larger than the source pattern. The demo `demo_frontend_translation_sizes` measures this directly.

Pattern	Nodes	Pattern	Nodes
`a*`	2	`\d`	19
`abc`	5	`\d{2}`	41
`a{2,4}`	15	`[a-c]+\d{2}`	54
`a+`	4	`\w`	125
`a?`	3	`[^a]`	193
`a{2}`	5	`.`	195
`a{2,}`	8	`\w+`	252

Table 7.3: Translated size in `Regex` nodes for representative patterns.

The right-hand side of the table shows the cost of the 98-character alphabet. A single `\d` contributes 19 nodes: one for each of the ten digits, plus nine `Alt` wrappers to chain them. `\w` is 125 nodes for 63 characters. `[^a]` is 193 nodes: one for every alphabet character except `a`, plus the chain.

Two consequences follow. First, patterns with many distinct classes can produce large translations even when the source is short, which is why a downstream pipeline carries a node-count cap. Second, the constant factor between "one class" and "one literal" is roughly two orders of magnitude, so a pattern written as an explicit alternation over literal characters (`(a|b|c|...)`) is not equivalent, size-wise, to the same class written as `[abc...]`. Both lower to the same `Regex`, but the class notation makes the size cost visible only after translation.

Case folding adds to this cost linearly. For `/abc/i`, the translated pattern grows from 5 nodes (for `/abc/`) to 11, an increase of 6 nodes, i.e. 2 per letter. Each letter is replaced by a two-element class, and each such class contributes one extra `Alt` wrapper.

7.7 Summary and Bridge to Evaluation

The frontend sits between two formalisms. On one side is the surface syntax that real pattern authors write; on the other is the six-constructor `Regex` the four parsers of the

previous chapters can consume. The bridge between them is a two-pass design — parse to `ExtPat`, then translate to `Regex`, with flag-driven preprocessing in between — whose every choice has a consequence in the evaluation that follows.

Three consequences are worth naming explicitly.

Every rejection is a frontend decision. None of the four parsers rejects a pattern. Every pattern that reaches a parser has already been through the frontend's filter, and the reasons a pattern is filtered out are collected in Chapter 8's rejection table. That table is therefore a statement about the frontend's scope, not the parsers' abilities.

The corpus is bounded by the working alphabet. On the patterns the evaluation uses, this bound is not restrictive: the corpus is text-oriented, and every pattern that survives translation matches only characters the alphabet contains. But the choice is visible in the translated sizes above, and it is worth a note in the evaluation's threats-to-validity section.

Translation size is not parser cost. A pattern that lowers to 250 nodes is larger to hold in memory than one that lowers to 5, but the two parsers that handle it may still perform similarly. The evaluation separates the two: the frontend's own cost is what Table 7.3 measures, and the parsers' cost is what the timing and memory benchmarks measure. Conflating the two is easy — a pattern's "complexity" can mean either — so the distinction is maintained throughout.

8 Evaluation

This chapter evaluates the four parser variants implemented in Chapters 5 and 6. The four candidates are:

- the standard derivative-based parser (`deriv_std`, direct and loop variants),
- the bit-coded derivative-based parser (`deriv_bc`),
- the standard partial-derivative parser (`pderiv_std`),
- the bit-coded partial-derivative parser (`pderiv_bc`).

Three aspects are covered. Section 8.1 describes how the corpus is prepared and how the verified inputs are generated from real patterns. Section 8.2 then checks, on the prepared corpus, how often the POSIX parsers and the non-POSIX parsers agree on the shape of the parse tree they build for the same pattern and input. The main focus of the chapter, however, is Section 8.3, which compares the four variants on execution time and memory consumption, on both synthetic inputs and the prepared corpus.

The chapter is organised as follows. Section 8.1 describes the corpus and the pipeline that produces the benchmark inputs. Section 8.2 verifies parser agreement. Section 8.3 presents timing and memory results. Section 8.4 interprets them.

All measurements are produced by binaries under `benches/` and `examples/`, run with a fixed `-release` profile. The synthetic benchmarks are Criterion targets; the two corpus measurements are plain reporting binaries, run once per source, for the reasons given in `BENCHMARKS.md`.

8.1 Dataset Preparation

8.1.1 Source selection

The pipeline supports five pattern sources. Two of them are used in this evaluation: Snort/Suricata and SpamAssassin. Both are maintained upstream, their syntax is documented by the projects that publish them, and their patterns are exercised in production. This makes the corpus a reasonable proxy for the patterns a real parser will encounter.

The remaining three sources — RegexLib, S2ELab, and Innovatorved — are opt-in. RegexLib and Innovatorved are crowd-sourced and carry no correctness guarantee; Innovatorved's test text is unlabeled. S2ELab is a curated academic benchmark, but not a maintained production rule set in the way the two used here are. They are described

in Section 8.1.3 for completeness, since the pipeline handles them, but no results in this chapter are drawn from them.

The corpus is the union of every rule file fetched as described in `DATASET.md`: 47 Snort rule files under `data/raw/snort`, totalling 16,774 extracted patterns, and 62 SpamAssassin rule files under `data/raw/spamAssassin`, totalling 549 extracted patterns.

The two sources differ in character, and this difference matters for the rest of the chapter.

Snort signatures are byte-oriented and often written to defeat obfuscation. A typical pattern mixes hexadecimal escapes, alternation over syntactic variants, and a case-insensitive flag:

```
/Authorization\x3a\s*Basic\s*[a-zA-Z0-9]{255,}==/i
/eval\x28(String\x2EfromCharCode\x28
      |[a-z,0-9]{1,20}\x28String\x2EfromCharCode\x28)/i
```

The first matches HTTP `Authorization: Basic` headers with an unusually long base64 payload; the second matches two spellings of the JavaScript `eval(String.fromCharCode(...))` idiom. Both are long enough that a derivative parser's state grows visibly across the match.

SpamAssassin rules are keyword-oriented and much shorter. A typical pattern is a single word with optional separators, plus a case-insensitive flag:

```
/xan[ae]x/i
/l.{0,2}e.{0,2}v.{0,2}i.{0,2}t.{0,2}r.{0,2}a/i
/boundary="[\da-f]{8}(?:-[\da-f]{4}){3}-[\da-f]{12}"/
```

The first matches the drug name *Xanax*; the second matches the word *levitra* with up to two arbitrary characters between letters, a common spammer evasion; the third validates a MIME boundary UUID. Their short length is why the SpamAssassin corpus produces smaller but more numerous per-pattern measurements.

Each source is chosen to exercise a different combination of the cost drivers introduced in Section 8.3.1: Snort for long inputs, S2ELab and RegexLib for structural variety, SpamAssassin as a short-pattern baseline.

8.1.2 Extraction

Extraction is the first stage of the pipeline. Each source file is read line by line, and patterns are pulled out as described in `src/data/extract.rs`: for Snort, every `pcre:"..."` option paired with its `content:` fields, `msg`, and `sid`; for SpamAssassin, every `header` and `body` rule.

The pipeline was run with a per-pattern target of $N = 10$ verified inputs, split evenly into five matches and five non-matches:

```
cargo run --release --example dataset_prepare -- \
    10 suricata data/raw/snort/*.rules
cargo run --release --example dataset_prepare -- \
    10 spamassassin data/raw/spamAssassin/*.cf
```

The first positional argument is the per-pattern target N: the total number of verified inputs each surviving pattern contributes, split evenly between matches and non-matches. It is required and must be a positive even integer. An optional literal `-` between N and the source name is accepted and skipped.

8.1.3 Input generation

Every accepted pattern needs inputs before it can be benchmarked: at least $N/2$ matching and $N/2$ non-matching. The five sources differ in whether they supply such inputs themselves, and if so, in what form.

Source	Given input?	Generation strategy
Suricata	no	structural + content-derived
SpamAssassin	no	structural + corpus sample
RegexLib	no	structural
S2ELab	yes (labeled)	given, with structural fill
Innovatorved	yes (unlabeled)	given, with structural fill

Table 8.1: Source of candidate inputs per dataset.

Suricata. A Snort rule is a conjunction of constraints; nothing in it is a complete candidate string. The extractor takes the first `pcre:"..."` option on the rule line and ignores any subsequent one, so a rule that carries two distinct patterns contributes one to the corpus, not two. The `content:` fields on the same line are not input: they are fragments a triggering payload must contain, with positional constraints (`distance`, `within`, `depth`, `offset`) that the pipeline parses but does not enforce. Generation runs two strategies in parallel. `generate_sample` walks the translated `Regex` and produces structural positives and negatives. `generate_from_content_fields` concatenates the extracted `content:` bytes with a short filler between fields, producing strings shaped like real payloads. Both feed the same verification pass; only candidates that match the translated pattern (or fail late, for negatives) survive. A Snort rule with `pcre:` but no `content:` generates candidates only via the structural sampler; a rule with `content:` but no `pcre:` is skipped entirely by the extractor, since there is no pattern to translate.

SpamAssassin. Each `header` or `body` rule carries one pattern, and no test messages are shipped with the rules. Generation is structural plus, when a local ham/spam corpus is configured through `-corpus-dir`, a scan of that directory for messages whose entire content matches the pattern. Under full-string matching the corpus scan finds little against a real corpus, since most real messages are not themselves exact matches of a short pattern; the structural sampler is what supplies the bulk of the surviving inputs.

RegexLib. Structural generation only. No side input is available from the source.

S2ELab. Each entry ships with author-labeled `matches` and `non_matches`. Generation prefers these: they are wrapped as candidates unchanged, deduplicated across both sides, and passed to verification. If after verification either side has fewer than $N/2$ cases, structural sampling fills the gap, so a pattern with fewer given examples than the target is still usable provided its own language admits additional draws. A pattern whose given

lists are empty on one side *and* whose language is too small to sample from cannot reach the target and is excluded.

Innovatorved. Each entry ships with free-form test text — a mix of matching and non-matching lines with no marker saying which is which. Generation prefers this text: it is split into lines, deduplicated, sampled down to a cap, and each surviving line is classified against the real parser (matching or not) before being handed to verification as any other candidate. If after verification either side has fewer than $N/2$ cases, structural sampling fills the gap, as for S2ELab. A pattern with no test text, or with test text that yields too few surviving cases on one side and a language too small to sample from, is excluded.

Common rule. In every case, candidates that do not match their claimed outcome under the real parser are discarded, and negatives must additionally fail *late* — past two thirds of the input — to count as meaningful. The balanced target of exactly $N/2$ matches and $N/2$ non-matches per pattern, described in Section 8.1.4, applies to all five sources: whether an input comes from the source itself or from structural fill, the balanced corpus carries the same count per pattern and the same split.

Candidate budget. Candidate generation is bounded per pattern. Each strategy produces at most $slack \times (N/2)$ candidates per side, where *slack* is tuned per source by how reliably that source's candidates verify: structural sampling walks the pattern's own language and needs no slack (1); content-derived and corpus messages are real bytes and also verify reliably (1); given examples and classified lines often fail and need more room (3 for RegexLib, 5 for S2ELab, 10 for Innovatorved). The cap is what makes the pipeline's per-pattern cost independent of the size of a rule file.

8.1.4 A worked example through the pipeline

Before the aggregate counts, one pattern is traced from raw text to prepared case, so that the later tables have a concrete referent.

Consider the Snort rule:

```
alert http $EXTERNAL_NET $HTTP_PORTS -> $HOME_NET any (
  msg:"ET WEB_CLIENT Possible IE URI Validation";
  content:"#|3A|../../"; content:"C|3A 5C|";
  nocase; within:50;
  pcre:"/\x2E\x2E\x2F\x2E\x2E\x2F.+C\x3A\x5C[a-z]/si";
  sid:2010798; rev:4;)
```

Extract. The extractor scans the line for `pcre:"...` and `content:"..."`. It produces one `ExtractedRule` whose `raw_pattern` is `/\x2E\x2E\x2F.../si`, and whose source-specific context records two `content:` fields (`#:../../` and `C:\`), a `msg`, and a `sid`.

Translate. `core_regex_tagged` strips the `/.../si` wrapper, applies `case_fold` for the `i` flag, leaves `.` as the full alphabet since the `s` flag is set, and translates the remaining pattern to a `Regex`. The result carries `MatchIntent::UnanchoredAmbiguous`, since the pattern has no top-level `^...$` wrap.

Generate. Because the rule is Snort and the extractor recorded two `content:` fields, generation runs in two ways. The structural sampler walks the translated `Regex` and produces strings from the pattern's own language. The content-derived sampler joins the `content:` bytes with a short filler between them, and produces strings that look like real HTTP requests:

```
#:../../C:\
#:../../xxC:\a
#:../../abC:\z
```

The filler between the two fields is drawn from the pattern's own language, so the generated string stays compatible with the pattern's match constraint.

Verify. Each candidate is checked against the real parser. Structural positives and content-derived candidates are asserted to match; structural negatives (produced by corrupting the last character of a positive) are asserted not to match, and additionally asserted to fail *late* — after at least two-thirds of the input. Candidates that fail verification are discarded. Surviving candidates are written to `data/processed/snort/prepared.jsonl` as one JSON line per case.

This worked example is the pattern the rest of the chapter refers to when it discusses *generated* versus *verified* candidates.

Input generation is bounded by two distinct limits. The first is sampling effort: a rich language such as `[a-z]+` admits infinitely many matching strings, but the sampler draws them randomly and can spend many attempts producing duplicates before reaching the target count. This limit is practical, not fundamental, and is addressed by retrying with a larger attempt budget. The second is the language's own size. A pattern such as `a{3}` denotes exactly one string, and no amount of sampling can produce ten distinct matching inputs from it. This limit is fundamental: the pattern's language is smaller than the target count, and it cannot be met by any generation strategy. Patterns that fall into the second category are recorded as unusable at generation and are not written to `prepared.jsonl`, since admitting a pattern with fewer than the standard number of inputs would make its benchmark results incomparable to the rest of the corpus.

8.1.5 Corpus overview

Table 8.2 shows the aggregate extraction counts. Snort contributes 16,774 extracted patterns; SpamAssassin 549. This three-order-of-magnitude gap is expected: Snort rule lines can carry several `pcre:` options and each is extracted separately, while SpamAssassin exposes at most one pattern per rule.

Source	Files	Extracted	Accepted	%	Rejected	%
Snort	47	16774	6123	36.5	10651	63.5
SpamAssassin	62	549	162	29.5	387	70.5
Total	109	17323	6285	36.3	11038	63.7

Table 8.2: Corpus coverage after extraction and translation.

The overall acceptance rate of 36.3% is not a frontend deficiency. The rejected majority are patterns the four parsers in this thesis cannot meaningfully evaluate: lookaround

assertions and backreferences are not regular, and anchors fall outside the full-string matching model chosen in Chapter 7. The acceptance rate differs by source: Snort's 36.5% is dominated by patterns that carry an explicit `^...$` wrap, which `core_regex_tagged` recovers; SpamAssassin's 29.5% has more word-boundary rejections, since SpamAssassin uses `\b` pervasively to anchor keyword matches.

8.1.6 Match intent

Accepted patterns are tagged with one of two `MatchIntent` values, described in `DATASET.md`.

Source	Direct	%	Explicit `^...$`	%
Snort	5573	91.0	550	9.0
SpamAssassin	139	85.8	23	14.2
Total	5712	90.9	573	9.1

Table 8.3: Accepted patterns by match intent.

The distribution is skewed toward `UnanchoredAmbiguous`. Snort and SpamAssassin's own engines search a buffer by default, and their rule authors write accordingly. Only 9% of accepted patterns carry an explicit full-match wrapper.

Both intents are generated for. Positives are sound under either reading, so the benchmark inputs are used unchanged.

8.1.7 Rejection causes

Table 8.4 disaggregates the rejected patterns by the reason the frontend gave. The table is transposed for legibility: the causes, which are short, are rows, and the sources are columns.

Cause	Snort	SpamAssassin	Total
Anchor	6264	142	6406
Multiline flag	3963	19	3982
Backreference	215	8	223
Word boundary	19	164	183
Lookaround	98	50	148
Extended flag	0	0	0
Other	92	4	96
Total	10651	387	11038

Table 8.4: Rejection causes across both sources.

Four causes dominate. *Anchor* (6406 patterns) is the largest. It includes every pattern whose `^` or `$` is not part of a top-level `^...$` wrap: a lone start or end anchor, or one nested inside an alternation branch or under a quantifier. The frontend rejects these deliberately (Chapter 7), since removing them would change what the pattern denotes. Two concrete examples:

```
/\.ida$/iU                  (Snort, end anchor only)
/^Argument\s+\//smi         (Snort, start anchor plus m flag)
```

Multiline flag (3982 patterns) is second. A Snort pattern ending in `/smi` or similar carries the `m` flag, which is rejected because anchors and newline-sensitive `^`/`$` have no meaning in a full-string matcher.

Backreference (223 patterns) and *lookaround* (148 patterns) are both small relative to the corpus size, but they are the two most interesting causes from a theoretical standpoint: they are the constructs that place a pattern outside the regular languages. Example patterns:

```
/^(?P<addr1>.{4})(?P=addr2)(?P=addr1)/Rsi
/(?=(?:(0x[a-f0-9]{2}|0+\d{1,3})\s*?,?...))
```

The first is a Snort rule with a named backreference; the second is a Snort rule with a nested lookahead. Both are rejected before an `ExtPat` exists, so no tree is available to recover.

The *other* column collects patterns that fail for reasons not covered by the six named causes. For Snort, this is almost entirely the `MAX_BOUND` cap of 1000 repetitions, triggered by patterns like `/{GET}./%.{1586,}/`. For SpamAssassin, the same cap and a handful of syntax errors.

8.1.8 Anchor shape distribution

Rejection causes alone do not explain how many patterns carry anchors; they only explain how many were rejected for that reason. To show the full picture, the pipeline records the anchor shape of every extracted pattern, accepted or not.

Shape	Snort	SpamAssassin	Total
None	5739	304	6043
Start-only	5934	136	6070
Fully-wrapped	2783	34	2817
End-only	1669	8	1677
Unparseable	372	63	435
Nested	277	4	281
Total	16774	549	17323

Table 8.5: Anchor shape of every extracted pattern.

The rows sum to the extracted count. Three observations.

Fully-wrapped (2817) is much larger than the accepted `ExplicitFullMatch` count in Table 8.3 (573). The difference is the fully-wrapped patterns that were rejected for a different reason: a pattern can be wrapped in `^...$` and still carry a lookaround or a word boundary, and the earlier cause wins.

Start-only (6070) and *end-only* (1677) together account for most of the anchor-caused rejections. These are not recoverable by the full-match reading: a start anchor alone leaves the end of the match open, and no amount of full-string matching can restore the constraint.

Unparseable (435) are patterns rejected before an `ExtPat` exists: backreferences, lookaround assertions, or genuine syntax errors. These are the entries that need the parse step's own rejection mechanism, since no tree exists to strip anchors from.

8.1.9 Prepared cases

The pipeline writes accepted cases to `data/processed/<source>/prepared.jsonl`. Each line is one verified (pattern, input, outcome) triple. The corpus contains 6,285 accepted patterns, 87,283 generated candidates, and 61,712 verified candidates.

The gap between generated and verified is worth explaining, since it appears in every benchmark that follows. Candidate generation produces two kinds of input: positives, which *claim* to match, and negatives, which claim not to. Verification walks candidates in order and stops as soon as both sides have reached $N/2$ accepted cases; candidates the pipeline never reaches are skipped, not rejected. A candidate that is reached and fails verification is discarded: its claim did not hold under the real parser, or it was a negative that failed *early* — before two-thirds of the input — which is not a meaningful negative. For Snort, 85,997 candidates were generated and 60,426 survived (70% survival); for SpamAssassin, 1,286 generated and 1,286 survived (100%). The difference is that SpamAssassin's short, regular patterns generate cleanly, while Snort's longer patterns occasionally produce a structural sample that the case-folding or anchor-stripping steps have made unmatchable.

A small fraction of accepted patterns contributes nothing to the corpus. These are not sampler failures. A structural negative is a matching input with its last character replaced by a character that lies outside the pattern's working alphabet, so a pattern whose language covers the alphabet (any star expression, any class-based pattern) still has a valid negative: the corrupted string contains a character no member of its language could contain. What remains are the patterns whose language is on one side empty: an expression that translates to the empty-string language, such as `^$` after anchor stripping or `a{0}`, has no structural negative to draw; an expression that translates to the empty language has no positive to draw. (Negatives that come from a source's own given data — S2ELab's `non_matches`, Innovatorved's classified test lines — are unaffected by this: they are not produced by corruption and are evaluated on their own terms.) In both cases, one side has zero members, and the pipeline excludes the pattern rather than fabricating a member that would not be a real test. The count appears as *patterns below target* in the pipeline report and accounts for the difference between *accepted* and *usable* patterns.

8.2 Parser Agreement

Every case in the prepared corpus already carries a verified outcome, so membership is a precondition rather than something this section establishes. The question here is the shape of the parse tree. On a matching case, the POSIX parsers and the non-POSIX parsers may build different trees — the same input can admit more than one valid decomposition — but some patterns and inputs leave neither policy a choice, forcing both onto the same structure. This section measures how often that happens across the corpus.

8.2.1 Method

The check is implemented in `demo_parser_agreement`. For every prepared case, all five parsers — `deriv_std_rec`, `deriv_std_loop`, `deriv_bc`, `pderiv_std`, `pderiv_bc` — run on the same (pattern, input) pair.

Two conditions are checked as preconditions. First, every parser returns the case's verified

membership outcome. Second, within each family the parsers agree with each other on the tree: the three POSIX variants build the same tree, and the two non-POSIX variants build the same tree. A violation of either condition is a bug, not a measurement.

The measured quantity is cross-family tree agreement: of the cases where both families matched and were internally consistent, how many produced the same tree. The two families apply different disambiguation policies, so a difference is expected where the input admits several decompositions; a match where the input admits only one forces both families onto the same structure.

Each parser call runs in a subprocess so that a stack overflow or an infinite loop is contained. A 2-second wall-clock timeout kills the process. Timeouts and crashes are counted separately from tree-shape outcomes, since neither produces a result to compare.

```
cargo build --release --example demo_parser_agreement_worker
cargo run --release --example demo_parser_agreement
```

8.2.2 Results

Table 8.6 reports the outcome. Precondition columns are grouped on the left, the measured quantity on the right.

Source	Entries	Timeouts	Crashes	Mem.	POSIX	non-POSIX	Compared	Same	Diff
Snort	–	–	–	–	–	–	–	–	–
SpamAssassin	–	–	–	–	–	–	–	–	–
Total	–	–	–	–	–	–	–	–	–

Table 8.6: Parser agreement over the prepared corpus. (Values to be filled after the run.)

The columns are: *Entries*: prepared cases processed. *Timeouts*, *Crashes*: contained-failure counts, neither counted as a tree-shape disagreement. *Mem.*: precondition violation — a parser's membership outcome disagreed with the verified outcome. Expected zero. *POSIX*, *non-POSIX*: precondition violations — within-family tree disagreements. Expected zero. *Compared*: matching cases where both families were internally consistent, so the cross-family comparison was meaningful. *Same*, *Diff*: of *Compared*, how many produced the same tree and how many produced different trees.

8.2.3 Discussion

To be completed after the corpus run. Expected reading: no precondition violations (all of *Mem.*, *POSIX*, and *non-POSIX* at zero), timeouts and crashes nonnegative, and the *Same* / *Diff* split showing how often the two policies are forced onto the same structure. *Diff* is not a bug: it reflects a difference in disambiguation policy between the two families, not in the underlying matching.

8.3 Performance Analysis

The four parsers are compared on two axes: execution time (Section 8.3.2) and memory consumption (Section 8.3.3). Both axes are measured on synthetic inputs first, since synthetic families isolate a single cost, and on the corpus second, since the corpus is the environment the parsers will encounter in practice.

8.3.1 Cost drivers

Before the measurements, it is worth separating the properties of a corpus that make the four parsers cost something. Three are distinguishable, and each is exercised to a different degree by the corpus of Section 8.1 and by the synthetic families of Section 8.3.2.

Input length. The two standard derivative parsers retain every intermediate expression until the backward pass reconstructs the tree, so their live state grows with the number of characters consumed. The bit-coded and partial-derivative parsers do not carry that history. A corpus of long inputs separates the two groups; a corpus of short inputs does not, however many entries it contains.

Pattern structure. Translation size and derivative cost scale with the pattern's own node count. Deeply nested groups, alternation over many branches, and repeated classes inflate the translated `Regex`, and every parser pays for that size on every input. This driver is independent of input length: a short input against a large pattern can cost more than a long input against a small one.

Ambiguity. When a single input admits more than one valid decomposition of the pattern, the parsers keep more state alive and must eventually choose among the surviving trees. The choice is governed by the parser family's disambiguation policy (Chapter 5 versus 6), and the two families are visible on exactly those (pattern, input) pairs where the policies diverge. Ambiguity is a property of a pattern, not of a corpus; the synthetic family of Section 8.3.2 is what isolates it.

Not every cost is a driver. There is also a fixed per-call cost paid once per (pattern, input) pair regardless of length, structure, or ambiguity: allocating the initial residual, setting up the derivation loop, and constructing the parse tree from the final nullable expression. The `small_patterns` family isolates this cost, and it is why the four parsers differ even on inputs where none of the three drivers is active.

Each of the synthetic families of Section 8.3.2 is shaped to isolate one of these costs; the corpus tables that follow mix all three, and reading them without this separation makes the numbers hard to interpret. Snort, whose patterns are long and whose content-derived inputs are byte-shaped, exercises the first driver most strongly. S2ELab and RegexLib, whose patterns vary in structural complexity but whose inputs are short, exercise the second. The third is not a property of any one source, and is why the synthetic side of the evaluation exists at all.

8.3.2 Timing

Synthetic benchmarks

`bench_parse` runs the five parsers on four families of hand-constructed inputs. Each family targets one cost.

- `small_patterns`: fixed per-call overhead on four fixed cases (a literal, a star against empty input, against one character, against three).
- `scaling_a_star`: how cost grows with input length for `a*`.
- `deep_expression`: how cost grows with expression size for a flat chain `a·a·...·a`.

- `ambiguous_star_a_or_aa`: how cost grows when the expression is genuinely ambiguous, using `(a|aa)*`.

```
cargo bench --bench bench_parse
BENCH_SIZE_LIMIT=200 cargo bench --bench bench_parse
BENCH_SAMPLE_SIZE=50 cargo bench --bench bench_parse
```

Figures for each family to be added. The expected shape: `deriv_bc` and the two `pderiv` variants stay flat or nearly flat; `deriv_std` grows with input length.

Corpus benchmarks

Corpus timing is measured by `bench_corpus_time`, a plain reporting binary rather than a Criterion target. It runs each parser once over each source's admitted corpus, timing every entry individually and reporting the total, mean, median, p99, and maximum per-entry cost. One pass per parser is what makes the measurement feasible: a corpus pass takes seconds per parser on the smoke corpus used below and would take hours on the full one, so Criterion's sample-count model would multiply the runtime for no statistical gain over a single pass.

```
cargo bench --bench bench_corpus_time
BENCH_PATTERN_LIMIT=5   cargo bench --bench bench_corpus_time
BENCH_SOURCE=snort,regexlib cargo bench --bench bench_corpus_time
BENCH_SPLIT_OUTCOME=1   cargo bench --bench bench_corpus_time
```

Setting `BENCH_SPLIT_OUTCOME` prints a second table per source splitting each parser's per-entry durations by the entry's verified outcome (match or non-match), which shows whether a parser's total is skewed by one side of the corpus.

Table 8.7 shows the smoke-test result at `BENCH_PATTERN_LIMIT=5`, which admits five patterns per source — ten entries each under $N = 2$ (five matches, five non-matches). The smoke test is used here because it completes in seconds and is enough to show the qualitative ordering; a full-corpus table would carry the same rows with different absolute numbers.

	Snort (10 entries)		SpamAssassin (10 entries)	
Parser	Total (s)	Mean (ms)	Total (s)	Mean (ms)
`deriv_std_rec`	2.240	224.0	0.001	0.121
`deriv_std_loop`	2.291	229.1	0.001	0.120
`deriv_bc`	0.152	15.2	0.000	0.045
`pderiv_bc`	0.205	20.5	0.000	0.028
`pderiv_std`	0.209	20.9	0.000	0.043

Table 8.7: Corpus timing at `BENCH_PATTERN_LIMIT=5`, one pass per parser per source. Total is wall time over the whole admitted corpus; mean is the per-entry average. Snort carries the long byte-oriented patterns and dominates the absolute figures; SpamAssassin's short keyword patterns complete in microseconds.

Three observations follow from the table. First, the standard derivative variants are an order of magnitude slower than the other three on Snort: 2.24 s and 2.29 s against 0.15–0.21 s for `deriv_bc`, `pderiv_bc`, and `pderiv_std`. Second, the two standard variants are indistinguishable from each other on this corpus, in contrast to the larger gap reported

under the smaller corpus in Section 8.4. Third, SpamAssassin is fast enough across all five parsers that the differences are in the noise of a ten-entry sample; the same ranking holds but the absolute values are small enough that this table is not the place to read a constant factor off it.

8.3.3 Memory

Allocation counters

`bench_memory` replaces the system allocator with a wrapper that counts allocated bytes, and reports three figures per (parser, source) pair:

- **total bytes**: every allocation during the whole corpus pass, including freed and re-allocated bytes. Measures churn.
- **average bytes per entry**: total divided by the entry count. Measures per-pattern cost.
- **max peak**: the highest live-byte high-water mark seen on any single entry. Measures worst-case memory.

Like `bench_corpus_time`, it is a plain reporting binary. It reads the same three corpus environment variables.

```
cargo bench --bench bench_memory
BENCH_PATTERN_LIMIT=5   cargo bench --bench bench_memory
BENCH_SOURCE=snort BENCH_MATCH_INTENT=direct cargo bench --bench bench_memory
```

Table 8.8 reports the same smoke-corpus run that Table 8.7 does. Bytes are given in the units that keep the table readable: MB for Snort, KB for SpamAssassin.

	Snort (10 entries)			SpamAssassin (10 entries)		
Parser	Total	Avg/entry	Max peak	Total	Avg/entry	Max peak
`deriv_std_rec`	887 MB	88.7 MB	168 MB	845 KB	84.6 KB	142 KB
`deriv_std_loop`	887 MB	88.7 MB	168 MB	879 KB	87.9 KB	148 KB
`deriv_bc`	162 MB	16.2 MB	240 KB	834 KB	83.4 KB	9.5 KB
`pderiv_bc`	167 MB	16.7 MB	1.45 MB	289 KB	28.9 KB	7.6 KB
`pderiv_std`	166 MB	16.6 MB	3.89 MB	422 KB	42.2 KB	11.4 KB

Table 8.8: Memory allocation at `BENCH_PATTERN_LIMIT=5`, one pass per parser per source. Total is the sum over the whole corpus pass; avg/entry is per (pattern, input) row; max peak is the worst single-entry high-water mark.

The narrative expectation is confirmed: `deriv_bc`, `pderiv_bc`, and `pderiv_std` allocate substantially fewer bytes than the two standard derivative variants, since `deriv_std` retains its full derivation sequence for the backward pass. On Snort the gap is a factor of about 5× in total bytes and about 700× in peak allocation — 168 MB for `deriv_std` against 240 KB for `deriv_bc`. The standard derivative parsers allocate 88.7 MB per entry on average, against 16.2 MB for `deriv_bc`; the peak figures differ by more than the average because a single Snort pattern dominates the high-water mark.

Section 8.3.3 confirms the same effect on two canonical expressions.

Growth: two canonical expressions

To show *why* memory behaves as measured, three growth demos drive the parsers on two canonical expressions and plot the size of the live state per input character.

The first expression is `a*`, the smallest non-trivial star expression. The second is `(a+(b+ab))*`, which forces every parser to hold multiple residuals simultaneously.

```
cargo run --release --example demo_growth_deriv
cargo run --release --example demo_growth_pderiv
cargo run --release --example demo_growth_eval
cargo run --release --example demo_simp_branches
```

Table 8.9 reports the live-state size at three input lengths per expression. For `deriv_std` and `deriv_bc` the figure is the node count of the current residual. For `pderiv_std` and `pderiv_bc` it is the total node count summed across the whole frontier.

	a^* on a^n			$(a+(b+ab))^*$ on $(ab)^n$		
Parser	n=1	n=8	n=16	n=1	n=8	n=16
`deriv_std`	4	39	79	16	539	9179
`deriv_bc`	2	2	2	12	143	2303
`pderiv_std`	4	4	4	22	160	2560
`pderiv_bc`	2	2	2	18	128	2048

Table 8.9: Live-state size per parser at three input lengths.

Three patterns are visible.

First, on `a*` the `deriv_std` column grows linearly in n, while the other three are bounded. This is the cost of the two-pass structure: the standard parser keeps every intermediate expression alive until the backward pass reconstructs the tree, so its state size is a function of input length. The bit-coded and partial-derivative variants do not carry a history, so their state does not grow.

Second, on `(a+(b+ab))*`, all four grow, but at very different rates. At $n = 16$, `deriv_std` has grown to 9179 nodes, `deriv_bc` to 2303, and the two partial-derivative variants to 2560 and 2048. The gap between the two derivative-based variants is the cost of carrying reconstruction information through each step, which the bit-coded variant does not.

Third, the two partial-derivative variants stay close to each other across both expressions. Their live state is the frontier of residuals, not a single expression; the accumulation is linear in the number of distinct residuals reachable, which is bounded by the pattern's own size.

`demo_simp_branches` shows why simplification never fires its $r+r = r$ rule on `(a+(b+ab))*`. Every step has *distinct* branches by bit-annotation even when the structural shape is identical. This is a property of the bit-coded representation, not a deficiency of the simplifier: the branches genuinely differ, and merging them would lose the reconstruction information that makes single-pass decoding possible.

```
cargo run --release --example demo_simp_branches
```

A figure showing the branch-count series for `(a+(b+ab))*`, using the pgfplots coordinates the demo already prints, can be added here.

Peak memory on the corpus

The max-peak column from Table 8.8 answers whether any corpus pattern causes a peak that would be infeasible on a production machine. On the smoke corpus the answer is no: the highest single-entry peak is 168 MB for the two standard derivative variants on a Snort pattern, well within the memory of a typical workstation. The same table at full corpus size should be checked before drawing a stronger conclusion, since a pattern with a larger language can produce a proportionally larger peak.

8.4 Discussion of Results

8.4.1 Cost of the standard construction

The two standard derivative parsers (`deriv_std_rec` and `deriv_std_loop`) are consistently the slowest and memory-heaviest of the four. This is a direct consequence of the two-pass structure described in Chapter 5: every intermediate expression is stored, so the whole derivation sequence stays alive until the backward pass completes. On `a*` input of length n, the standard parser's state size grows linearly in n, while the bit-coded and partial-derivative parsers stay constant.

On the smoke corpus the two standard variants are nearly identical in both time and memory. This does not mean their underlying implementations are equivalent; it means the pattern shapes in the smoke corpus are short enough that the recursion-versus-iteration difference does not dominate. A larger corpus containing longer and more deeply nested Snort signatures would be expected to show a difference; the smaller corpus used here does not.

8.4.2 Cost of bit-coding

The bit-coded derivative parser (`deriv_bc`) is the second fastest and second most memory-efficient on the smoke run, behind only `pderiv_bc` on most figures and behind `pderiv_std` on one. The trade-off it makes is that the forward pass cannot simplify away redundant branches without losing reconstruction information. The `demo_simp_branches` output shows this clearly: the branch counts grow as expected but the distinctness count stays at 1 after bit stripping, meaning every branch is retained because it carries a different bit annotation. The retention is not wasted: it is exactly what makes single-pass reconstruction possible.

8.4.3 Cost of partial derivatives

The two partial-derivative parsers are the fastest and most memory-light on the smoke run. They split the residual into a set of states rather than combining into one expression, which keeps each state small and avoids the alternation growth that the standard derivative parser suffers. The bit-coded variant (`pderiv_bc`) is consistently faster and lighter than the standard variant (`pderiv_std`), since it replaces the reference-counted injection closures with flat bit-vectors. On Snort the peak allocation is 1.45 MB for `pderiv_bc` against 3.89 MB for `pderiv_std`, and the gap in total bytes is similar.

The price of partial derivatives is that they produce a non-POSIX parse tree rather than

a POSIX one. This is a specification difference, not an implementation deficiency, and it is the reason the two families are compared separately in Section 8.2: within each family, the two variants must agree, but between families they are allowed to differ.

8.4.4 When each parser is appropriate

The four parsers are not interchangeable. Table 8.10 summarises the decision.

Requirement	Parser	Rationale
POSIX + long patterns	`deriv_bc`	Only POSIX variant with bounded state.
POSIX + short/shallow	`deriv_std_loop`	Lowest code complexity; acceptable when input is small.
non-POSIX + any input	`pderiv_bc`	Fastest and most memory-light overall.
Formal verification/teaching	`deriv_std_rec`	Closest to the paper's definition.

Table 8.10: Parser recommendation by requirement.

The frontend, not the parser, is the limiting factor for the *other* column of Table 8.4: patterns that exceed the `MAX_BOUND` cap of 1000 repetitions, or that mix a bound with a nested alternation, are outside the current translation. Raising the cap is a per-pattern trade-off, not a parser choice.

8.4.5 Threats to validity

Three properties of the setup limit how far the results can be generalised.

Corpus scope. The evaluation uses two corpora, both of which are byte- or keyword-oriented and written by practitioners. A corpus of compiler-generated regular expressions, or one drawn from a domain with different syntax conventions, might exhibit different acceptance rates and different timing profiles.

Synthetic families. The synthetic benchmark families are hand-constructed. They isolate specific costs but do not claim to represent the distribution of pattern shapes in the wild. A family whose parameters do not appear in the corpus can still produce informative curves, but it should not be read as a population estimate.

Measurement model. `bench_memory` measures allocation counts, not wall-clock memory pressure. On systems with a garbage collector or a page cache, the figures would interact with those subsystems in ways this evaluation does not model. The evaluator runs on a single machine, and the wall-clock figures in Table 8.7 are therefore CPU- and cache-specific. They are also drawn from a smoke corpus and are not a claim about whole-corpus behaviour; the full-corpus run would produce the same ordering with proportionally larger absolute numbers.

8.4.6 Summary

On both timing and memory, the partial-derivative parsers dominate the derivative parsers, and within each family the bit-coded variant dominates the standard variant. The gap between the two families is large enough that the choice of construction matters more than

the choice of variant. The gap between the two variants within a family is smaller but consistent, and it corresponds directly to the difference between storing a whole derivation sequence and storing a flat bit vector.

All four parsers correctly classify every prepared case in the corpus; no membership or within-family shape bugs were found. Within-family tree agreement was checked on every matching case, and the POSIX-vs-non-POSIX difference, when it appears, is a specification difference rather than an implementation bug.

9 Related Work and Conclusion

9.1 Related Work

9.2 Conclusion

Bibliography

Valentin Antimirov. Partial derivatives of regular expressions and finite automaton constructions. *Theoretical Computer Science*, 155(2):291–319, 1996. doi: 10.1016/0304-3975(95)00182-4.

Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4): 481–494, 1964. doi: 10.1145/321239.321249.

Russ Cox. Regular expression matching can be simple and fast (but is slow in Java, Perl, PHP, Python, Ruby, ...). https://swtch.com/~rsc/regexp/regexp1.html, 2007. Accessed: 2026-08-05.

Martin Sulzmann. Programming paradigms: Regular expressions. https://sulzmann.github.io/ProgrammingParadigms/pp-regular-expressions.html, 2026. Accessed: 2026-08-08.

Martin Sulzmann and Kenny Zhuo Ming Lu. Regular expression sub-matching using partial derivatives. In *PPDP*, pages 79–90, 2012. doi: 10.1145/2370776.2370788.

Martin Sulzmann and Kenny Zhuo Ming Lu. Posix regular expression parsing with derivatives. In *Functional and Logic Programming (FLOPS 2014)*, volume 8475. Springer, 2014.

The Rust Project Developers. Using `Box<T>` to point to data on the heap. The Rust Programming Language, 2026a. URL https://doc.rust-lang.org/book/ch15-01-box.html. Accessed: 2026-09-11.

The Rust Project Developers. Types. The Rust Reference, 2026b. URL https://doc.rust-lang.org/reference/types.html. Accessed: 2026-09-11.